

Inteligencia Artificial aplicada al libre ejercicio profesional

Las catorce guías del webinar, con todas sus pestañas desplegadas, y el glosario de siglas y términos. Pensadas para leer después de la sesión.

Versión del 16/09/2026. En la Guía 11 aparecen enunciados los 47 marcos; cada uno con su ficha, ejemplo, anatomía y taller ocupa por sí solo más que el resto del material junto, así que va en un PDF aparte, «Catálogo de marcos». Los diagramas escritos en Mermaid (Guía 8) aparecen aquí como código; dibujados, en la versión HTML que acompaña a este PDF.

Contenido

1. [Guía 1 — Tu propia IA, desde cero](#)
2. [Guía 2 — Herramientas comerciales: lo que puedes usar mañana](#)
3. [Guía 3 — Lo que hemos construido](#)
4. [Guía 4 — El RAG: tu normativa, respondida con fuente](#)
5. [Guía 5 — El despacho automatizado](#)
6. [Guía 6 — Construir tu IA con la ayuda de una IA comercial](#)
7. [Guía 7 — El ecosistema de vecinos](#)
8. [Guía 8 — Saber y aprender de verdad](#)
9. [Guía 9 — El método](#)
10. [Guía 10 — La caja de herramientas](#)
11. [Guía 11 — Catálogo de marcos](#)
12. [Guía 12 — Formatos y representaciones](#)
13. [Guía 13 — Librerías de Python](#)
14. [Guía 14 — Enlaces y recursos](#)
15. [Glosario — Glosario: siglas y términos](#)

Tu propia IA, desde cero

Qué necesitas de verdad para tener inteligencia artificial funcionando en tu despacho, sin depender de que nadie te la aloje ni de mandar la documentación de tus clientes a un servidor ajeno.

Nivel: desde cero

Lectura: ~20 min

Ingeniería en libre ejercicio

Por qué tener una IA propia

Las herramientas comerciales —ChatGPT, Gemini, Copilot— son excelentes y para muchas cosas son la respuesta correcta. Esta guía no va contra ellas: va de **lo que solo puedes hacer si el modelo corre en tu casa**.

Las tres razones que de verdad importan en un despacho

Razón	Qué significa en tu día a día
Confidencialidad	El proyecto de un cliente, un informe pericial o una medición no salen de tu equipo. No hay términos de servicio que interpretar ni dudas sobre dónde acaba el documento.
Tu conocimiento	Un modelo genérico sabe de todo un poco. El tuyo puede conocer <i>tu</i> normativa, <i>tus</i> plantillas y <i>tus</i> criterios. Ahí está el salto real.
Coste predecible	Pagas el equipo una vez. No hay tarifa por consulta ni sorpresas si un mes trabajas el triple.

LA IDEA QUE SOSTIENE TODO LO DEMÁS

Lo que cambia el ejercicio profesional no es «una IA que escribe textos». Es **una IA que responde sobre tus documentos y te dice de dónde ha sacado la respuesta**. Eso se llama **RAG** (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) y tiene su propia guía; todo lo de aquí es la preparación para llegar ahí.

Qué NO vas a conseguir, dicho al principio

- **No va a firmar por ti.** Ni a asumir responsabilidad. Prepara y localiza; tú decides, mides y firmas.
- **No va a ser más lista que el mejor modelo comercial.** Un modelo de 8B en tu ordenador no es GPT-5. Es suficiente para muchísimo trabajo real, que es distinto.
- **No se monta sola.** La primera tarde se va en instalar y probar. A partir de ahí es cuestión de minutos.

Qué equipo hace falta

La respuesta corta: **menos del que crees para empezar, y más del que crees si quieres los modelos grandes.** La pieza que manda es la **memoria de la tarjeta gráfica (VRAM** (memoria de la tarjeta gráfica)), porque el modelo tiene que caber entero ahí para ir rápido.

Qué se consigue con cada nivel de memoria, y cuánto cuesta

No hay un equipo «correcto»: hay lo que cabe en tu memoria. Esta tabla relaciona cada nivel con los modelos que admite, lo que permite hacer y lo que cuesta hoy en España. **Los precios son de la tarjeta sola** (el ordenador que la aloja va aparte), de mercado, consultados el **14 de septiembre de 2026**, y en este momento están muy por encima del precio recomendado por la escasez de memoria.

Memoria	Ejemplos y precio orientativo	Modelo mayor que cabe (4 bits)	Qué te permite
8 GB de GPU (Graphics Processing Unit, tarjeta gráfica)	RTX 5060 · ~350 €	8B (≈5 GB)	Correo, Excel y VBA (Visual Basic for Applications, macros de Office), borradores, resúmenes
12 GB de GPU	RTX 3060 · desde ~420 € · RTX 5070 · ~710 €	14B (≈9 GB)	Mejor redacción y razonamiento, y un primer RAG sobre normativa
16 GB de GPU	RTX 5060 Ti 16 GB · ~800–860 € · RTX 5070 Ti · ~1.300 € · RTX 5080 · 1.370–1.800 €	24B (≈15 GB, justo)	Modelos de código y visión serios, varios servicios a la vez
24–32 GB de GPU	RTX 3090 usada · 700–1.200 € · RTX 4090 · desde ~3.400 € · RTX 5090 · 4.800–5.600 €	32B (≈20 GB)	Visión de alta calidad, documentos largos con más contexto
128 GB unificados	Mini-PC Ryzen AI Max+ 395 · desde ~4.000 € · NVIDIA DGX Spark · 4.800–6.000 € · Mac Studio M5 Max · desde 3.029 € (la configuración de 128 GB cuesta más: consultar)	70B (≈43 GB) y 120B (≈65 GB)	Modelos de calidad cercana a los grandes comerciales, sin salir de la oficina
96 GB de GPU profesional	RTX PRO 6000 · desde ~12.000 €	70B holgado · 120B	Lo mismo que el anterior, pero muy rápido y con muchos usuarios a la vez

CÓMO LEER LA TABLA

Por euro de memoria, hoy lo más barato es la **memoria unificada** (mini-PC con Ryzen AI Max+, DGX Spark o Mac): te da acceso a modelos de 70B por el precio de una sola tarjeta de gama alta. A cambio, cada respuesta es más lenta que en una GPU dedicada.

Si lo tuyo son muchas consultas pequeñas, GPU; si son pocas y exigentes, memoria unificada.

Lo que cuesta tenerlo encendido, y la alternativa por horas

- **Electricidad:** con el precio medio de agosto de 2026 (0,1665 €/kWh), ocho horas diarias a plena carga cuestan del orden de **0,24 €/día** con una tarjeta de 180 W y **0,77 €/día** con una de 575 W, contando solo la tarjeta. En la práctica pocas veces está a plena carga todo el día.
- **Alquilar GPU por horas** para probar antes de comprar: una RTX 4090 sale por unos **0,34–0,74 \$/hora** y una H100 por unos **2–2,9 \$/hora**.

Lo que tenemos nosotros, y cómo nos hemos adaptado

El sistema que se enseña en este webinar funciona sobre **los equipos que había**, no sobre una compra diseñada para esto: un portátil con 8 GB de GPU, un puesto de trabajo con 12 GB, un servidor con una RTX 5080 de 16 GB y un Mac Studio con 128 GB de memoria unificada. Cada tarea acabó en la máquina que mejor la resolvía: los servicios continuos y el RAG en el servidor, los modelos grandes en el Mac. **Con otros equipos el reparto sería otro, y estaría igual de bien.** El detalle está en la Guía 3.

LA REGLA DE LA VRAM

Un modelo cuantizado a 4 bits ocupa, muy aproximadamente, **la mitad de su número de parámetros en gigabytes**: un 8B $\approx$ 5 GB, un 14B $\approx$ 9 GB, un 24B $\approx$ 14 GB. Si cabe en la VRAM, vuela. Si no cabe, se reparte con la RAM del sistema y se vuelve lento.

Por eso Apple Silicon es un caso aparte: su memoria es *unificada* —la comparten procesador y gráfica—, así que 128 GB permiten modelos de 70B que ninguna tarjeta de consumo puede sostener.

Qué cambia con 128 GB de memoria unificada: el caso del Mac Studio

El último peldaño de la escalera no es «una GPU más grande»: es **otra arquitectura**. En un Mac con chip M2 Ultra y 128 GB, procesador y gráfica comparten la misma memoria, así que

el techo ya no son los 16 GB de una tarjeta, sino casi todo el equipo. Con el framework de Apple para IA (**MLX**) y con **Ollama** (programa gratuito para ejecutar modelos en tu equipo), en un Mac así funcionan hoy estos modelos:

Modelo que corre en el Mac	Tamaño	Para qué sirve en un despacho
Llama 3.3 70B (4 bits, MLX (librería de Apple para ejecutar modelos en la GPU del Mac))	70B	Redacción y razonamiento de calidad cercana a los grandes comerciales, sin salir de la oficina
Qwen 2.5 72B (4 bits, MLX)	72B	Lo mismo, con muy buen español y buen trabajo con documentos largos
Qwen3-VL 32B (Ollama)	32B · 20 GB	Visión: leer planos, fotos de obra, fórmulas escaneadas y tablas sin marco de una norma en PDF
Qwen 3.6 35B (4 bits, MLX , «mixture of experts»)	35B	Rápido para su tamaño: solo activa una parte del modelo en cada respuesta
Whisper large v3 turbo (MLX)	—	Transcripción de reuniones, visitas y notas de voz

LA COMPARACIÓN QUE LO EXPLICA

En el servidor con la RTX 5080 (16 GB) el modelo más grande que se usa es de **24B**. En el Mac corren de **70B y 72B**. No es que el Mac sea «más potente» en todo —para servir muchas consultas a la vez, una **GPU** dedicada sigue siendo mejor—: es que **le cabe más modelo**. Por eso conviven los dos: el servidor para el trabajo continuo y el **RAG**, el Mac para los modelos que no caben en una tarjeta.

Si hoy no quieres comprar nada

- **Empieza en el portátil que ya tienes.** Incluso sin **GPU** dedicada, un modelo de 3B corre en **CPU** (Central Processing Unit, procesador) a ritmo aceptable para redactar y resumir.
- **Alquila por horas** para experimentar y apagar al terminar (precios de referencia más arriba, en «Lo que cuesta tenerlo encendido»).
- **Empieza por las herramientas comerciales** con material público (Guía 2): te enseñan qué tareas merece la pena llevar después a local.

AVISO PRÁCTICO AL COMPRAR

Mira la **VRAM**, no el nombre de la tarjeta. Y desconfía de lo que te diga el propio sistema operativo: hay contadores de Windows que **se quedan en 4 GB** por un límite antiguo del propio contador, y declaran lo mismo para una tarjeta de 12 GB que para una de 4. Compruébalo con la herramienta del fabricante (`nvidia-smi`).

Los modelos: cuál y por qué

Tres palabras que hay que entender

Término	En cristiano
Parámetros (3B, 8B, 70B)	El «tamaño del cerebro». Más grande suele ser mejor y siempre más lento y más pesado.
Cuantización (Q4, Q5, 4bit)	Guardar el modelo con menos precisión para que ocupe menos. Un 14B sin cuantizar no cabe en 12 GB; cuantizado a Q5, sí. Se pierde algo de calidad, casi siempre poco.
Contexto	Cuánto texto puede leer de una vez. Importa mucho si le vas a dar documentos largos.

Familias útiles para trabajo técnico

Para qué	Qué buscar
Uso general en español, razonamiento	Un modelo generalista de 8B–14B cuantizado a Q5
Código, macros, scripts	Un modelo específico de código de 7B–24B
Leer planos, fotos, PDF escaneados	Un modelo <i>de visión</i> (los que llevan «VL» en el nombre) o uno de OCR (Optical Character Recognition: convertir imagen o PDF escaneado en texto)
Buscar en tus documentos (RAG)	Un modelo de embeddings (<i>texto convertido en números que representan su significado</i>), que no escribe: convierte texto en números para poder buscar por significado
Transcribir audio	Un modelo tipo Whisper

CÓMO ELEGIR SIN VOLVERSE LOCO

Empieza con **un** generalista de 8B cuantizado y **un** modelo de embeddings. Con esos dos ya puedes trabajar y montar un RAG. Añade especializados solo cuando tengas una tarea concreta que lo pida.

Un paso más: modelos propios

Cuando llevas tiempo, puedes partir de un modelo abierto y **ajustarlo a tu especialidad** con tu propio material. Deja de ser «un modelo genérico» para hablar tu idioma técnico. No es el primer paso ni el segundo, pero conviene saber que el techo está más arriba de lo que parece.

Montarlo: la primera tarde

- 1 **Instala el motor.** [Ollama](#) es la vía más corta: instalador normal en Windows, macOS y Linux. Al terminar tienes un servicio escuchando en `localhost:11434`.

- 2 **Descarga tu primer modelo** y habla con él:

```
ollama pull qwen3:8b
ollama run qwen3:8b
```

Ya tienes una IA funcionando sin conexión y sin cuenta en ningún sitio.

- 3 **Ponle una cara.** Trabajar en la terminal cansa. **Open WebUI** te da una interfaz de chat como las comerciales, pero contra tus modelos. Se levanta con Docker (empaqueta programas para que funcionen igual en cualquier máquina) y se abre en el navegador.

- 4 **Añade el modelo de embeddings,** que es lo que después permitirá buscar en tus documentos:

```
ollama pull bge-m3
```

- 5 **Prueba con trabajo de verdad.** No con acertijos: coge un correo que tendrías que contestar y una hoja de cálculo que tendrías que limpiar. Ahí se ve si te sirve.

QUÉ TENDRÁS AL ACABAR EL DÍA

Un chat privado contra un modelo local, el motor listo para que otras aplicaciones lo usen por su API (Application Programming Interface: puerta para que un programa hable con otro), y la base montada para el RAG. **Sin cuota mensual y sin que salga un byte de tu equipo.**

Lo que viene después, cuando esto se te quede corto

Pieza	Para qué la querrás
Base de datos vectorial	Guardar tus documentos «por significado» para el <u>RAG</u>
Gestor documental	Digitalizar e indexar expedientes, facturas y certificados
Transcripción	Reuniones con cliente y visitas de obra convertidas en acta
Avisos al móvil	Que te llegue un aviso cuando un proceso largo termina o falla
Registro de lo que preguntas	Saber qué se consultó, qué se respondió y cuánto costó

Buenas prácticas: las pautas que evitan disgustos

Esta es la parte que menos se cuenta y más vale. Son hábitos, no tecnología.

1 · El número lo da el cálculo, nunca el modelo

Un modelo de lenguaje predice texto; puede producir una cifra perfectamente formada y perfectamente falsa. **Que la IA escriba el script o la hoja, y que el número salga de ejecutarlo.** Para un cálculo justificativo esto no es una preferencia: es la diferencia entre un dato trazable y uno inventado.

2 · Toda cita normativa, con su fuente y su edición

«Lo que dice la ISO 16283» no vale. Vale «UNE-EN ISO 16283-1:2015, apartado tal». Si el sistema no te puede decir *de dónde* sale una afirmación, esa afirmación no entra en un documento que vas a firmar.

3 · Verifica con un instrumento distinto

Si la comprobación la hace la misma herramienta que produjo el resultado, no estás comprobando: estás repitiendo. Contrasta contra el PDF oficial, contra otra herramienta o contra un caso cuya respuesta ya conoces.

EL TRUCO DEL CASO CONOCIDO

Antes de fiarte de un flujo nuevo, pásale un caso del que **ya sabes la respuesta**. Si acierta, has calibrado el instrumento. Si falla, lo has descubierto con un caso inofensivo en vez de con el de un cliente.

4 · Declara lo que no has comprobado

En un informe, distinguir «medido» de «estimado» de «no verificado» cuesta una línea y te protege entero. Vale igual para lo que te dé la IA.

5 · Un cero puede ser «no hay» o «no he podido mirar»

Si una búsqueda no encuentra nada, antes de concluir que no existe, comprueba que tu herramienta *habría encontrado* algo que sí está. Es la diferencia entre una ausencia real y un punto ciego.

6 · Privacidad: decide antes, no después

Tipo de trabajo	Dónde conviene hacerlo
Documentación de cliente, periciales, datos personales	Local , sin discusión
Normativa pública, formación, redacción general	Comercial sin problema
Borradores con datos anonimizados	Comercial, si de verdad están anonimizados

LA REGLA DE ORO

La IA prepara y localiza; tú decides, mides y firmas. Todo lo demás de esta guía son detalles de implementación de esa frase.

Guía 1 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Los equipos y cifras citados corresponden a instalaciones reales medidas el 14/09/2026.

[¿Una sigla que no conoces? Consulta el glosario](#)

Herramientas comerciales: lo que puedes usar mañana

Antes de montar nada propio, las herramientas de Google, Microsoft, OpenAI, Anthropic y Perplexity resuelven mucho trabajo de un despacho. Qué hace cada una, cuánto cuesta, ejemplos concretos de uso y dónde está el límite con los datos de tus clientes.

Nivel: desde cero Lectura: ~25 min Precios consultados el 14/09/2026

Panorama: cada herramienta tiene su terreno

No compiten todas por lo mismo. Conviene elegir por la **tarea**, no por la marca.

Herramienta	Donde brilla	¿Gratis?	De pago, desde
Gemini Notebook (antes NotebookLM)	Preguntar a <i>tus</i> documentos con cita a la fuente	Sí, con límites	incluido en Google AI
Gemini	Asistente general y trabajo dentro de Gmail, Docs y Sheets	Sí	4,99 €/mes
Copilot / Microsoft 365	Word, Excel, PowerPoint y Outlook, donde ya trabajas	Sí, limitado	10 €/mes (particular)
ChatGPT	Generalista, análisis de datos, redacción	Sí	7,99 €/mes
Claude	Documentos largos, revisión detallada, código	Sí	17–20 \$/mes
Perplexity	Buscar en internet con fuentes citadas	Sí, con límites diarios	20 \$/mes

LA REGLA QUE VA ANTES QUE CUALQUIER HERRAMIENTA

Con datos de clientes, planes de empresa. Con planes personales, solo material público —normativa, formación, redacción general— y con el uso de tus datos para entrenamiento desactivado. El detalle, en la pestaña 6.

CÓMO LEER LOS PRECIOS

Son los publicados o de referencia en España el **14 de septiembre de 2026**. Algunos se muestran sin IVA (planes de empresa) y los de Claude y Perplexity solo están en dólares. **Cambian a menudo: confírmalos en la web oficial antes de contratar.**

Google: Gemini Notebook, Gemini y Workspace

Gemini Notebook (antes NotebookLM)

Google lo renombró en julio de 2026 manteniéndolo como producto independiente. Su principio es el que más interesa a un ingeniero: **solo responde con las fuentes que tú le cargas**, y cita el pasaje del que sale cada afirmación. Es, en la práctica, un RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) ya montado.

	Gratis	Con Google AI Pro
Cuadernos	100	500
Fuentes por cuaderno	50	300
Consultas al día	50	500
Resúmenes en audio al día	3	más

Ejemplo · ICT: carga el RD 346/2011 y su reglamento técnico en un cuaderno y pregunta «¿qué exige para el recinto de instalaciones de telecomunicación superior?». Obtienes la respuesta con el pasaje señalado. Añade al cuaderno el Reglamento (UE) 2024/1309: su artículo 10.1 obliga a que los edificios nuevos o con reforma importante cuyo permiso de construcción se solicite después del 12/02/2026 lleven infraestructura adaptada a la fibra y cableado de fibra hasta el punto de conexión del usuario.

Ejemplo · ordenanzas: un cuaderno por municipio donde trabajas, con su ordenanza de ruido o de actividades. Deja de haber que buscar el artículo cada vez.

Ejemplo · formación: convierte una norma nueva en un resumen en audio para escucharla en el coche de camino a obra.

Gemini (app y Google AI)

Plan	Precio
Gratis	0 €, con límites
Google AI Plus	4,99 €/mes
Google AI Pro	21,99 €/mes
Google AI Ultra	desde 99,99 €/mes (<i>varían las fuentes: confirmar</i>)

Gemini dentro de Google Workspace

Para quien trabaja con Gmail y Google Drive como empresa. Por usuario y mes, **sin IVA:** Business Starter **6,80 €** · Standard **13,60 €** · Plus **21,10 €** (el pago anual ahorra en torno a un 16 %).

Ejemplo · correo: redactar la respuesta a un requerimiento de la administración a partir del hilo de Gmail.

Ejemplo · hojas: limpiar y ordenar una tabla de presupuesto en Sheets.

Microsoft: Copilot en las herramientas que ya usas

Su gran ventaja es que vive **dentro de Word, Excel, PowerPoint y Outlook**, que es donde está el trabajo de la mayoría de los despachos.

Para profesionales individuales

Plan	Mensual	Anual
Copilot gratis	0 €, «sujeto a capacidad y límites»	
Microsoft 365 Personal	10 €	99 €
Microsoft 365 Familia	13 €	129 €
Microsoft 365 Premium	22 €	219 €

Para empresa (con protección de datos corporativa)

Plan	Precio por usuario y mes, sin IVA
Complemento Microsoft 365 Copilot	15,60 € en oferta (18,20 € normal) en pago anual · 21,84 € en pago mensual
Empresa Estándar + Copilot	20,36 € (anual)
Empresa Premium + Copilot	27,73 € (anual)

Ejemplo · Word: «redacta el apartado de antecedentes de la memoria a partir de estas notas de visita» y luego corriges tú.

Ejemplo · Excel: «analiza este registro de mediciones y dime qué puntos superan el límite y en qué franja horaria».

Ejemplo · PowerPoint: convertir el informe final en una presentación para la reunión con el cliente.

Ejemplo · Outlook: resumir un hilo largo de correos de obra y listar lo pendiente de cada parte.

ChatGPT y Claude: los asistentes generalistas

ChatGPT (OpenAI)

Plan	Precio orientativo en España
Free	0 €
Go	7,99 €/mes
Plus	22,99 €/mes
Pro	desde 102,99 €/mes
Business	en torno a 21 € por usuario y mes en pago anual (mínimo 2 usuarios) (<i>confirmar</i>)

Ejemplo · pliegos: revisar la coherencia de un pliego de prescripciones técnicas y señalar contradicciones entre apartados.

Ejemplo · datos: subir un CSV de mediciones y pedir un gráfico y un resumen estadístico, generado con código que puedes revisar.

Claude (Anthropic)

Plan	Precio (en dólares, sin impuestos)
Free	0 \$
Pro	17 \$/mes en pago anual · 20 \$ mensual
Max	desde 100 \$/mes
Team	20–25 \$ por usuario y mes (estándar)

Ejemplo · revisión: pasarle un proyecto completo y una lista de comprobación, y pedirle qué puntos no están resueltos y en qué página debería estar cada uno.

Ejemplo · herramientas: escribir un script de cálculo o una macro de Excel, explicando qué debe hacer; y probarlo tú con un caso conocido.

Ejemplo · expediente: un «proyecto» con la documentación de un encargo, para consultarlo durante semanas sin volver a subir nada.

CUÁL ELEGIR ENTRE LOS DOS

Para la mayoría de las tareas, **los dos sirven**. Prueba la misma tarea real en ambos durante una semana con el plan gratuito y quédate con el que mejor encaje con tu forma de trabajar. Pagar dos a la vez rara vez compensa.

Perplexity: buscar con fuentes

Es un buscador conversacional: en lugar de una lista de enlaces, te da una respuesta **con las fuentes citadas** para que las compruebes.

Plan	Qué incluye	Precio
Free	3 búsquedas avanzadas y 5 investigaciones profundas al día	0
Pro	uso intensivo	20 \$/mes (≈20 € con IVA)
Max	el máximo	200 \$/mes

Ejemplo · vigilancia normativa: «¿ha habido cambios este año en la normativa de ICT (Infraestructura Común de Telecomunicaciones (RD 346/2011))?», y abrir las fuentes que devuelve.

Ejemplo · productos: comparar las fichas técnicas de tres fabricantes de un mismo material o equipo.

Ejemplo · estado del arte: una investigación profunda antes de un informe sobre una tecnología que no dominas.

UNA CITA NO ES UNA VERIFICACIÓN

Que venga con enlace no significa que el enlace diga eso. Para cualquier cosa que vaya a un documento que firmas, **abre la fuente y compruébalo**, y prefiere siempre el BOE, el boletín autonómico o la norma UNE a un artículo que la resuma.

Tus datos: quién puede usarlos para entrenar

Es la pregunta que decide qué puedes subir a cada herramienta. La respuesta cambia según el plan, y en casi todos los personales depende de un ajuste que conviene revisar.

Herramienta	Planes personales	Planes de empresa
Gemini Notebook	No entrena con tus fuentes, salvo que valores una respuesta (pulgar arriba o abajo): entonces esa conversación puede revisarla una persona	Sin revisión humana ni entrenamiento
App Gemini	Con el historial de actividad activado, puede haber revisión y entrenamiento	Workspace tiene condiciones propias
Copilot	En el Espacio Económico Europeo no se entrena por defecto; se puede comprobar en la configuración	Microsoft 365 Copilot no entrena con los datos de tu organización
ChatGPT	Ajuste «mejorar el modelo para todos», que se puede desactivar	Business y Enterprise no entrenan por defecto
Claude	Solo entrena si el ajuste está activado	Team, Enterprise y <u>API</u> (Application Programming Interface: puerta para que un programa hable con otro), excluidos
Perplexity	Retención activada por defecto, desactivable	Enterprise no entrena

TRES REGLAS PRÁCTICAS

1. **Documentación de cliente, periciales o datos personales:** plan de empresa o IA local. Nunca un plan personal.
2. **Normativa y material público:** cualquier plan, con el entrenamiento desactivado.
3. **Revisa el ajuste cada cierto tiempo:** las condiciones cambian y los valores por defecto también.

¿Comercial o propia? Casi siempre, las dos

	Comercial	IA propia
Arranque	Inmediato	Una tarde para lo básico; semanas para un <u>RAG</u> serio
Calidad bruta del modelo	La mejor disponible	Menor, salvo con mucha memoria
Tu normativa y tus plantillas	Hasta donde permita subir documentos	Sin límite y con control de vigencia
Confidencialidad	Depende del plan	Total: nada sale del despacho
Coste	Cuota fija por usuario y mes	Inversión inicial + electricidad

Una cuenta orientativa

Un único usuario con el complemento de Microsoft 365 Copilot en pago anual paga del orden de **190–220 € al año sin IVA**. Una tarjeta de 16 GB de gama media cuesta hoy en torno a **800 €** (solo la tarjeta) y permite modelos de hasta unos 24B en local. Para una sola persona las cifras se parecen a varios años vista; lo que inclina la balanza **no es el precio**, sino qué datos manejas y si necesitas tu propia normativa con vigencia. *(Cálculo orientativo con los precios de esta guía; no incluye el equipo que aloja la tarjeta ni la electricidad.)*

EL MODELO QUE MEJOR FUNCIONA

Comercial para lo público y lo general; propia para lo confidencial y lo tuyo. Y una herramienta comercial de pago es, además, la mejor ayuda para construir la propia: es lo que cuenta la Guía 6.

Guía 2 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Precios y condiciones consultados el 14/09/2026 en las webs de los fabricantes y en referencias de precios para España. Cambian con frecuencia: confírmalos antes de contratar.

¿Una sigla que no conoces? [Consulta el glosario](#)

Lo que hemos construido

Un recorrido por un sistema real de IA propia en un despacho de ingeniería: qué hace cada pieza, con qué está hecha y qué se puede aprender de ella para montar la tuya. No es una receta de compra: es lo que tenemos, y cómo lo hemos adaptado a ello.

Casos reales

Lectura: ~30 min

Inventario verificado el 14/09/2026

El mapa: cuatro máquinas, cada una con su papel

Este sistema no se diseñó de golpe. Se fue construyendo sobre los equipos que había, y cada tarea acabó en la máquina que mejor la resolvía. **Esa es la primera lección: adapta el reparto a lo que tienes, no al revés.**

Máquina	Qué es	Papel en el sistema
Portátil	Windows 11 · i7-14700HX · 16 GB · RTX 4060 Laptop 8 GB	Trabajo de campo y movilidad
Puesto de trabajo	Windows 11 · i9-10900 · 64 GB · RTX 3060 12 GB	El día a día: aplicaciones de ingeniería, Office, QGIS, generación de proyectos
Servidor	Ubuntu 26.04 · 122 GB · RTX 5080 16 GB	Servicios continuos: modelos locales, <u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos), búsqueda, documentación, transcripción
Estación Apple	Mac Studio M2 Ultra · 128 GB memoria unificada	Modelos grandes (70B), imagen, voz y realidad virtual

POR QUÉ DOS MOTORES DE IA DISTINTOS

El servidor tiene una GPU (Graphics Processing Unit, tarjeta gráfica) de 16 GB: rápida, buena para atender muchas consultas, pero con techo de memoria. El Mac tiene 128 GB compartidos entre procesador y gráfica: más lento por consulta, pero le caben modelos que en la GPU no entran. **Cada uno hace lo que su memoria le permite.** Con otra combinación de equipos, el reparto sería otro.

Cómo se hablan entre sí

- **Carpetas compartidas en red** para que los documentos estén en un solo sitio.
- **Servicios con API:** cada modelo o herramienta escucha en un puerto y cualquier otra aplicación puede usarlo.
- **Varios asistentes de IA trabajando en paralelo**, uno por máquina, que se coordinan por escrito (se cuenta en la Guía 6).

Nuestros modelos: los de siempre y los propios

Un sistema de IA propio no es un modelo: es un **conjunto de modelos**, cada uno para lo suyo, y una forma de elegir cuál responde.

En el servidor: 16 modelos servidos con Ollama

Tipo	Ejemplos	Para qué
Generalistas cuantizados	Qwen3 8B y 14B (Q5), GPT-OSS 20B, Phi-4 reasoning 14B	Redactar, resumir, razonar
Código	Devstral 24B, Qwen2.5-Coder 7B	Macros, scripts, automatizaciones
Visión y OCR	Qwen3-VL 8B, Qwen2.5-VL 7B, DeepSeek-OCR 3B	Leer planos, fotos y documentos escaneados
Embeddings y reordenación	bge-m3, Qwen3-Embedding, mxbai-rerank	Buscar por significado (el RAG)
Propio	nmr-acustica:v1 (8,1 GB)	Modelo especializado en el dominio de la acústica

En el Mac: modelos grandes siempre disponibles

Dos servidores **compatibles con la API de OpenAI** arrancan solos con el equipo, así que cualquier aplicación que sepa hablar con ChatGPT puede hablar con ellos cambiando solo la dirección:

- **Llama 3.3 70B** a 4 bits, ocupando unos 38 GB de memoria.
- **Qwen3 8B** a 4 bits, para respuestas rápidas.
- Y, bajo demanda, Qwen2.5 72B, Qwen3.6 35B, bge-m3 y un modelo de visión ligero.

Además hay **tres variantes de Qwen3 8B afinadas con material propio**. Un uso concreto del 70B: redactar la parte narrativa de un informe a partir de los datos ya calculados.

AFINAR UN MODELO NO ES LO MISMO QUE DARLE UN RAG

Afinar cambia cómo habla el modelo: su vocabulario, su estilo, su forma de estructurar. **El RAG** le da datos con fuente en el momento de responder. Para normativa y cifras, lo que manda es el **RAG**; afinar ayuda a que el texto «suene» a tu especialidad.

Quién elige el modelo

Con tantos modelos, alguien tiene que decidir cuál responde cada pregunta. Aquí lo hace un **enrutador** (SmarterRouter): mira la consulta y la manda al modelo adecuado, guarda en caché respuestas repetidas y libera la **GPU** cuando se llena. Y el espacio de trabajo principal (Odysseus) tiene datos de alta, como si fueran un solo sistema, los modelos del servidor y los del Mac.

Ahorrar **tokens**: pagar solo por el juicio

Los asistentes de pago cobran por lo que leen y escriben. La idea que atraviesa todo este sistema es muy simple: **lo que una máquina puede hacer gratis, que lo haga una máquina local; el modelo de pago se reserva para donde hace falta criterio**.

Buscar en internet sin gastar **tokens**

Cuando un asistente de pago navega por su cuenta, lee páginas enteras y paga cada palabra. Aquí la navegación la hace el sistema propio y al asistente solo le llega un resumen con citas.

Pieza	Qué hace
SearXNG	Metabuscador instalado en casa: pregunta a varios buscadores públicos a la vez y junta los resultados, sin cuenta ni cuota. <i>Estado a 15/09/2026: los buscadores públicos bloquean o limitan las consultas automáticas, así que hoy no da resultados fiables; en producción hace falta una fuente estable, como una API de búsqueda.</i>
Odysseus	Espacio de trabajo de IA autoalojado: chat, agente, investigación profunda, documentos y memoria. Usa los modelos locales para leer y resumir.
Conector para el asistente	Dos herramientas: <i>buscar</i> (título, dirección y fragmento, con un tope de 4.000 caracteres) e <i>investigar</i> (un informe de hasta 6.000 caracteres). Con reglas incorporadas: citar siempre la fuente y dar prioridad a BOE, UNE y boletines autonómicos.

EL AHORRO, EN UNA FRASE

Las páginas las lee un modelo local, que no cuesta nada por consulta. El asistente de pago solo paga por **pedir la búsqueda y leer el resultado compacto**.

Preparar la normativa sin gastar tokens

El RAG normativo (Guía 4) necesita convertir normas en PDF en fragmentos fiables. El proceso se organizó en **tres vías locales** —extracción mecánica, texto con modelo local, y modelo de visión local para fórmulas e imágenes—, y solo la **verificación final** se hace con el modelo de pago.

- Una detección sin modelo de las páginas de texto a varias columnas las saca del modelo de visión, que es lo más caro de procesar.
- Una medida concreta: solo el detector de páginas escaneadas evita del orden de **35.000 tokens**.
- Y una vez preparado, **consultar el RAG cuesta 0 €**: todo corre en local.

Herramientas de ingeniería hechas en Python

Aquí está buena parte del valor diario: aplicaciones pequeñas que hacen en segundos lo que antes llevaba una tarde. **Este tipo de herramienta es exactamente lo que hoy se construye trabajando con un asistente de IA** (Guía 6).

Suite de herramientas NMR en uso

Aplicación de escritorio para Windows con **18 herramientas**.

Acústica: cálculo por lotes de EDT, T20, T30, C50, C80, D50, Ts y STI a partir de respuestas al impulso; visores de ARTA y CATT-Acoustic; **calibración del modelo CATT contra lo medido**; optimizador que busca los materiales para un T30 objetivo.

Documentos: datos del sonómetro a Excel, Word/Excel/PDF a Markdown, metadatos, extraer y analizar VBA (Visual Basic for Applications, macros de Office), renombrado por lotes, duplicados.

Python 3.11 · tkinter · numpy · scipy · matplotlib · PyMuPDF · ejecutable único

Aplicación de proyectos ICT en uso

Guía el proyecto conforme al **RD 346/2011**: datos del proyecto → edificio → infraestructura → servicios.

Genera los **planos DWG**, los **cálculos** en Excel y la **memoria** en Word, con un catálogo de 488 productos de fabricante y pruebas automáticas.

Python 3.11 · customtkinter · openpyxl · ezdxf · ODA File Converter

QGIS: propagación acústica

experimental

Complemento propio: se marcan en el mapa fuentes, receptores y ruido de fondo; calcula distancias en 3D y el nivel teórico, y lanza el cálculo de propagación en el servidor.

Junto a él: descarga de cartografía del Catastro y un conector para que el asistente de IA maneje QGIS.

PyQGIS · GeoPackage · NoiseModelling

Librerías y scripts en uso

Evaluador normativo: con un valor medido o calculado indica *cumple* / *no cumple*, con el margen en dB y el artículo aplicable.

Lectura de bases de precios **.bc3** (CYPE, Presto, BEDEC), predicción de aislamiento, psicoacústica, geometría descriptiva, e importación automática de materiales en software de acústica.

Python · pyva · mosquito · librerías propias arta_pir, catt_parser, iso3382

UN DETALLE DE OFICIO QUE VALE PARA CUALQUIER GENERADOR DE PLANOS

El generador de DWG (formatos de dibujo técnico CAD) **no reescribe el dibujo**: sustituye los textos de las cartelas sobre una plantilla y convierte el resultado. Así no se rompen las tildes ni se altera nada del plano original. Cuando automatices documentos técnicos, **parte de una plantilla buena y toca lo mínimo**.

Imagen, voz y realidad virtual

La memoria unificada del Mac abre un terreno que en una GPU de 16 GB no cabe: modelos de imagen y de voz grandes funcionando en local.

Editor de imágenes por instrucción

funcionando

Le dices con palabras qué cambiar en una foto —«quita el coche del fondo»— y lo hace. Es una versión local de la idea de «Nano Banana» de Google, montada con **modelos abiertos**: el servidor hace un borrador rápido y el Mac la versión de calidad.

Qwen-Image-Edit 2511 a 4 bits (18,3 GB, Apache 2.0) · FLUX.2 klein 4B · MLX (librería de Apple para ejecutar modelos en la GPU del Mac) · aplicación web

Laboratorio de voz en desarrollo

A partir de una grabación clona una voz, la transcribe y mide rasgo a rasgo cuánto se parece a la original (timbre, entonación, acento), con detección de voz sintética.

Chatterbox (MIT) · FreeVC · OpenVoice · RVC · faster-whisper · métricas ECAPA y NISQA

Motor acústico en tiempo real

funcionando

Calcula al vuelo la acústica de una sala — RT60, STI, aislamiento, nivel sonoro— con el micrófono calibrado, y la sirve a otras aplicaciones.

Python · servidor WebSocket

Instrumentos de medida en realidad virtual en desarrollo

Cliente para **Meta Quest 3**: sonómetro, analizador de espectro, ISO 3382 y respuesta al impulso flotando en el espacio, alimentados por el motor acústico. Con batería de pruebas automáticas.

WebXR · Three.js · navegador de las gafas

LICENCIAS: MIRADLAS ANTES DE TRABAJAR PARA UN CLIENTE

En el laboratorio de voz, uno de los motores disponibles **está excluido de los trabajos de cliente** porque su licencia no permite uso comercial. El editor de imágenes usa uno con licencia Apache 2.0 precisamente por eso. **Que un modelo sea descargable no significa que puedas usarlo en un encargo.**

El despacho: servicios que ahorran horas

No todo es IA en sentido estricto. Estos servicios corren en el servidor, se instalan con Docker (empaqueta programas para que funcionen igual en cualquier máquina) y son de código

abierto. Juntos cubren buena parte de la trastienda de un despacho.

Servicio	Qué resuelve
Paperless-ngx	Archivo documental: digitaliza con <u>OCR</u> (Optical Character Recognition: convertir imagen o PDF escaneado en texto), indexa y busca expedientes, facturas y certificados
Scriberr	Transcripción de audio: reuniones con cliente, visitas de obra, notas de voz
Docmost	Wiki interna: procedimientos, plantillas y conocimiento del despacho
Open WebUI	El «ChatGPT propio»: chat contra los modelos locales
Qdrant	La base de datos donde vive el <u>RAG</u>
ntfy	Avisos al móvil cuando un proceso largo termina o algo falla
Langfuse	Registro de qué se preguntó, qué se respondió y cuánto costó

POR DÓNDE EMPEZAR SI SOLO QUIERES UNO

Paperless-ngx. Resuelve un problema que todo despacho tiene —encontrar un documento—, no exige GPU y, cuando más adelante montes un RAG, ya tendrás tus documentos digitalizados y ordenados.

Pautas que salen de construirlo

1. **Reparte según la memoria de cada equipo.** No hay una arquitectura correcta: hay la que encaja con lo que tienes.
2. **Paga solo el juicio.** Buscar, extraer, convertir y comparar lo hace una máquina local. El modelo de pago, donde hace falta criterio.
3. **Cada servicio, con su dirección y con arranque automático.** Y con una forma sencilla de preguntarle si está vivo.
4. **Compatible con la API de OpenAI siempre que se pueda.** Así cambias de modelo, local o de pago, sin tocar tus aplicaciones.
5. **Revisa la licencia antes del primer encargo** con cada modelo nuevo.
6. **Voz e imagen de personas, con consentimiento.** Clonar una voz o editar la foto de alguien es una cuestión legal, no solo técnica.

7. **Anonimiza antes de enseñar.** Las plantillas de trabajo real suelen llevar datos de clientes y firmas: límpialas antes de compartirlas o proyectarlas.
8. **Di en qué estado está cada cosa.** «Funcionando», «experimental» o «en desarrollo» no restan: evitan que alguien confíe en lo que aún no está listo.
9. **Mide antes de fiarte** (Guía 4): lo que parece que funciona y lo que funciona se separan con un caso cuya respuesta ya conoces.

Guía 3 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

[¿Una sigla que no conoces? Consulta el glosario](#)

El RAG: tu normativa, respondida con fuente

La pieza que convierte una IA que «sabe cosas» en una herramienta de trabajo profesional: responde sobre *tus* documentos y te dice de dónde ha sacado cada afirmación.

Nivel: tras la Guía 1

Lectura: ~25 min

El bloque que más cambia el día a día

Qué es un RAG y por qué importa en ingeniería

RAG significa *Retrieval-Augmented Generation*: generación aumentada por recuperación.

Dicho en cristiano: **antes de contestar, el sistema busca en tus documentos los pasajes que tratan de lo que preguntas, y contesta a partir de ellos** en lugar de tirar de su memoria.

Sin <u>RAG</u>	Con <u>RAG</u>
El modelo responde con lo que «recuerda» de su entrenamiento.	El modelo responde con los pasajes que ha encontrado en tu corpus.
No sabe qué edición de una norma está en vigor.	Puede saberlo, si el corpus lo registra.
Si no sabe, a veces inventa con total aplomo.	Si no encuentra fuente, puede —y debe— decir que no la tiene.
No puede decirte de dónde sale una cifra.	Te da la norma, el apartado y el fragmento.

POR QUÉ ES EL CASO PERFECTO PARA UN INGENIERO

Buena parte del trabajo técnico consiste en **encontrar el precepto que aplica y citarlo bien**: el reglamento de ICT (Infraestructura Común de Telecomunicaciones (RD 346/2011)), el código técnico, la normativa autonómica, la norma UNE de ensayo. Es un trabajo de búsqueda con consecuencias legales, y es justo donde un RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) bien hecho rinde más y donde uno mal hecho es más peligroso.

Un ejemplo real en funcionamiento

Estas son las cifras de un RAG normativo en producción en un despacho de ingeniería, medidas el **14 de septiembre de 2026**:

188

normas en el corpus
normativo servido

10.574

fragmentos normativos
consultables

17.253

fragmentos en total
(normativa, manuales, ICT,
materiales...)

0 €

coste por consulta: todo corre
en local

Normativa acústica española y europea, materiales del Código Técnico, manuales de los equipos del laboratorio, normativa ICT y el conocimiento propio del despacho. Todo en una máquina propia, sin que ninguna consulta salga de la oficina.

LA PREGUNTA QUE SEPARA UN RAG ÚTIL DE UNO DE DEMOSTRACIÓN

No es «¿qué modelo usa?». Es «**¿cómo sabe que lo que te devuelve sigue en vigor?**» y «**¿cómo distingue el texto de la norma de un comentario sobre la norma?**». Todo lo que sigue en esta guía responde a esas dos preguntas.

Cómo funciona: el recorrido de una pregunta

Cuando preguntas algo, la consulta pasa por siete etapas. No hace falta saber programarlas para entender qué hace cada una, y entenderlo te permite saber qué pedir y qué desconfiar.



Las etapas 1 a 4 son las de cualquier RAG. **Las 5 y 6 son las que lo hacen utilizable para normativa**, y son las que casi nunca traen las soluciones de manual.

Las piezas, sin tecnicismos

Pieza	Qué hace	Ejemplo que funciona
Modelo de embeddings	Traduce texto a vectores de significado	<code>bge-m3</code> , multilingüe, bueno en español
Base vectorial	Guarda los fragmentos y busca por parecido	Qdrant
Reordenador (reranker)	Afina el orden de los resultados	<code>bge-reranker-v2-m3</code>
Motor	Orquesta las siete etapas	un script propio
Puerta de entrada	Por dónde preguntas	el propio asistente de IA, o una interfaz de chat

CINCO DECISIONES DE DISEÑO QUE MERECE LA PENA COPIAR

1. **Todo local.** Ninguna consulta sale de tu red y el coste de preguntar es cero.
2. **Determinista.** La misma pregunta devuelve el mismo resultado. En normativa, que sea reproducible vale más que que sea «listo»: es lo que te permite auditarlo.
3. **La vigencia es una capa, no un campo.** Las normas derogadas no se esconden — la consulta histórica es legítima—: se muestran marcadas.
4. **Lo que no es norma, se rotula.** Para que nadie cite una nota del editor como si fuera la ley.
5. **Vive en el ambiente.** No hay que ir a buscarlo: el asistente lo consulta antes de afirmar cualquier dato normativo.

UNA BUENA PRÁCTICA DE INGENIERÍA

Si una pieza puede fallar de forma brusca —el reordenador, por ejemplo—, **áislala en su propio proceso**. Si cae, el sistema no se cae: esa consulta se resuelve con el orden de la búsqueda básica y la siguiente vuelve a funcionar. El sistema se degrada, no se rompe.

Tipos de RAG y qué aporta cada uno

Cuando empiezas a leer sobre el tema aparecen muchos apellidos: híbrido, correctivo, de grafo, agéntico... No son productos rivales entre los que haya que elegir uno. **Casi todos son piezas que se añaden encima de un RAG básico** para resolver un problema concreto. La pregunta útil no es «¿cuál es el mejor?», sino «**¿qué problema tengo yo, y qué pieza lo resuelve?**».

La última columna dice qué hay de cada tipo en el sistema de ejemplo de esta guía, tal como estaba medido el 15/09/2026. Incluye también lo que *no* tiene, y por qué.

Tipo	Qué aporta	Cuándo compensa	En nuestro sist
RAG básico búsqueda por parecido	Busca los fragmentos cuyo <i>significado</i> se parece más a tu pregunta y responde a partir de ellos. <i>Ejemplo:</i> preguntas «¿qué aislamiento hace falta entre dos viviendas?» y encuentra el apartado aunque la norma no use ninguna de tus palabras.	Siempre. Es el punto de partida y lo que traen las interfaces de chat con documentos.	Sí. Búsqueda v <i>embeddings</i> b Qdrant, con filt metadatos: juri rango y «solo v
Con reordenador dos etapas	Una primera búsqueda rápida trae muchos candidatos. Después, un segundo modelo, más lento pero más fino, lee <i>pregunta y fragmento juntos</i> y los reordena. <i>Ejemplo:</i> entre veinte fragmentos que mencionan las tomas de telecomunicación, sube arriba el que fija cuántas hacen falta por estancia, no el que solo las nombra.	Cuando el fragmento bueno aparece entre los candidatos, pero no en los primeros puestos. Cuesta tiempo por consulta; con muchos candidatos, una tarjeta gráfica lo hace llevadero.	Sí. bge-reran aislado en su p si falla, la consu orden de la bú: y un aviso.
Híbrido significado + palabras clave	Combina la búsqueda por significado con una búsqueda clásica por palabras exactas (BM25 (Best Matching 25: búsqueda clásica por palabras clave)). <i>Ejemplo:</i> un código como «DnT,w» o una referencia como «UNE-EN ISO 717-1» no tienen un «significado» que el modelo capte bien; una búsqueda por palabras los encuentra a la primera.	En corpus con muchos códigos, símbolos y referencias exactas. Mídelo antes de adoptarlo: puede subir fragmentos que desplazan al que traía la cifra (pestaña «Medir si funciona»).	No en producc el 15/09/2026: guarda el vecto significado, sin clave). Se ha es mejora para có del tipo «DnT,v
CRAG (Corrective RAG: comprueba lo recuperado antes de responder)	Antes de responder, evalúa lo que ha recuperado y actúa si no es bueno. En la versión publicada, un modelo pequeño puntúa los documentos y, si no sirven, el sistema busca en otra parte (incluso en internet). <i>Ejemplo:</i>	Cuando citar mal tiene consecuencias: normativa, informes firmados, dictámenes. Con prudencia: un evaluador genérico puede puntuar mal en	Sí, en variante (reglas, no un r juzgue): · <i>de vigencia:</i> m fragmento corr sustituida, peno derogada; si es

recupera un artículo de un reglamento ya sustituido y, en lugar de citarlo como si estuviera en vigor, lo marca y nombra el vigente.	un dominio muy especializado, y salir a internet no es buena idea si tu corpus es la fuente autorizada.	sustituida y la s vigente está en avisa «vigente para histórico» nada. · <i>de atribución</i> : texto no es la v norma (nota ec recomendaciór marca de «pen verificar», docu interna). Además, filtro t estaba vigente dada) y anotac jerarquía norm «de libro» usa l evaluador y pu fuera; el nustr propósito, repr
--	---	--

GraphRAG RAG sobre un grafo	Además de fragmentos de texto, organiza el conocimiento como entidades y relaciones y recupera siguiendo esas relaciones. <i>Ejemplo</i> : «¿qué normas remiten a la UNE-EN ISO 717-1, y cuáles de ellas siguen vigentes?» es una pregunta sobre relaciones entre documentos, no sobre el contenido de un párrafo.	Cuando las preguntas importantes cruzan muchos documentos o dependen de relaciones. Construir el grafo completo es caro, así que conviene tener claro que se necesita.	No en el RAG Hay aparte un q hechos del equ (entidades y re hace qué, qué i depende de qu consulta por se es GraphRAG. E la variante Lazy Microsoft, cuar versión abierta
RAG agéntico	Un asistente (un «agente») decide cuándo buscar, dónde y cuántas veces , y vuelve a buscar si lo encontrado no basta. <i>Ejemplo</i> : para redactar un informe, consulta la norma, ve que remite a otra, consulta esa, y después revisa el expediente del cliente.	En tareas de varios pasos. A cambio, el recorrido cambia de una vez a otra y es más difícil de auditar que una búsqueda fija.	El motor de re no tiene capa una decisión de que sea deterr práctica, los asi trabajan con él cuándo consult grafo de hecho

los ficheros, va
tarea.

RAFT (Retrieval-Augmented Fine-Tuning: afinar un modelo para usar mejor lo recuperado)

No es una forma de buscar, sino de **entrenar al modelo que responde**. Cada ejemplo de entrenamiento lleva el documento con la respuesta mezclado con documentos que no la tienen, y el modelo aprende a apoyarse en el bueno e ignorar el resto. Una variante añade ejemplos donde ningún documento tiene la respuesta, para enseñarle a decir «esto no está en el contexto». *Ejemplo:* un modelo local pequeño que, ante tres fragmentos de ordenanzas, cita el que corresponde y dice «no consta» si ninguno lo contiene.

Cuando ya tienes un RAG que funciona y quieres que un modelo local use mejor lo que se le entrega. Complementa al RAG; no lo sustituye. Hay que comprobar que sigue sabiendo abstenerse: está documentado que este tipo de afinado puede empujar a responder incluso con documentos que no sirven.

Probado, no e producción. Se afinados ligero Rank Adaptation: con un adaptador modelos pequeños y 8 000 millones de parámetros, propio. Ninguna mejora medible en la base con RAG. que se sostiene **motor anti-aliasing** modelo afinado no es fiable para

OTROS NOMBRES QUE OIRÁS

- **Self-RAG.** Entrena al propio modelo que responde para que decida cuándo necesita buscar y señale si lo que afirma está respaldado. A diferencia de CRAG, exige afinar ese modelo.
- **HyDE** (Hypothetical Document Embeddings: busca a partir de una respuesta hipotética). El modelo redacta primero una respuesta imaginaria y se busca por el parecido con ella. Puede ayudar en preguntas vagas o conceptuales, pero en dominios de datos exactos —umbrales, fórmulas— tiende a colar invenciones y añade tiempo. Para normativa, mejor no.
- **Adaptive-RAG.** Un clasificador pequeño decide, antes de nada, si la pregunta necesita buscar, basta con una búsqueda o hacen falta varias encadenadas. Ahorra tiempo en las preguntas sencillas.
- **Mejoras del troceado** (*Contextual Retrieval, late chunking*). No cambian la búsqueda: dan a cada fragmento contexto del documento entero antes de guardarlo, para que un párrafo suelto no pierda de qué norma o apartado habla.

RAG O AFINAR: NO ES LO MISMO

RAG no cambia el modelo: en cada pregunta le pone delante el texto que necesita. Si una norma cambia, cambias el documento y ya está.

Afinar (*fine-tuning*; LoRA es su versión ligera) cambia el propio modelo reentrenándolo con ejemplos. Sirve para enseñarle *cómo* trabajar —un formato, un estilo, a apoyarse en el contexto—, pero **no es una forma fiable de meterle hechos nuevos**: la investigación publicada indica que, al forzarle datos que no conocía, tiende a sonar más seguro y a inventar más. Y cada cambio de norma obligaría a reentrenar.

RAFT está en medio: afina, pero para que el modelo *use mejor* lo que el RAG le entrega. Sigue necesitando el RAG.

POR DÓNDE EMPEZAR

RAG básico con reordenador y diez preguntas cuya respuesta ya conoces. Después, las capas correctivas de vigencia y atribución, que en normativa son las que más rinden. El híbrido, solo si lo mides y mejora. Grafos, agentes y afinados, cuando tengas un problema concreto que los pida, no antes.

Que no se invente nada: el sistema anti-alucinación

En un RAG normativo, una respuesta inventada no es un error simpático: puede acabar en un informe que sostiene una licencia de actividad o un dictamen. Por eso el diseño **no confía en que el modelo se porte bien**. Pone tres capas.

Capa	Qué hace	Quién decide
1 • Anclaje	Cada fragmento arrastra su procedencia: norma, artículo, página. Sin fuente no hay afirmación.	la estructura de los datos
2 • Vigencia	Si el fragmento es de una norma derogada o sustituida, se marca y se nombra la sucesora.	una tabla, no el modelo
3 • Atribución	Detecta que un fragmento es cabecera, nota o resumen, y avisa: «esto no es texto de la norma».	una regla, con su límite declarado

Por qué la tercera capa es la más sutil

Un fragmento puede estar perfectamente anclado —viene del fichero de la norma X, página 34— y aun así **no ser la norma**: puede ser la portada del documento o una nota de quien lo preparó. Si se cita como normativo, tiene toda la apariencia de una cita legítima y es falso. Es la peor alucinación posible: **la que suena a ley**.

LA CITA CADUCADA

Hay un tipo de error que la literatura especializada nombra expresamente: **una fuente válida pero desfasada, presentada como vigente**. Es especialmente traicionera porque el lector baja la guardia justo al ver una cita. La capa de vigencia existe para eso, y es algo que los asistentes generalistas no suelen hacer.

EL PRINCIPIO QUE GOBIERNA TODO

No mentir nunca no es lo mismo que acertar siempre. El objetivo no es responder a todo: es que, cuando responda, lo que diga sea trazable, y que cuando no sepa, lo diga. Un RAG que contesta siempre es exactamente el problema que se quiere evitar.

Cómo se construye: de un PDF a un fragmento fiable

El trabajo de verdad no está en el motor: está en **preparar bien el corpus**. Convertir una norma en PDF en fragmentos consultables y fiables pasa por nueve etapas, y la pregunta que gobierna el diseño es: **¿qué puede hacer una máquina gratis, y dónde hace falta juicio?**

9

etapas del proceso

6

sin coste: Python o modelos
locales

2

requieren juicio (modelo de
pago)

1

decisión humana: qué norma
entra

#	Etapas	Quién o qué	Coste
0	Elegir qué norma entra	la persona responsable	—
1	Localizar la fuente oficial	búsqueda	gratis
2	Extraer texto y tablas	Python	gratis
3	Borrador de fórmulas-imagen y tablas sin marco	modelo de visión local	gratis
4	Redactar el fragmento final	modelo de pago	juicio
5	Verificación mecánica	Python	gratis
6	Comparar contra el borrador	Python — mide, no decide	gratis
7	Doble revisión: glifos y omisiones	modelo de pago	juicio
8-9	Incorporar al corpus y cargar en la base	control de versiones + modelo local	gratis

LA LECCIÓN DE MÉTODO, QUE VALE MÁS QUE LA HERRAMIENTA

Separa lo que una máquina puede verificar de lo que requiere juicio, y no pagues por lo primero. Las etapas que miden no opinan; las que opinan son pocas, y ahí es donde se gasta.

Tres pautas que salen de construirlo

1 · La autoridad es el PDF renderizado, no su capa de texto. Al extraer texto de un PDF normativo se pierden o se desordenan los símbolos de las fórmulas: subíndices, letras griegas, operadores. Antes de dar un fragmento por bueno se compara *carácter a carácter* contra lo que se ve en la página. Un diff, no una lectura.

2 · Un modelo puede «corregir» en silencio. Ante un paréntesis desbalanceado en el original, algún modelo de visión lo cierra y entrega un texto plausible que ya no es el de la fuente. Por eso la revisión final se hace con otra herramienta que la que redactó, y por eso se prueba cada modelo con un caso trampa antes de fiarse de él.

3 · Comprueba cuánto texto ve de verdad tu modelo de embeddings. La ficha del modelo dice cuánto *puede* leer; la configuración del servidor decide cuánto *lee*. Si un

fragmento es más largo de lo que llega al vector, se sirve entero pero solo se puede *encontrar* por su primera parte. Se comprueba con una prueba sencilla: dos textos con el mismo inicio y distinto final deben dar vectores distintos.

La arquitectura completa, de un vistazo

Preparar el corpus es la primera capa. Estas son todas las del sistema de ejemplo, de la fuente a la respuesta, y **por qué existe cada una**: casi todas nacieron de un problema real que se detectó midiendo.

1 · FUENTE CURADA

Fragmentos en texto con su norma, apartado, vigencia y rango, bajo control de versiones



2 · INGESTA

Trocear con parámetros fijos y documentados
+ embeddings bge-m3 en local (Ollama)



3 · ALMACÉN

Qdrant: una sola base, con los dominios
separados por metadato



4 · MOTOR DE BÚSQUEDA

Filtrar y buscar → quitar duplicados → reordenar
(bge-reranker-v2-m3, en proceso aparte)



5 · CAPAS CORRECTIVAS

Vigencia · atribución · jerarquía · fecha
reglas fijas: siempre corren, nunca se saltan



6 · PUERTA DE ENTRADA

Herramientas para el asistente + instrucciones
de uso: consulta antes de afirmar



7 · MEDIDA

Preguntas con respuesta conocida y
comprobación contra lo que se sirve de verdad

De arriba abajo, el camino del dato: de la fuente revisada a la respuesta citada.

Capa	Qué hace	Por qué existe
1 · Fuente curada	Guarda cada fragmento como texto revisado, con sus metadatos, en un repositorio con historial de cambios.	Para poder reproducir cualquier dato desde su fuente documental . Si alguien corrige directamente la base de datos en lugar de la fuente, la corrección se pierde en la siguiente carga y ya nadie sabe qué copia manda.
2 · Ingesta	Trocea los textos, calcula sus <i>embeddings</i> con <code>bge-m3</code> y los carga en la base.	Porque la forma de trocear decide qué se puede encontrar. Se descubrió que un mismo corpus se había troceado de dos maneras distintas según cómo se subió cada fichero; la solución fue fijar y documentar los parámetros hasta poder reproducir el troceado idéntico .
3 · Almacén	Una única base Qdrant con todos los dominios (normativa, materiales, manuales de equipo, <u>ICT</u> (Infraestructura Común de Telecomunicaciones)...), distinguidos por un metadato.	Un solo modelo de <i>embeddings</i> y un solo sitio que mantener. La lección: si una base guarda dominios que no todos deben ver, el filtro protege la consulta, no el dato . Lo que no deba salir junto, guárdalo en bases separadas.
4 · Motor de búsqueda	Búsqueda por significado con filtros, eliminación de duplicados y reordenación.	Los fragmentos se solapan a propósito para no cortar frases, así que la misma respuesta llega a veces dos veces. El reordenador va en un proceso aparte porque es la pieza con más riesgo de quedarse sin memoria: si cae, se degrada con aviso, no se rompe .
5 · Capas correctivas	Vigencia, atribución y jerarquía normativa (pestaña «Que no se invente nada»).	Porque una cita correcta de una norma derogada, o de una nota que no es la norma, es el error más peligroso. Una lección práctica: los avisos tienen que llevar una marca que la regla reconozca ; un «pendiente de verificar»

escrito en prosa libre pasó sin aviso hasta que se amplió la regla.

6 · Puerta de entrada

Un pequeño servidor MCP (Model Context Protocol: estándar abierto para conectar herramientas a un asistente) ofrece al asistente unas pocas herramientas de consulta; unas instrucciones aparte le dicen cuándo usarlas y cómo leer los avisos.

La herramienta da capacidad; las instrucciones dan **criterio**. Sin ellas, el asistente tiene la herramienta pero no sabe cuándo usarla ni qué hacer si el corpus no cubre la pregunta.

7 · Medida

Conjunto de preguntas de evaluación y comprobaciones periódicas.

Porque «**corregido en la fuente**» **no es «corregido en lo que se sirve»**: se llegó a anunciar una norma como cerrada cuando el sistema en producción seguía devolviendo el texto antiguo. Cada corrección se verifica consultando el sistema real, no el repositorio.

DOS REGLAS PEQUEÑAS QUE EVITAN ERRORES GRANDES

- **Los metadatos no deben afirmar el estado de un proceso del que forman parte.** Una nota que diga «la recarga está pendiente» se vuelve falsa en cuanto se hace la recarga, y se sigue sirviendo. Mejor decir *dónde* consultar ese estado.
- **Un sello de integridad cubre lo que había al sellarlo.** Si añades ficheros a una carpeta sellada sin volver a sellarla entera, la verificación sigue diciendo «correcto»... sobre una parte.

Medir si funciona

Sin medir, un RAG «parece que va bien». Y parecer que va bien es el estado más peligroso, porque nadie lo revisa. La herramienta es sencilla: **un conjunto de preguntas reales cuya respuesta correcta ya conoces**.

El conjunto de evaluación

- **Preguntas reales**, de las que se hacen en el trabajo, no de ejemplo.
- **Cada una con su respuesta y su fuente** conocidas de antemano.

- **Si puede ser, de varias personas:** cada uno pregunta distinto, y eso destapa huecos que uno solo no ve.

Dos métricas que se entienden sin fórmulas

Métrica	Qué responde	Ejemplo real (14/09/2026, 41 preguntas)
recall@8	¿Está el fragmento con la respuesta entre los 8 primeros resultados?	0,966 — en 96-97 de cada 100 preguntas, sí
MRR	¿En qué posición aparece? Cuanto más cerca de 1, más arriba	0,843 — casi siempre en primera posición

UN LÍMITE QUE SOLO SE VE MIDIENDO

El modelo que convierte el texto en números solo «ve» los primeros **2.055 tokens** de cada fragmento. En nuestro sistema, **617 fragmentos** largos (553 de ellos de normativa) solo se pueden encontrar por su primer tramo (medido el 14/09/2026). El texto que se entrega está completo; lo que puede fallar es *encontrar* una cifra escondida al final de un fragmento largo. Por eso los fragmentos se trocean con cuidado y, ante una cifra concreta, se contrasta con el documento oficial.

LA PREGUNTA QUE HAY QUE HACER ANTE CUALQUIER CIFRA DE CALIDAD

No basta con el nombre de la métrica. Pregunta **con qué juez y con qué parámetros se ha medido**. Dos formas de «comprobar si la respuesta contiene el dato» pueden dar cifras muy distintas: una que acepta el número como trozo de texto da por buena una respuesta donde el «20» estaba dentro de «2022». La métrica honesta es la que sabría fallar.

Antes de incorporar una mejora, mídela

Hay técnicas que sobre el papel prometen mucho y, medidas sobre un corpus concreto, **empeoran el resultado que importa**. La combinación de búsqueda por palabras y búsqueda por significado, por ejemplo, se da por «línea base mínima» en la literatura y en un corpus normativo real puede meter en los primeros puestos fragmentos que desplazan al que traía la cifra. **Ninguna pieza entra sin medir su efecto sobre el conjunto de evaluación.**

LO QUE NO SE VE EN LAS MÉTRICAS

Un conjunto de evaluación mide lo que pregunta. Si hay texto que el sistema nunca llega a encontrar, el conjunto no lo detectará salvo que alguna pregunta caiga ahí.

«Sano» no es lo mismo que «completo»: por eso además de medir calidad se revisa la cobertura.

Tu primer RAG, y hasta dónde puede llegar

Empezar esta semana

1. **Elige un corpus pequeño y que conozcas bien:** el reglamento que más usas y dos o tres normas asociadas. No empieces por todo.
2. **Consigue las fuentes oficiales** en PDF, y guarda de dónde y cuándo las descargaste.
3. **Usa una interfaz que ya traiga RAG.** Open WebUI permite crear una base de conocimiento, subir documentos y preguntar contra ellos con un modelo local y `bge-m3` como `embeddings` (texto convertido en números que representan su significado) (Guía 1).
4. **Escribe diez preguntas** de las que ya sepas la respuesta y la fuente. Pásalas. Anota cuántas acierta y, sobre todo, **cómo falla**.
5. **Solo entonces amplía:** más normas, mejores fragmentos, un reordenador.

Hasta dónde se puede llegar

Nivel	Qué tienes
1 • Chat con documentos	Subes PDF a una interfaz y preguntas. Útil desde el primer día; sin control fino de vigencia ni de atribución.
2 • Corpus curado	Fragmentos preparados a mano o con ayuda, con su norma, apartado y edición. La calidad sube de golpe.
3 • Capas correctivas	Vigencia y atribución automáticas: sabes si lo que citas sigue en vigor y si es texto de la norma.
4 • En el ambiente	El asistente lo consulta solo antes de afirmar cualquier dato normativo. Deja de ser una herramienta que abres.
5 • Medido y mantenido	Conjunto de evaluación, controles periódicos entre lo que está curado y lo que se sirve, y un proceso para incorporar normas nuevas.

Lo que conviene mirar de las herramientas comerciales

NotebookLM, de Google —desde julio de 2026 llamado **Gemini Notebook**—, se basa en el mismo principio: el modelo solo puede responder con los pasajes de las fuentes que tú le das, nunca con su memoria. Hay dos cosas tuyas que merece la pena copiar:

- **La cita al pasaje exacto.** No basta con decir «de la norma X»: la unidad útil es la línea concreta de la que sale la afirmación, para que el lector pueda comprobarla de un vistazo.
- **El contexto cerrado.** Sin salir a internet ni mezclar cuadernos: es lo que hace fiables sus citas. Un RAG local lo tiene por construcción.

Para documentación pública y formación, una herramienta así es estupenda y no exige montar nada. Para documentación de clientes, la regla de la Guía 1 sigue en pie: **local**.

RESUMEN EN TRES FRASES

Un RAG responde con tus documentos y te dice de dónde. Lo que lo hace fiable en normativa es saber qué está en vigor y qué es texto de la norma. Y lo que lo mantiene fiable es medirlo con preguntas cuya respuesta ya conoces.

Guía 4 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Cifras del sistema de ejemplo medidas el 14/09/2026 (dossier técnico del RAG, v04) y el 15/09/2026 (tipos de RAG). Las métricas de calidad corresponden a un conjunto de 41 preguntas reales.

[¿Una sigla que no conoces? Consulta el glosario](#)

El despacho automatizado

El proyecto es lo que se factura; el correo, los documentos, las hojas de cálculo, las cuentas y las mil tareas pequeñas son lo que se come las horas. Aquí es donde la IA se nota antes, y donde menos riesgo tiene empezar.

Nivel: práctico

Lectura: ~25 min

Vale con herramientas comerciales o locales

Por dónde empezar

La mejor primera automatización no es la más espectacular: es **la tarea repetitiva que más te molesta** y cuyo resultado sabes comprobar de un vistazo.

Tarea	Cómo se ataca	Dificultad	Lo que ahorra
Normativa: encontrar el artículo que aplica	<u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) sobre tu corpus (Guía 4) o un cuaderno de fuentes (Guía 2)	media	el que más cambia el día a día
Correo: clasificar, redactar, hacer seguimiento	Asistente dentro del gestor de correo	fácil	minutos varias veces al día
Excel y <u>VBA</u>: fórmulas, macros, limpieza de datos	La IA escribe; tú revisas y pruebas	fácil	mucho, y desde el primer día
Cálculo: repetir un cálculo con datos nuevos	Un script que la IA prepara	media	horas, y menos errores de tecleo
Documentos: informes a partir de plantilla y datos	Plantilla + datos → documento	media	la parte mecánica del informe
Archivo: renombrar, clasificar, encontrar	Scripts pequeños y un gestor documental	fácil	el tiempo de buscar

LA REGLA DE REPARTO

La IA prepara, redacta y localiza. El cálculo lo hace un programa. La decisión y la firma son tuyas. Cada automatización de esta guía respeta ese reparto.

Correo y documentos

Correo

Triaje: «de estos correos, ¿cuáles piden algo con plazo y cuál es el plazo?».

Borrador: «redacta la respuesta al requerimiento, con tono formal, citando el informe adjunto»; la revisas y la envías tú.

Seguimiento: «resume el hilo con el constructor y lista lo que queda pendiente de cada parte».

Con Gmail o Outlook, las herramientas comerciales ya lo hacen dentro del propio correo (Guía 2). Si el correo contiene datos de cliente, recuerda la regla: **plan de empresa o IA local**.

Documentos

- **Digitalizar y encontrar:** un gestor documental con OCR (Optical Character Recognition: convertir imagen o PDF escaneado en texto) (por ejemplo Paperless-ngx, libre) lee los PDF escaneados, los etiqueta y los hace buscables. Es además la base perfecta para un RAG más adelante.
- **Convertir para la IA:** pasar Word, Excel y PDF a texto estructurado (Markdown) hace que cualquier modelo los entienda mucho mejor que el fichero original.
- **Generar desde plantilla:** la estructura, los logotipos y los textos fijos en una plantilla; los datos del encargo, aparte. El documento se *regenera*, no se edita a mano cada vez.

UN HÁBITO QUE SE PAGA SOLO

Guarda las **notas de visita en audio** y pásalas por un transcriptor. Una nota de voz de dos minutos en obra se convierte en el borrador del apartado de antecedentes.

Excel y VBA: la automatización más rentable

Casi todos los despachos tienen hojas de cálculo críticas. La IA es especialmente buena escribiendo **fórmulas, macros VBA y scripts** que las manejan.

Cómo pedirlo bien

1. **Describe la hoja:** qué hay en cada columna, en qué fila empiezan los datos, qué formato tienen las fechas y los decimales.
2. **Describe el resultado** con un ejemplo: «para esta fila, el resultado debe ser este».
3. **Pide que explique** la macro línea a línea antes de ejecutarla.
4. **Pruébala sobre una copia**, nunca sobre la hoja original.
5. **Compara con un caso que ya tengas calculado a mano.** Si coincide, confías; si no, has encontrado el error sin coste.

Ejemplo: «Tengo en la columna B niveles en dB y en la C la franja horaria (día, tarde, noche). Escribe una macro que marque en rojo las filas que superen el límite de cada franja, tomando los límites de la hoja «Límites»».

Entender las macros que ya tienes

Las hojas heredadas suelen tener macros que nadie recuerda. **Extraer el código VBA y pedir a la IA que lo explique** —qué hace, qué hojas toca, qué puede romperse— es una de las tareas más útiles y menos arriesgadas: no cambia nada, solo lo documenta.

CUIDADO CON LOS SEPARADORES

En España el decimal es la coma y el separador de miles el punto. Una macro escrita con convenciones inglesas puede leer `1.234` como «uno coma dos tres cuatro» sin dar ningún error. **Dilo explícitamente al pedirla y compruébalo con un número grande.**

Cálculo: el número lo da el programa

Un modelo de lenguaje puede escribir una cifra perfectamente formada y perfectamente falsa. Por eso la pauta es innegociable: **la IA escribe el cálculo, y el resultado sale de ejecutarlo.**

Qué se puede automatizar

- **Cálculos repetitivos por lotes:** procesar de una vez todas las mediciones de una campaña y obtener los parámetros normalizados, en lugar de uno a uno.
- **Comprobación de cumplimiento:** un programa que, a partir de un valor medido o calculado, indica si cumple o no, con el margen y el artículo que lo exige.
- **Predicción:** estimaciones previas con modelos de cálculo abiertos, para comparar soluciones antes de proyectar.
- **Calibración:** ajustar un modelo de simulación para que reproduzca lo medido.

Ejemplo: «Escribe un script en Python que lea este Excel de mediciones, calcule el resultado según el procedimiento que te adjunto y genere una tabla con el resultado de cada punto». Luego lo ejecutas, y lo contrastas con un punto que ya tenías calculado.

TRES COMPROBACIONES ANTES DE FIARTE DE UN CÁLCULO AUTOMATIZADO

1. **Caso conocido:** reproduce un resultado que ya tienes validado.
2. **Unidades:** revisa que entradas y salidas estén en las unidades que crees.
3. **Extremos:** prueba con valores límite (cero, muy grande, dato ausente) y mira que no dé un resultado absurdo en silencio.

LA FUENTE DEL MÉTODO

Si el cálculo sigue una norma, el script debe decir **qué norma, qué edición y qué apartado** implementa. Así, cuando la norma cambie, sabes qué hay que revisar.

Cuentas y presupuestos

Facturas y gastos

- **Extraer datos a tabla:** emisor, fecha, base, IVA y total de cada factura en PDF, para cuadrar con la contabilidad.
- **Revisar, no inventar:** pide que marque las facturas donde la suma no cuadre o falte un dato, en lugar de que las «corrija».
- **Datos fiscales, en local o en plan de empresa.** Son exactamente el tipo de información que no conviene subir a un plan personal.

Presupuestos y bases de precios

Las bases de precios de construcción se intercambian en formato **FIEBDC (.bc3)**, el que usan programas de mediciones y presupuestos habituales. Con un pequeño script se leen y se llevan a una hoja de cálculo, y la IA puede ayudarte a buscar partidas o a comparar precios entre bases.

Ejemplo: «De esta base de precios, lista las partidas de canalizaciones de telecomunicaciones con su precio unitario, y compáralas con las del presupuesto del año pasado».

DONDE DE VERDAD AYUDA

En la **preparación**: localizar, ordenar, comparar y detectar incoherencias. La cifra final del presupuesto la cierra una hoja o un programa de presupuestos, y la revisas tú.

Tareas repetitivas: pequeños scripts, grandes ahorros

Muchas automatizaciones útiles son programas de pocas líneas que la IA escribe en un minuto:

Tarea	Qué hace el script
Renombrar por lotes	Pone a cientos de fotos o documentos el nombre con fecha, obra y tipo
Encontrar duplicados	Detecta ficheros repetidos aunque tengan nombres distintos
Ordenar por fecha	Organiza carpetas de campo en orden cronológico
Vigilar una carpeta	Cuando llega un fichero nuevo, lo procesa solo (convierte, extrae, archiva)
Avisar al móvil	Te notifica cuando termina un proceso largo o algo falla
Transcribir	Convierte las grabaciones de reuniones y visitas en texto
Convertir formatos	Word, Excel, PDF a texto estructurado; datos del instrumento a Excel

De script suelto a herramienta

Cuando varios scripts se usan a menudo, compensa reunirlos en **una pequeña aplicación de escritorio** con un botón por tarea. Es exactamente lo que se ha hecho en el sistema de ejemplo de la Guía 3: una suite con dieciocho herramientas de medición acústica y gestión

de documentos, y otra que genera un proyecto **ICT** (Infraestructura Común de Telecomunicaciones (RD 346/2011)) completo —planos, cálculos y memoria— a partir de los datos del edificio.

Pautas para automatizar sin sustos

1. **Copia de seguridad antes de ejecutar** cualquier cosa que modifique ficheros.
2. **Primero en seco:** que el programa diga qué haría antes de hacerlo.
3. **Empieza por lo que puedes comprobar de un vistazo.**
4. **Un caso conocido** para validar cada automatización antes de usarla con trabajo real.
5. **No automatices lo que no entiendes.** Si no sabes explicar qué hace, pide que te lo explique antes de usarlo.
6. **Deja rastro:** que cada proceso escriba qué ha hecho y cuándo.
7. **Datos de clientes y fiscales, en local o en plan de empresa.**
8. **Revisa lo que sale.** La automatización ahorra el trabajo mecánico, no la responsabilidad de revisar.

RESUMEN EN UNA FRASE

Automatiza lo repetitivo y comprobable, deja que un programa calcule, y guarda tu tiempo para lo que solo tú puedes decidir.

Construir tu IA con la ayuda de una IA comercial

No hace falta ser programador para montar un sistema propio. Hace falta un buen asistente de pago que haga de técnico a tu lado, y un método de trabajo que te permita confiar en lo que construye. Esta guía cuenta ese método.

Nivel: todos

Lectura: ~25 min

El método detrás de todo lo anterior

La idea: un técnico de sistemas que escribe por ti

Los asistentes de IA de pago más capaces ya no solo conversan: **pueden trabajar en tu ordenador**. Leen ficheros, ejecutan órdenes, instalan programas, escriben scripts y los prueban. Con uno de ellos, montar un modelo local, un RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) o una herramienta de cálculo pasa de «proyecto de informático» a «encargo que supervisas».

Lo hace el asistente	Lo haces tú
Instalar y configurar programas y servicios	Decidir qué se instala y dónde
Escribir scripts, macros y aplicaciones	Definir qué deben hacer y comprobar que lo hacen
Buscar el error cuando algo falla	Decidir si la solución propuesta es aceptable
Documentar lo que ha hecho	Leerlo y aprobarlo
Preparar el borrador técnico	Firmar

EL CAMBIO DE PAPEL

Pasas de ejecutar a **dirigir y verificar**. Es un papel que un ingeniero conoce bien: el de director de obra. El asistente construye; tú defines, inspeccionas y das por buena cada fase.

Todo lo que se cuenta en las Guías 3 a 5 —los modelos, el RAG, las herramientas de ingeniería, la búsqueda sin gastar tokens (trozo de texto con el que trabaja el modelo)— se ha construido así.

Cómo se trabaja: el ciclo de cada encargo

1 · Plan

Antes de tocar nada, el asistente propone qué va a hacer. Tú lo apruebas o lo corriges.

2 · Copia

Copia de seguridad de lo que se va a modificar.

3 · En seco

Si la operación cambia o borra algo, primero se simula.

4 · Ejecución

Pasos pequeños, uno cada vez.

5 · Verificación

Se comprueba el resultado con un caso conocido, no con «ha terminado sin error».

6 · Registro

Qué se hizo, cuándo y cómo deshacerlo.

Cómo pedir las cosas

- **Di el objetivo, no solo la orden.** «Quiero consultar mi normativa con cita» da mejor resultado que «instala Qdrant».
- **Pide opciones con su pega.** «Dame dos formas de hacerlo y qué pierdo con cada una.»
- **Pide que te explique lo que no entiendas** antes de aprobarlo.
- **Pide que distinga lo medido de lo supuesto.** «¿Esto lo has comprobado o lo deduces?» es la pregunta más útil que puedes hacerle.

«HA TERMINADO SIN ERRORES» NO SIGNIFICA «FUNCIONA»

Un proceso puede acabar limpio y haber hecho otra cosa. La verificación es comprobar el **resultado** —abrir el documento, consultar el dato, repetir un cálculo conocido—, no el mensaje de que acabó.

Que no olvide nada: el contexto persistente

Un asistente empieza cada sesión sin recordar la anterior. Si el conocimiento del sistema vive solo en la conversación, **se pierde**. La solución es que viva en ficheros.

Fichero	Para qué sirve
Instrucciones del proyecto	Qué es el sistema, qué máquinas hay, qué reglas se siguen siempre. El asistente lo lee al empezar.
Estado	Qué está hecho, qué está a medias y qué falta.
Hoja de ruta	El plan vigente, con fecha. Una nueva cada vez que el plan cambia de verdad, sin borrar la anterior.
Memoria de lecciones	Lo aprendido de los errores, para no repetirlos en la sesión siguiente.
Texto de arranque	Al cerrar, un resumen para pegar al abrir la siguiente sesión.

LA COSTUMBRE QUE MÁS RINDE

Cerrar cada sesión con el estado escrito. Al día siguiente, en lugar de explicar otra vez dónde estabas, pegas el texto de arranque y el asistente continúa.

Las **reglas** merecen su propio sitio: «nunca borrar sin copia», «toda cifra normativa con su fuente», «no instalar sin anotar la versión». Escritas una vez en las instrucciones, el asistente las aplica siempre.

Darle herramientas: conectores y procedimientos

Un asistente es mucho más útil cuando puede usar directamente tus sistemas. Eso se hace con **conectores** (un estándar llamado MCP (Model Context Protocol: conecta el asistente con herramientas), *Model Context Protocol*): pequeños programas que le dan acceso controlado a una herramienta concreta.

Conector	Qué le permite al asistente
Tu <u>RAG</u> normativo	Consultar la normativa con vigencia antes de afirmar una cifra
Acceso al servidor	Ejecutar órdenes en otra máquina de forma controlada
Búsqueda web local	Buscar en internet haciendo que lea las páginas un modelo local (Guía 3)
Programas de ingeniería	Manejar QGIS o generar documentos a partir de datos

Procedimientos reutilizables

Las tareas que se repiten —preparar un informe de jornada, dar de alta una norma en el RAG, cerrar una sesión— se escriben una vez como **procedimiento** que el asistente sigue paso a paso. Así el resultado sale igual cada vez, lo haga quien lo haga.

CREDENCIALES: NUNCA EN EL CHAT

Si el asistente necesita una contraseña, no la escribas en la conversación: queda en el historial. Usa el mecanismo de credenciales del sistema operativo o del conector, que la guarda cifrada sin que el asistente llegue a verla.

Varios agentes que se coordinan y se revisan

Cuando el sistema crece, conviene tener **un asistente por máquina o por área**, cada uno con su trabajo. Para que no se pisen, necesitan una forma ordenada de hablar entre ellos. En el sistema de ejemplo se llama, con humor, el **Patio de Vecinos**.

Pieza	Qué es
Tablón común	Un fichero compartido donde cada agente anota mensajes: avisos, entregas, preguntas, acuses. Solo se añade, nunca se borra.
Buzones	Cada agente recibe lo que va dirigido a él, y un vigía le avisa en cuanto llega algo.
Salas de trabajo	Cuando dos o tres agentes trabajan juntos en algo, tienen un espacio propio para no llenar el tablón común.
Reparto por carriles	Cada agente reclama su tarea antes de empezar, para que no la haga otro a la vez.

La parte que más vale: la revisión cruzada

Ningún trabajo importante se da por bueno con la palabra de quien lo hizo. Antes de publicarlo pasa por dos revisiones con papeles distintos:

- **Un adversario**, cuyo trabajo es intentar demostrar que está mal.
- **Un auditor**, que comprueba referencias, coherencia y que no falte nada.

Y los agentes se revisan entre sí: es habitual que uno detecte en la herramienta de otro un caso que su autor no había probado, y que el primer arreglo se vuelva a revisar antes de

darlo por cerrado.

LO QUE SE LLEVA UN DESPACHO DE UNA SOLA PERSONA

No necesitas varios agentes para aplicar la idea. El valor no está en que la IA acierte, sino en **montar un sistema que detecte cuándo se equivoca**: pedir a un segundo asistente —o a una segunda sesión— que revise con ojos críticos lo que produjo el primero.

Coste y límites

Cuánto cuesta

Los asistentes capaces de trabajar sobre tu ordenador se contratan por suscripción mensual. Como referencia, los planes de Claude cuestan **17–20 \$/mes (Pro)** y **desde 100 \$/mes (Max)**, en dólares y sin impuestos, a 14 de septiembre de 2026; los demás fabricantes tienen planes de rango parecido (Guía 2). El uso intensivo consume antes los límites del plan.

GASTA EL PLAN DE PAGO EN JUICIO

Lo mecánico —extraer, convertir, comparar, buscar— que lo haga un modelo local o un script. El asistente de pago, donde hace falta criterio: diseñar, decidir, revisar. Es la misma idea con la que se construyó el RAG de la Guía 4.

Sus límites, dichos claro

- **Se equivoca con aplomo.** Una respuesta segura no es una respuesta comprobada.
- **Puede dar por hecho algo que no ha verificado** si no le pides que distinga.
- **No recuerda** lo que no esté escrito en ficheros.
- **Hace lo que le permitas.** Configura qué puede ejecutar sin preguntar y qué no.
- **No asume responsabilidad.** Esa sigue siendo tuya.

Pautas de trabajo con un asistente

1. **Plan antes de código.** Nada se toca sin un plan aprobado.
2. **Copia y ensayo en seco** antes de cualquier operación que modifique o borre.
3. **Pasos pequeños** y verificación después de cada uno.

4. **Verifica con un instrumento distinto** del que produjo el resultado.
5. **Distingue medido de supuesto**, en lo que te dice y en lo que escribes.
6. **Toda cifra con su fuente y su fecha.**
7. **El estado, escrito**: instrucciones, estado, hoja de ruta, lecciones.
8. **Credenciales fuera del chat.**
9. **Dos intentos fallidos, cambia de enfoque.** Insistir por tercera vez con la misma idea rara vez funciona.
10. **Una segunda mirada crítica** antes de dar por bueno algo importante.

PARA EMPEZAR ESTA SEMANA

Contrata un plan de un asistente capaz de trabajar en tu ordenador. Pídele que te ayude a instalar Ollama (programa gratuito para ejecutar modelos en tu equipo) y un modelo pequeño (Guía 1). Escribe con él un fichero de instrucciones con tus tres reglas básicas. Y cierra la sesión pidiéndole el estado por escrito. **Ya tienes el método en marcha.**

Guía 6 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio

Oficial de Ingenieros de Telecomunicación.

Precios de referencia consultados el 14/09/2026.

[¿Una sigla que no conoces? Consulta el glosario](#)

El ecosistema de vecinos

Cuando un solo asistente no basta, se trabaja con varios. En nuestro sistema hay más de veinte agentes de IA repartidos por cuatro máquinas, y los llamamos **vecinos**: cada uno vive en su casa, tiene su oficio y se comunica con los demás por un tablón común. Esta guía explica quién hace qué, cómo se coordinan y cómo arrancan y cierran cada jornada sin perder el hilo.

Nivel: divulgativo

Lectura: ~45 min

Sistema en uso real, septiembre de 2026

El vecindario: quién vive en cada máquina

Cada máquina es una **escalera** de la comunidad, y en cada una viven varios agentes. Hay dos tipos: **Claude Code** (`CC` (Claude Code · Claude Desktop)), que trabaja directamente sobre ficheros, terminal y servidores, y **Claude Desktop** (`CD`), la aplicación de escritorio, más orientada a análisis, planificación y revisión.

Escalera	Vecinos principales	A qué se dedican
PC de despacho (Windows)	CC_BM · CCHESC27 · CD27 · y especialistas por proyecto (<u>ICT</u> (Infraestructura Común de Telecomunicaciones (RD 346/2011)), acústica de edificios, fotografía...)	Supervisión y coordinación, lo que solo funciona en Windows (Office, <u>VBA</u> (Visual Basic for Applications, macros de Office), aplicaciones de ingeniería), auditoría transversal y MATLAB
Servidor Linux (<u>GPU</u> (Graphics Processing Unit, tarjeta gráfica) de 16 GB)	CC_220 · CCHESC220 · CCH27	Motor de IA: modelos locales, <u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos), contenedores, el servicio que reparte los mensajes; instalación del ecosistema; Manual de Calidad
Mac Studio (128 GB unificada)	CC_OSX55 · CDH55 · CD55 · CC55 · CCK55 · CCPY55 · CC_PAT55	Modelos grandes, generación del <u>RAG</u> normativo, verificación carácter a carácter, realidad virtual y campo, memoria de contexto, herramientas del Mac y mantenimiento del Patio
Portátil	CC26 · CD26	Ayudantes y trabajo de campo

Cómo leer los nombres

- **CC** = Claude Code · **CD** = Claude Desktop.
- **H** = *help*, ayudante · **ESC** = escudero de otro vecino.
- El **número final** es la máquina donde vive. En una misma máquina puede haber varios Claude Code; se distinguen por la carpeta de trabajo de cada uno.

POR QUÉ TANTOS

No es por tener más potencia: es por **separar papeles**. El que construye no es el que revisa, y cada uno conoce bien su terreno. Exactamente como en un despacho con varias personas.

Los papeles: quién lidera, quién ayuda, quién revisa

Capa	Quién	Qué hace
Árbitro	Joaquín Serrat González, el ingeniero	Decide lo importante y firma. Tiene la última palabra
Cúspide	Los agentes principales del servidor y del Mac, con Joaquín	Decisiones de arquitectura del sistema, con veto
Supervisor	CC_BM, en el PC de despacho	Traduce las decisiones en encargos para los especialistas y coordina
Especialistas	Un agente por proyecto o dominio	Hacen el trabajo de su terreno
Soporte	Claude Desktop y otros	Análisis, planificación, segunda opinión

Quien inicia y lidera un proyecto

El que lo convoca (o Joaquín), que define objetivo y criterio de cierre. En el RAG normativo, por ejemplo, un único agente **integra** los cambios: los demás proponen, él decide qué entra, para que no haya dos versiones de la verdad.

Ayudantes y escuderos

Algunos vecinos tienen un **escudero**: otro agente en su misma máquina que le ayuda con tareas y, sobre todo, **revisa su trabajo**.

Segundas lentes y verificación

Una **segunda lente** es un agente que mira lo mismo con un instrumento o un enfoque distinto. Ningún trabajo importante se da por bueno con la palabra de quien lo hizo; pasa por la **doble puerta**:

1 · Adversario

«Abogado del diablo»: intenta demostrar que está mal, probando cada sospecha.

2 · Auditor

Puerta final: referencias cruzadas, coherencia, fuentes completas. Emite SÍ o NO.

3 · Si cambia algo

El veredicto no se hereda: el fichero modificado vuelve a pasar.

Altas de nuevas habilidades: tres revisores

Antes de dar de alta un agente o una habilidad nueva, lo revisan tres: un **especialista** (que declara cuándo *no* debe activarse), un **auditor** (que comprueba que no duplica algo existente) y un **adversario** (que busca falsos positivos).

Oficios automáticos

Además de los agentes, hay pequeños procesos con nombre de oficio: el **vigía** (avisa de mensajes), el **cronista** (indexa las conversaciones), el **sereno** (ronda de salud)... Norma: *los oficios duermen con su vecino*: si la sesión está cerrada, no corren gastando recursos.

El Patio de Vecinos: el tablón común

Es el **único canal** entre agentes. Vive en una carpeta compartida por red y es, literalmente, un fichero de texto con una tabla a la que todos añaden filas.

Pieza	Cómo funciona
El tablón (BUS)	Cada mensaje es una fila: fecha y hora, quién, para quién, tipo y contenido. Solo se añade, nunca se edita ni se borra : es el registro de todo lo que ha pasado.
Buzones (INBOX)	Un servicio en el servidor lee el tablón cada 15 segundos y copia cada mensaje al buzón de su destinatario, con aviso al móvil.
Tipos de mensaje	Aviso, informe, tarea, consulta, acuse, hallazgo, propuesta, decisión para Joaquín... Separar <i>lo que he medido</i> de <i>lo que propongo</i> evita mezclar diagnóstico y decisión.
Vigía	Cada agente, mientras su sesión está abierta, tiene un vigía que le avisa en cuanto llega algo para él.
Firma	Cada agente escribe solo en su propio nombre.

UNA GUARDA QUE NO DEPENDE DE ACORDARSE

Si un agente intenta publicar una **propuesta o una decisión** sin decir quién es responsable, quién aprueba, a quién se consulta y a quién se informa (**RACI** (Responsable · Aprueba · Consultado · Informado)), la herramienta de publicación **la rechaza**. No hace falta recordarlo: el sistema no deja.

LO APRENDIDO ESCRIBIENDO EN UN FICHERO COMPARTIDO

Dos agentes escribiendo a la vez en el mismo fichero pueden corromperlo. Por eso las escrituras se hacen con **candado exclusivo** y se **verifican releyendo**; y los ficheros que se reescriben enteros solo se sustituyen si nadie los cambió mientras se preparaba el cambio.

Reunión de Vecinos, Sala de Debate y Laboratorio

El tablón sirve para el día a día. Cuando hace falta algo más, hay tres espacios, cada uno con un propósito distinto:

	Reunión de Vecinos	Sala de Debate	El Laboratorio
Para qué	Resolver una contradicción : dos vecinos discrepan sobre un hecho medido, o algo va a escalar a Joaquín	Que 2–5 vecinos decidan juntos algo sin llenar el tablón	Ejecutar hasta el final un plan ya decidido
Ficheros	Reunión activa + acta	Historial (solo se añade) + pizarra (estado vivo)	Plan (no se reescribe) + lista de tareas + historial
Papeles	Convocante, presidente, secretario (distinto del presidente), asistentes y adversario con turno obligatorio	Participantes que reclaman cada tarea antes de empezar	Coordinador, escriba y «blindaje de ruido», que filtra lo que llega de fuera
Cómo termina	Por quórum, no por reloj. Mejor un cierre parcial que uno falso	Marcada como cerrada, con el resultado en una línea	Cuando se cumple el criterio de cierre verificable

Reglas de la Reunión que valen para cualquier comité

- **No se convoca para informar**, solo para decidir un conflicto.
- **Ausente no es conforme**. Quien no ha votado, no ha aprobado.
- **No se cierra con casillas vacías**. Se escribe «aprobada 3 de 4, pendiente» antes que fingir unanimidad.
- **El adversario habla antes del consenso**, no después.

- Quien preside ordena el debate; **no decide el fondo**.

RESUMEN

La Reunión **resuelve conflictos**, la Sala **decide**, el Laboratorio **ejecuta**, y el Patio **cuenta lo que pasa**.

Arrancar la jornada

Como un agente empieza cada sesión sin memoria (Guía 8), el arranque es un **protocolo fijo** que le devuelve el contexto en orden:

1 · El vigía, lo primero

Si se arma tarde, otros agentes hablan al aire sin que nadie se entere.

2 · Ponerse al día

Lee lo que llegó al tablón con la sesión cerrada, desde el último punto guardado.

3 · Conectar

Acceso al servidor y a las carpetas compartidas.

4 · Hoja de ruta

Lee la más reciente: es el plan vigente, no lo que recuerde.

5 · Pendientes y buzón

Panel de preguntas pendientes y mensajes sin procesar.

La hoja de ruta, con fecha

Cada vez que el plan cambia de verdad, se escribe una **hoja de ruta nueva** con fecha y hora, sin borrar la anterior. Así queda trazado cómo evolucionó el plan, y siempre se actúa sobre la más reciente. Nació de un día en que la sesión se desvió tanto con urgencias que se perdió de vista el plan original.

COMPROBAR QUE SIGUE VIVO, NO SOLO QUE ARRANCÓ

Un vigía que arrancó bien puede morir en silencio una hora después. Por eso se comprueba que el proceso existe y que su registro avanza. **«Arrancó sin error» no es «funciona»**.

Cerrar la jornada: cómo se mantiene la memoria

Hay **dos niveles** de cierre:

Nivel	Cuándo	Qué se hace
Punto de control ligero	Pausas dentro de una sesión larga	Tres a cinco líneas de estado en la memoria del proyecto. El detalle fino se puede recuperar después del historial indexado
Cierre formal	Fin de jornada, traspaso a otro agente, antes de varios días	El protocolo completo de abajo

El cierre formal, paso a paso

1. **Informe de jornada** en una página con pestañas: qué se hizo, qué queda, qué decisiones esperan a Joaquín con la opción recomendada, y el estado del Manual de Calidad.
2. **Mirar el Patio y contestar** lo que esté pendiente: el tablón no despierta a nadie, y cerrar sin mirar deja a otros esperando.
3. **Escribir la memoria**: lecciones nuevas, estado del proyecto y la hoja de ruta.
4. **Indexar las conversaciones** de la sesión, para poder consultarlas después.
5. **Publicar el cierre** en el tablón.
6. **Texto de arranque**: un bloque para pegar al abrir la próxima sesión, con número de sesión, descripción, plan numerado por prioridad y contexto.

«CERRAR ES PARAR»

Una vez presentado el texto de arranque, la sesión ha terminado. Lo que aparezca después —aunque sea un hallazgo importante— se convierte en el **primer punto de la sesión siguiente**, no en una excusa para seguir tocando ficheros.

LO QUE EL CIERRE NO PUEDE SALVAR

El historial indexado recupera **hechos y decisiones escritos**, no el razonamiento que nunca se escribió. Por eso el cierre formal escribe a mano lo que de verdad importa.

La infraestructura por dentro

La pestaña 3 cuenta *qué* es el Patio. Esta cuenta *cómo* está construido y, sobre todo, **por qué** cada pieza es como es. Casi todas las decisiones nacieron de un fallo real que se midió, no

de un diseño en pizarra. Si quieres montar algo parecido, aquí está lo que nos costó aprender.

La idea de partida: un fichero, no una centralita

En vez de conectar los agentes entre sí por programas que se llaman unos a otros, se les da **un sitio compartido**: una carpeta de red que todas las máquinas leen y escriben, con un tablón de texto en el centro. El único contrato entre agentes es **un fichero de texto con columnas fijas**. Por eso funciona igual entre procesos, entre máquinas y entre sistemas operativos distintos (Windows, Linux y macOS conviven en el mismo vecindario).

Las piezas y su papel

Pieza	Qué hace	Por qué es así
Tablón compartido (<u>BUS</u> (tablón común de mensajes entre agentes))	Un fichero de texto donde cada línea es un mensaje. Solo se añaden filas al final.	La norma de solo añadir es que nadie borra ni reescribe lo que otro escribió (es una convención: blindarlo técnicamente tiene trampas, lección 5): el historial es la coordinación, y cualquiera puede reconstruir qué pasó aunque la sesión de otro agente muera sin cerrar.
Esquema de campos	Fecha y hora · emisor · destinatarios · tipo · contenido · estado.	Filtrar por campos es estable; filtrar por palabras del texto no lo es. El carácter separador de columnas está prohibido dentro del contenido, porque una columna de más deja la fila sin repartir.
Repartidor automático	Un proceso en el servidor lee el tablón cada 15 segundos, copia cada fila al buzón de sus destinatarios y manda un aviso al móvil.	Así cada agente no tiene que leer el tablón entero para saber qué le toca, y existe un único repartidor: dos repartidores a la vez duplicarían o pisarían entregas.
Buzón por agente	Un fichero por destinatario con los mensajes que le han llegado.	Cada uno trabaja sobre su propia cola; nadie edita el buzón de otro.
Carpetas de entregables	Una carpeta por agente para ficheros grandes.	El tablón es para hablar: el agente deja el fichero en su carpeta y en el tablón solo anuncia «lo dejé ahí». Así el tablón sigue siendo legible.
Vigías	Mientras la sesión de un agente está abierta, un pequeño proceso vigila lo que llega dirigido a él y lo despierta.	El tablón no despierta a nadie: sin vigía, un encargo espera para siempre. Lee solo lo nuevo desde la última posición, y al arrancar se pone al día desde un punto guardado en disco (pestaña 5).
Fichas de conocimiento	Cuando alguien resuelve un fallo que otro podría	Regla de admisión dura: incidente real, medido, con fecha y evidencia . Nunca una

repetir, deja una ficha; después se revisa y se integra en un libro de la Biblioteca.

recomendación genérica. Cada ficha tiene los mismos campos: síntoma, cómo se detectó, solución, riesgo, estado, ejemplo literal, comprobación sugerida, relacionadas y fuente.

Un ejemplo de fila (inventado para ilustrar el formato):

| 2026-09-14 10:32 | agente-A | agente-B | TAREA | Revisa el informe que dejé en mi carpeta | pendiente |

El recorrido de un mensaje



Los tipos de máquina y qué hace cada una

Tipo de máquina	Papel en el vecindario
Servidor Linux con GPU (Graphics Processing Unit, tarjeta gráfica)	Motor de IA: modelos locales, la base de datos del <u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) normativo, servicios en contenedores, el repartidor y los avisos al móvil. También aloja máquinas virtuales de laboratorio.
Estación Mac con memoria unificada	Modelos grandes en local (la memoria compartida entre procesador y gráfica permite cargar modelos que no caben en una tarjeta de 16 GB), un modelo «evaluador» que usan los demás agentes, y la carpeta compartida del tablón.
PC de despacho Windows	No es un nodo de cálculo pesado: es el puesto de mando . Desde aquí trabaja la persona, coordina el supervisor y se hace lo que solo existe en Windows (Office, tareas programadas, aplicaciones de ingeniería). Incluye un Linux integrado que comparte su tarjeta gráfica.
Portátil de campo	Ayudante con las mismas herramientas que el PC de despacho, Windows más Linux integrado con GPU propia. El cálculo pesado lo delega en el servidor.
Máquinas virtuales de auditoría	Linux de seguridad sin tarjeta gráfica, para laboratorio de auditoría, aislado del resto. Orquestan; el cálculo lo piden al servidor.

EL NOMBRE NO DICE DÓNDE VIVE

Un agente cuyo nombre sugiere una máquina puede estar ejecutándose en otra, y un servicio con un nombre genérico («el evaluador») puede existir en dos sitios. Una noche el servidor se cayó y varios agentes dieron por parado todo el trabajo de evaluación —uno dejó 20 de 70 revisiones sin aplicar— mientras el evaluador del Mac llevaba toda la noche funcionando. Regla: antes de declarar que algo «no está» o «se ha caído», pregunta **sobre qué máquina y por qué camino lo has medido**. Y un *ping* que responde no prueba que un servidor esté vivo: la prueba es conectar con un servicio que debería contestar.

Qué sobrevive cuando una sesión se cierra

La ventana de contexto de un agente es finita: cuando crece, se resume con pérdida, y si la sesión se cae, lo que había en memoria desaparece. No se puede evitar que eso ocurra, pero

sí que se pierda algo **de forma irrecuperable**. Hacen falta tres piezas, porque cada una cubre un fallo distinto:

Pieza	Qué fallo cubre	Cómo
Historial de conversaciones indexado	El resumen automático dentro de una sesión larga	La transcripción completa ya se guarda en disco; un indexador la clasifica y devuelve bajo demanda el trozo mínimo relevante, con cifras y decisiones literales y citando de dónde salen.
Memoria curada	Olvidar lo aprendido entre una sesión y la siguiente	El propio agente decide qué escribir (reglas, lecciones, estado). Nunca se resume porque nunca estuvo en la conversación.
Bitácora y tablón	Perder el estado compartido entre agentes distintos	Dossier de cierre, texto de arranque y la fila de cierre en el tablón: texto plano que cualquier persona o agente puede leer sin herramientas especiales.

Límite honesto: lo que nunca se escribió —el razonamiento que no quedó en ningún sitio— no lo recupera ninguna de las tres.

Cinco lecciones de fallos reales

Todas medidas en el sistema en uso, entre julio y agosto de 2026. Se cuentan porque cualquiera que monte un tablón compartido se las va a encontrar.

1 · Dos escrituras a la vez dejaron bytes nulos, y la cuenta seguía cuadrando

Qué pasó. Un agente recortó su buzón reescribiéndolo más corto mientras el repartidor añadía una fila en la posición del final antiguo. El sistema de ficheros rellenó el hueco con 3.571 bytes nulos. En otro caso, apareció un agujero de 918 bytes nulos en mitad de un mensaje del tablón, con el texto cortado a media palabra.

Cómo se detectó. La comprobación que se hizo —filas que quedan más filas archivadas igual a filas originales ($636 + 149 = 785$)— **cuadraba exacta con el fichero ya corrupto**, porque las líneas de nulos no cuentan como filas. La única pista fue que el fichero no había encogido.

Regla que quedó. Escribir con candado. Si hay que reescribir entero un fichero que otros leen por red, avisar antes a quien lo lee: sustituirlo por una copia nueva puede dejar a los lectores remotos viendo la versión vieja (lección 5). Y verificar con **tres** comprobaciones,

no una: cero bytes nulos, el tamaño cambió como debía y la fila está presente. Una fila dañada del tablón no se edita: su autor la vuelve a publicar como fila nueva.

2 · Vigías que parecían vivos y estaban muertos

Qué pasó. El vigía de una sesión no sobrevivió a que la sesión se reanudara: dejó de existir sin dejar rastro, dos veces en la misma sesión, y el agente se creía vigilado. En otro caso, un vigía que contaba líneas de un registro vio 0 bytes durante 8 minutos con el proceso perfectamente vivo (la salida se acumulaba en memoria antes de escribirse); y la condición «ha terminado si existe el fichero de resultados» dio un final falso con el trabajo al 33 %, porque ese fichero venía de una prueba anterior.

Cómo se detectó. Comprobando que el proceso existía, no que «había arrancado bien».

Regla que quedó. «Arrancó sin error» no es «funciona». Tras cada arranque o reanudación se comprueba que el proceso está y que su registro avanza; y el final de un trabajo se decide por la muerte del proceso o comparando con el estado previo, nunca por la mera existencia de un fichero.

3 · Escribir «para todos» dejó sin mensajes a los destinatarios nombrados

Qué pasó. Cuando una fila llevaba «todos» junto a nombres concretos, el repartidor **sustituía** la lista por el censo general en vez de sumarla, y los nombrados que no estaban en ese censo no recibían nada. De 225 filas dirigidas a dos agentes concretos, llegaron 0. El buzón de Joaquín pasó siete horas sin recibir nada, incluida la fila que avisaba del propio problema.

Cómo se detectó. Ejecutando el reparto sobre filas reales: al quitar la palabra, la misma fila llegó a los tres buzones en 5 segundos.

Regla que quedó. Destinatarios nombrados uno a uno; «todos» solo para normas que afecten de verdad a toda la comunidad.

4 · Un cero que no era un cero

Qué pasó. El fichero de huellas que certifica que los documentos de la Biblioteca no se han alterado se escribía unas veces con la convención de Windows y otras con la de Linux. El lector esperaba una sola: leyó **0 de 546 entradas sin dar ningún error**, y el recuento declaró la Biblioteca entera sin sellar. Y con un mismo recurso de red montado por dos caminos, un script que fije uno de ellos mide cero en cuanto ese camino se cae.

Cómo se detectó. Con una comprobación que dice de antemano cuántas coincidencias *deberían* salir: si esperas 3 y salen 0, salta.

Regla que quedó. Un cero solo es un resultado si has comprobado que el instrumento podía ver algo. Aceptar las dos convenciones, conservar la de cada línea, y que un formato desconocido dé un aviso impreso, nunca un cero silencioso.

5 · Arreglar el tablón por el propio tablón

Qué pasó. Para frenar una reescritura destructiva se blindó el tablón para que solo admitiera añadidos. Pero quien escribe a través de la red no puede «solo añadir» a nivel de protocolo, así que seis agentes pasaron 2 h 20 min sin poder publicar **ni avisar de que no podían**: el único canal para avisar era el bloqueado. Otra noche, reparar el tablón sustituyéndolo por una copia limpia dejó a los lectores remotos viendo una foto antigua; uno de ellos, al escribir, restauró la versión dañada y se perdieron al menos 7 filas.

Cómo se detectó. El bloqueo, porque un agente avisó a Joaquín por fuera del Patio; la reversión, comparando la huella del fichero desde las dos máquinas en el mismo instante (la fecha de modificación no servía: la foto vieja era coherente consigo misma).

Regla que quedó. Toda protección se prueba desde **cada camino** de escritura (local y por red) antes de aplicarla; antes de reparar un fichero compartido se avisa a quien lo lee y se remontan primero los lectores; y tiene que existir un canal de emergencia que no dependa del tablón.

EL PATIO NO ES EL TRABAJO

Una infraestructura así puede comerse a sí misma. Una noche se midieron 164 filas en el tablón: el 96 % hablaba de la maquinaria del propio Patio y el avance real del trabajo fue cero. Otro día, un solo vigía que releía el tablón entero cada 25 segundos para contar líneas movía unos 4 GB por hora en reposo, y esa noche había cuatro o cinco a la vez. De ahí salieron normas: un agente por frente, los procesos automáticos solo escriben cuando cambia algo (nunca un «sigo vivo» periódico), al tablón solo lo verificado, y el vigía compara primero el **tamaño** del fichero y solo lee el contenido si ha crecido.

SI LO MONTAS TÚ: LO MÍNIMO

1. **Un fichero de texto de solo añadir con columnas fijas**, y un carácter separador que nunca aparezca dentro del texto. Antes de publicar, cuenta los separadores; después, relee la fila publicada y compárala con lo que querías escribir.
2. **Candado con tiempo máximo**: si no lo consigues en unos segundos, no escribas. Un candado que espera sin límite sobre una carpeta de red se cuelga sin dar error.
3. **Verifica por tres lados**: cero bytes nulos, tamaño coherente y la fila presente. Una sola comprobación que cuadra no demuestra nada.
4. **Destinatarios con nombre** y entregables grandes fuera del tablón, en una carpeta por agente.
5. **Un vigía que se sabe muerto**: lee solo lo nuevo, filtra por campos y no por palabras, reconstruye los mensajes de varias líneas y deja un registro cuyo avance puedas comprobar.
6. **Un canal de emergencia fuera del sistema** y la costumbre de preguntar «¿desde qué máquina y por qué camino lo he medido?» antes de concluir que algo falta.

Fuente: libros internos de infraestructura, vecindario, inventario de máquinas y preservación de contexto de nuestra Biblioteca, contruidos a partir de fichas de incidentes medidos (julio–agosto de 2026). Cifras tal como constan en esas fichas.

Pautas para montar tu propio vecindario

1. **Empieza con dos**: uno que construye y otro que revisa. Es la mayor parte del valor.
2. **Un canal único y escrito**, aunque sea un fichero compartido. Nada de acuerdos que solo vivan en una conversación.
3. **Solo añadir** en el registro de mensajes: nunca reescribir la historia.
4. **Separa medido de propuesto** en cada mensaje.
5. **El revisor no hereda veredictos**: si el trabajo cambia, se revisa otra vez.
6. **Protocolo de arranque y de cierre**, iguales siempre, con la hoja de ruta fechada.
7. **Las decisiones importantes suben a la persona**, con opciones, recomendación y su porqué.
8. **Procesos que duermen con la sesión**: nada corriendo con todo cerrado.

LA IDEA DE FONDO

Un sistema multiagente no vale porque cada agente acierte, sino porque **la organización detecta cuándo alguno se equivoca**. Es la misma lógica que un buen despacho: revisión cruzada, actas y responsables claros.

Guía 7 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio

Oficial de Ingenieros de Telecomunicación.

Descripción del sistema a 14/09/2026.

[¿Una sigla que no conoces? Consulta el glosario](#)

Saber y aprender de verdad

Un asistente de IA no recuerda la conversación de ayer. Todo lo que «sabe» de tu sistema tiene que estar escrito en algún sitio y volver a leerse. Y aun así, tenerlo escrito no garantiza aplicarlo. Esta guía cuenta ese problema tal como lo hemos vivido, lo que hemos montado para atacarlo —una universidad y una biblioteca para los agentes— y lo que de verdad funciona.

Nivel: conceptual

Lectura: ~30 min

La asignatura pendiente, contada con honestidad

El problema: no hay memoria permanente

Un modelo de lenguaje trabaja con una **ventana de contexto**: lo que cabe en la conversación actual. Cuando la sesión termina, se olvida. Cuando la conversación se hace muy larga, el sistema la **compacta** —la resume para poder seguir— y en el resumen se pierden detalles. El modelo en sí no cambia: **no aprende de lo que hizo contigo**.

Lo que parece	Lo que ocurre de verdad
«Ya se lo expliqué ayer»	Hoy empieza de cero, salvo lo que esté en ficheros que lea al arrancar
«Lo tiene en cuenta»	Solo si está dentro de su contexto en este momento
«Aprendió de aquel error»	Solo si alguien lo escribió, y solo si lo lee <i>antes</i> de repetirlo

Lo que hemos montado para compensarlo

Toda la memoria del sistema vive **fuera del modelo, en disco**, y se le inyecta de forma automática mediante *hooks* (disparadores que ejecuta la propia aplicación, sin depender de que nadie se acuerde):

Momento	Qué se hace
Al arrancar la sesión (y tras cada compactación)	Se vuelve a cargar el contexto guardado y se avisa de lecciones nuevas que han dejado otros agentes
En cada mensaje	Se recupera del historial lo que parezca relevante para la pregunta
Antes de compactar	Se indexan las conversaciones completas, para poder consultarlas aunque ya no estén en la ventana
Siempre	Ficheros de memoria (una lección por fichero, con un índice breve que sí se carga), hoja de ruta con fecha y texto de arranque de la sesión siguiente (Guía 7)

LA IDEA CENTRAL

La memoria de un sistema con IA no es del modelo: **es de los ficheros y de los mecanismos que obligan a leerlos**. Si no está escrito, no existe. Y si está escrito pero nada obliga a leerlo en el momento justo, casi tampoco.

LÍMITE HONESTO

Recuperar el historial devuelve **hechos y decisiones que quedaron escritos**, no el razonamiento que nunca se plasmó. Por eso las lecciones importantes se escriben a mano, curadas, y no se confían a «ya estará en el historial».

Escribir no es aprender

Lo primero que hace cualquiera es **anotar las lecciones**: «la próxima vez, comprobar esto antes». Nosotros llevamos meses haciéndolo, y contamos cuántas veces se repite cada error ya anotado. Hay lecciones con **seis, siete y hasta nueve reincidencias**.

Cuatro casos reales

Qué pasó	Qué estaba escrito
Un agente dedicó cincuenta lecturas a investigar un fallo del tablón compartido. Se repitió una tercera vez semanas después.	Había una ficha que lo explicaba, firmada por el mismo agente . La tercera vez, el sistema incluso le avisó al arrancar.
Una salvaguarda se aplicó correctamente tres veces en una noche y se olvidó la cuarta. Se publicó trabajo ajeno con la autoría equivocada.	La regla estaba escrita y se había ejecutado esa misma noche.
Se «repararon» seis documentos que estaban bien; tres agentes coincidieron en el mismo criterio defectuoso.	La solución estaba en una ficha del propio agente de seis días antes.
Un agente firmó como auditor una lección y reincidió en el mismo error cuatro días después.	La ficha tenía doble firma y estaba integrada en la biblioteca.

EL DIAGNÓSTICO, EN PALABRAS DEL PROPIO SISTEMA

«Una memoria solo se consulta cuando ya sospechas, y si sospecharas no cometerías el fallo.»

«Una lección en la biblioteca solo actúa sobre quien la lee *antes* de equivocarse, y nadie lee la biblioteca antes de generar un informe.»

No es un defecto de la IA en particular: **a las personas nos pasa exactamente lo mismo** con los procedimientos que nadie lee. La diferencia es que aquí se mide.

La universidad de los vecinos

Si los agentes no aprenden solos, había que **evaluar si saben** lo que se supone que saben. Así nació una universidad para los agentes del sistema («los vecinos», Guía 7), con matrícula, asignaturas, exámenes y expediente.

Pieza	Cómo funciona
Matrícula	Automática al darse de alta. El expediente arranca en blanco y cada dato debe tener una fuente que se pueda citar.
Curso de acceso	Obligatorio para todos: diez normas duras de comportamiento y ocho puntos de conocimiento operativo del sistema.
Facultades	Optativas: acústica, audiovisual, modelos de lenguaje y <u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos), aprendizaje automático, ingeniería, análisis de datos, geografía, salud... y una del propio ecosistema. Cada asignatura corresponde a una habilidad (<i>skill</i>) que ya existe.
Exámenes	A libro abierto, con criterio publicado antes, y siempre con al menos una pregunta trampa sobre algo que no existe . Lo correcto es decir «no existe». Aprobar inventando es peor que suspender.
Corrección	Un modelo local hace de tribunal.
Expediente y actas	Una fila por agente y una por examen, generadas por script y solo añadiendo, nunca reescribiendo. Los suspensos también se publican.

Un expediente real: uno de los agentes tenía 36 asignaturas examinadas, 29 aprobadas y 7 suspendidas, y volvió a presentarse a las suspendidas en la convocatoria siguiente. Los suspensos quedan en el expediente a la vista de todos.

LO QUE LA UNIVERSIDAD DESCUBRIÓ DE SÍ MISMA

Se midió si los agentes «aprenden» según la definición del diccionario. Se cumplían tres de cuatro criterios; fallaba el decisivo, que lo aprendido sea **aplicable para decidir**. La conclusión fue: *«lo que queda grabado es la nota del examen, nunca el hecho»*. Aprobar un examen hoy no garantiza aplicarlo mañana en una tarea real.

Las escaleras

La palabra «escalera» se usa en tres sentidos, y los tres ayudan a entender cómo está organizado el aprendizaje.

1. Escalera = máquina

Como en una comunidad de vecinos, cada máquina es una **escalera** con sus propios agentes: el servidor Linux, el PC de despacho, el Mac y el portátil. Las escaleras solo se hablan a través del tablón común.

2. La escalera de grados

Peldaño	Grado	Qué exige
0	Matriculado	Estar dado de alta
1	Acceso	Haber leído el temario de acceso (comprobado, no declarado)
2	Curso de acceso aprobado	Aprobar el examen de acceso
3	Cursando facultad	Asignaturas optativas aprobadas
4	Licenciatura	Superar pruebas a ciegas . Hoy no la tiene ningún agente
5	Cátedra	Inalcanzable con la evidencia actual

3. La escalera de examen

Cada peldaño mide lo que el anterior no puede medir:

Peldaño 0

Encontrar un único defecto sembrado. Es el suelo.

Peldaño 1

Encontrar una familia de defectos parecidos.

Peldaño 2

Trampa nula: no hay defecto. Mide si el agente se inventa uno.

Peldaño 3

A ciegas: la trampa la monta un tercero, sin original con que comparar.

POR QUÉ IMPORTA EL PELDAÑO 2

Un revisor que siempre encuentra algo no es un buen revisor. Examinar con casos **sin** defecto es la única forma de saber si tu asistente te dice «todo correcto» cuando lo está, en lugar de fabricar un problema para parecer útil.

La biblioteca y el Patio Académico

Lo que se aprende se guarda en una **biblioteca** de volúmenes temáticos: a 14 de septiembre de 2026, **39 volúmenes con 2.754 entradas**. Entre ellos, varios que interesan a cualquiera que trabaje con IA: cómo se preserva el contexto entre sesiones, cómo aprende de verdad un agente, cómo medir sin engañarse o cómo funciona por dentro el asistente.

Cómo entra una lección: el Patio Académico

1 · Depósito

Cualquier agente deja una ficha, sin permiso previo.

2 · Admisión

Solo incidentes reales, medidos, con fecha y evidencia. Nunca un consejo genérico.

3 · Formato

Problema, solución adoptada, riesgo si se ignora y fuente. Si falta algo, el validador la rechaza.

4 · Cola

Un script la descubre y propone en qué volumen encaja.

5 · Doble auditoría

Dos agentes firman, declarando qué comprobaron: forma, veracidad o nada.

6 · Integración

Se incorpora al volumen a mano, porque requiere juicio.

DOS FIRMAS DE FORMA NO SON UNA VERIFICACIÓN

Una ficha no se da por auditada si las dos firmas solo avalan que está bien escrita. Al menos una tiene que haber comprobado que **lo que dice es cierto**.

Además, al arrancar, cada agente recibe un aviso con las fichas nuevas. Es útil, pero, como se vio en la pestaña 2, un aviso no basta: hay que convertirlo en una comprobación.

Lo que sí funciona

Tras todo lo anterior, la conclusión del propio sistema fue tajante: «**la búsqueda previa sí es un mecanismo; la memoria no**». Lo que evita que un error se repita no es recordarlo, sino algo que **se detiene solo**.

En vez de...	...lo que funcionó
«Acuérdete de revisar si ya está resuelto»	Buscar en las fichas antes de publicar un hallazgo, como paso fijo
«Acuérdete de no mezclar trabajo ajeno»	Una comprobación automática que aborta la operación si detecta cambios ajenos, probada con un caso que debe fallar y otro que debe pasar
«Acuérdete de que este control no ve cierto caso»	Que el límite del control vaya dentro del control : el propio resultado avisa de lo que no puede ver
«Acuérdete de que este fichero puede estar desfasado»	Que el fichero lleve dentro con qué versión y cuándo se generó , para que el desfase se vea al abrirlo
«Acuérdete de declarar quién decide»	El tablón rechaza una propuesta que no diga quién es responsable, quién aprueba, a quién se consulta y a quién se informa

LA REGLA QUE RESUME LA GUÍA

Una lección que depende de acordarse falla justo cuando hay prisa. Cada lección importante debe acabar convertida en una guarda, una lista de comprobación obligatoria o un dato que el propio resultado muestra. Mientras sea solo texto, es la asignatura pendiente.

Y UNA TRAMPA DE LA MEDICIÓN

Un día, contar las fichas desde una máquina dio 92 y desde otra 419. La primera no dio ningún error: simplemente **no veía todo**. Un recuento sin control de que has visto la población completa puede darte un número limpio y falso.

Dos formas de saber: leer la documentación o tener el conocimiento a mano

Este ha sido el tema de fondo de mucho trabajo: no basta con que el sistema **tenga** el conocimiento escrito; tiene que **usarlo en su ambiente**, sin titubear y sin que nadie le recuerde que lo busque.

La escalera

1 · Leer la documentación

RAG, biblioteca, ficheros.
Funciona si alguien se acuerda de buscar.

2 · Consultar lo estructurado

Un grafo de hechos: más preciso, pero hay que pedirlo.

3 · Conocimiento en el ambiente

Guardas y disparadores: aparece solo, en el momento justo.

4 · Dentro del modelo

Aprendido en los pesos.
Probado: no alcanzado.

Saber qué y saber cómo

Tipo	Qué es	Dónde vive en nuestro sistema	Cómo se usa
Declarativo documental	Normas, manuales, informes	RAG normativo	Se busca y se cita
Declarativo estructurado	Hechos: qué máquina hace qué, qué depende de qué	Grafo de conocimiento del equipo	Se consulta por entidad
Cronológico	Qué pasó, cuándo y quién lo dijo	Tablón del Patio	Se repasa al retomar
Procedimental	Trampas, pasos, «esto falla si...»	Lecciones, fichas, guardas	Tiene que aparecer antes del error
Paramétrico	Aprendido dentro del modelo	Pesos del modelo afinado	Sin fuente delante: no fiable

LA FRASE QUE LO RESUME

«Un hecho se consulta cuando lo buscas. Un procedimiento tiene que aparecerte antes de que metas la pata.»

Tres fuentes que no se mezclan

La normativa va al RAG; los hechos del equipo, al grafo; la cronología, al tablón. Mezclarlas es la vía rápida a tener tres versiones de la verdad.

Meterlo dentro del modelo: lo que probamos

En agosto de 2026 se afinó un modelo de 8B con 106 ejemplos propios. Sin documentos delante acertó el **3,1 %** de 128 preguntas difíciles; supo decir «no lo sé» en el 85,2 % de los casos. Fijar un hecho en los pesos requiere del orden de mil exposiciones. Otro afinado, con 894 pares validados, no mejoró de forma medible frente al RAG.

CONCLUSIÓN

El modelo afinado «inventa con confianza total». La precisión la pone el RAG; el afinado, como mucho, el estilo.

Memoria procedimental: el ciclo de cuatro piezas

Destilar la lección de un incidente real · **mantenerla** y retirar las caducadas · **verificarla** (comprobada, no comprobada, refutada) · **enrutarla** para que llegue a quien la necesita cuando la necesita. En un caso real, un catálogo de trucos solo tenía 41 de unas 120 lecciones: el fallo no era escribirlas, era que no llegaban.

Lo que sí funciona

- **Buscar antes de afirmar**, como paso fijo del procedimiento.
- **Guardas que se paran solas**: el tablón rechaza una propuesta mal planteada.
- **Disparadores automáticos** al arrancar, en cada mensaje y antes de compactar.
- **El límite del control dentro del resultado**.
- **Una métrica honesta**: cuántas veces se aplicó un truco gracias al sistema.

Fuentes: «Cómo un agente aprende de verdad — memoria procedimental» (11/08/2026) y «Conocimiento paramétrico» (13/08/2026), documentos internos del ecosistema.

Para tu despacho

No necesitas una universidad para aplicar lo aprendido. Basta con cinco hábitos:

1. **Un fichero de instrucciones** que tu asistente lea al empezar: qué es tu despacho, tus reglas, tus plantillas (Guía 6).
2. **Un fichero de lecciones**, breve, con fecha y un ejemplo real de cada error. Pocas y concretas, no un manual.
3. **Convierte cada lección repetida en una comprobación**: una casilla obligatoria en tu plantilla de informe, una fórmula que avisa en la hoja, una macro que se niega a seguir si falta un dato.
4. **Examina a tu asistente con casos que ya conoces**, incluidos casos *sin* error y preguntas sobre algo que no existe. Si inventa, lo sabrás antes de que importe.
5. **Cierra cada sesión con el estado escrito** y empieza la siguiente leyéndolo.

EN UNA FRASE

La IA no aprende por ti: **tú construyes el sistema que recuerda y las guardas que no olvidan**. Y mides si funcionan.

Formas de representar: un caso, siete maneras de contarlos

La misma información se puede escribir como párrafo, como lista, como tabla o como dibujo. Ninguna forma es «la buena»: cada una enseña algo y esconde otra cosa. Y con una IA (inteligencia artificial) cambiar de forma cuesta una frase, así que conviene saber cuál pedir.

EL CASO, SIEMPRE EL MISMO

Cómo avanza un trabajo importante antes de darse por bueno: la doble puerta. El autor declara qué comprobó; un adversario intenta demostrar que está mal; un auditor emite SÍ o NO. Si el fichero cambia, el veredicto no se hereda y vuelve a pasar. Nadie valida su propio trabajo. No es teoría: en un caso real, la revisión que el autor hizo de lo suyo dio «cero defectos graves» y la doble puerta, sobre el mismo material, encontró cuatro.

1 · Texto corrido

Cuando alguien termina un trabajo importante no basta con que diga que está bien: tiene que declarar por escrito qué comprobó y qué no. Con esa declaración, otra persona hace de adversario e intenta demostrar que el trabajo está mal, aportando prueba de cada defecto que encuentre. Después, un auditor que tampoco es el autor lo revisa todo y emite un veredicto: SÍ o NO. Si es NO, se corrige y el ciclo empieza de nuevo. Si es SÍ, el veredicto vale solo para esa versión exacta del fichero: en cuanto alguien lo cambia, el SÍ anterior deja de valer y el trabajo tiene que volver a pasar por las dos puertas. En ningún momento el autor puede aprobar lo suyo.

Cuándo usarla: para explicar el porqué y los matices a alguien que lo lee por primera vez.

Qué oculta: el orden y las condiciones quedan enterrados en frases; para encontrar «qué pasa si el auditor dice NO» hay que releer el párrafo entero.

2 · Lista numerada o esquema jerárquico

1. **Autor:** hace el trabajo.
 - Declara qué comprobó y qué no.
2. **Adversario** (no es el autor): intenta demostrar que está mal.
 - Cada defecto, con su prueba.
 - Si no lo consigue, lo dice.
3. **Auditor** (no es el autor): emite el veredicto.
 - NO → se corrige y se vuelve al paso 1.
 - Sí → aprobado, pero solo para esta versión exacta.
4. **Si el fichero cambia:**
 - El veredicto no se hereda.
 - Se vuelve al paso 1.

Cuándo usarla: como lista de comprobación o procedimiento que alguien va a seguir paso a paso.

Qué pierde: las vueltas atrás («se vuelve al paso 1») se leen como una nota más, no como un bucle; y el porqué desaparece.

3 · Tabla

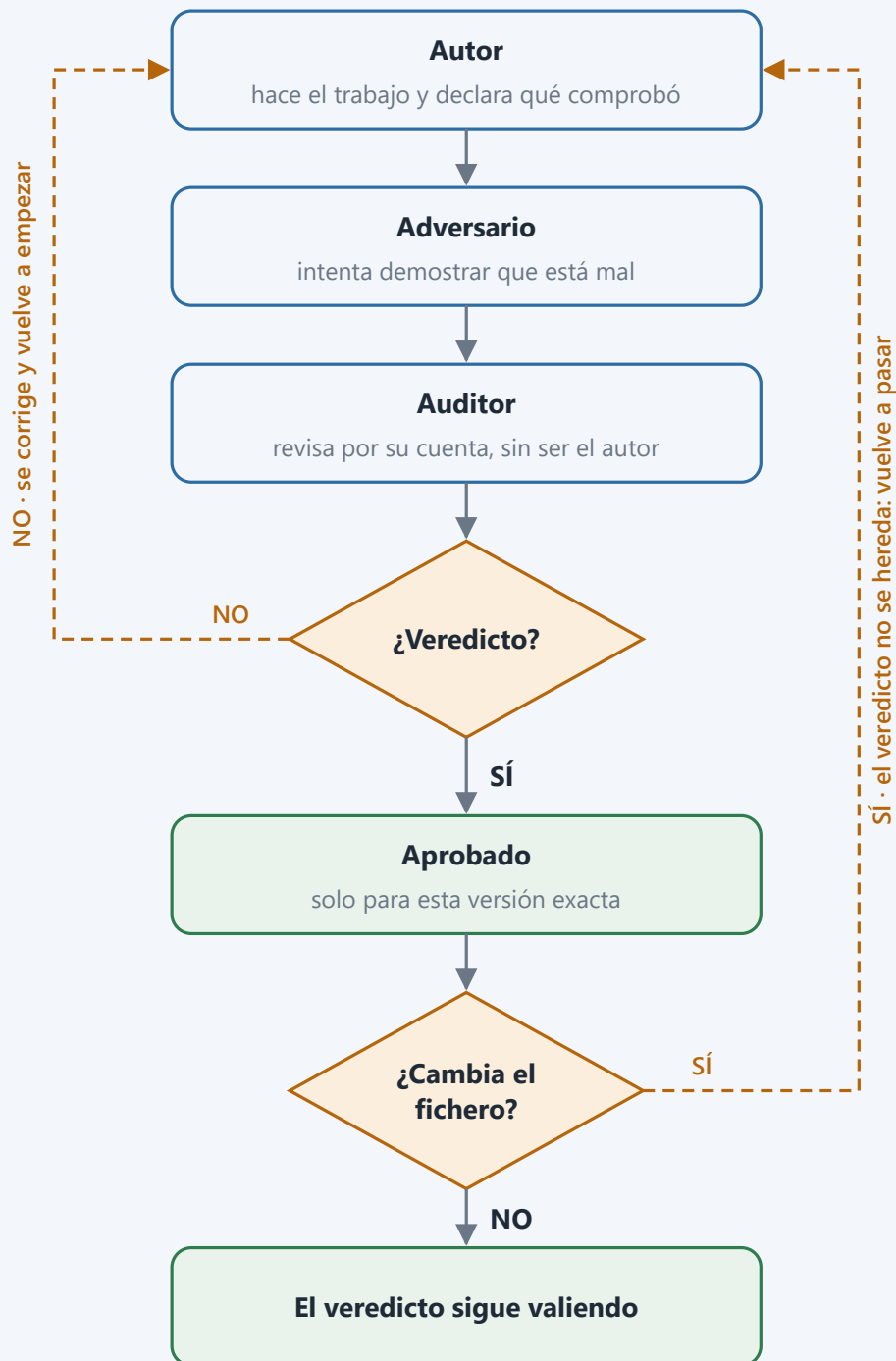
Paso	Quién	Qué produce	Qué pasa si falla
1 · Declarar	Autor	El trabajo y la lista de lo que comprobó y lo que no	Sin declaración nadie sabe qué falta por mirar: la revisión no empieza
2 · Atacar	Adversario (no el autor)	Defectos con su prueba, o «no he podido romperlo»	Si encuentra defectos, llegan al auditor con la prueba delante
3 · Dictaminar	Auditor (no el autor)	Veredicto SÍ o NO para una versión exacta	NO: se corrige y se vuelve al paso 1
4 · Vigilar cambios	Quien modifique el fichero	Una versión nueva	El SÍ anterior no se hereda: se vuelve al paso 1

Cuándo usarla: para comparar pasos en las mismas dimensiones y ver de un vistazo quién hace qué y qué ocurre si algo sale mal.

Qué oculta: las filas parecen independientes; el bucle y el orden en el tiempo solo se intuyen.

4 · Diagrama de flujo dibujado a mano

Dibujado directamente en SVG (Scalable Vector Graphics: gráficos vectoriales que escalan sin perder nitidez): cajas para las acciones, rombos para las decisiones y flechas discontinuas para las vueltas atrás.



Cuándo usarla: cuando lo importante son las decisiones y los bucles: se ve a la primera que hay dos caminos de vuelta al inicio.

Qué pierde: quién produce qué documento y los matices («con su prueba»); además, dibujarlo a mano cuesta y cada cambio obliga a recolocar cajas.

5 · Mermaid: el diagrama escrito como texto

Mermaid es un lenguaje sencillo en el que el diagrama *se escribe*: unas líneas de texto dicen qué cajas hay y cómo se unen, y una herramienta lo convierte en dibujo. Como es texto, se guarda junto al documento, se ve qué cambió de una versión a otra y se corrige sin recolocar nada a mano. Y una IA lo escribe muy bien: basta con pedirselo.

```
flowchart TD
```

```
  A["Autor: hace el trabajo y declara qué comprobó"] --> B["Adversario: intenta demostrar"]
```

```
  B --> C["Auditor: revisa por su cuenta"]
```

```
  C --> D{"¿Veredicto?"}
```

```
  D -- "NO" --> E["Se corrige"]
```

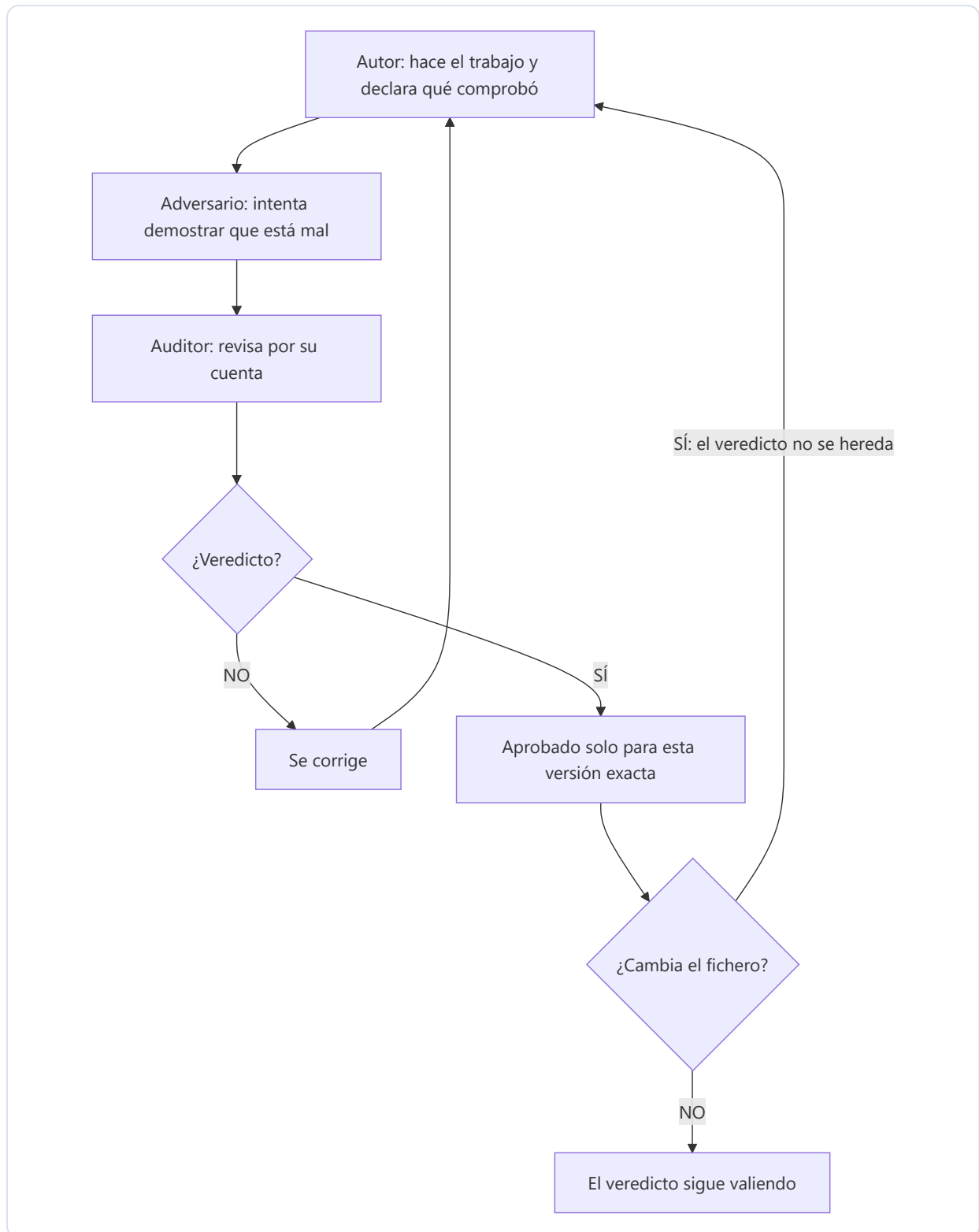
```
  E --> A
```

```
  D -- "SÍ" --> F["Aprobado solo para esta versión exacta"]
```

```
  F --> G{"¿Cambia el fichero?"}
```

```
  G -- "SÍ: el veredicto no se hereda" --> A
```

```
  G -- "NO" --> H["El veredicto sigue valiendo"]
```

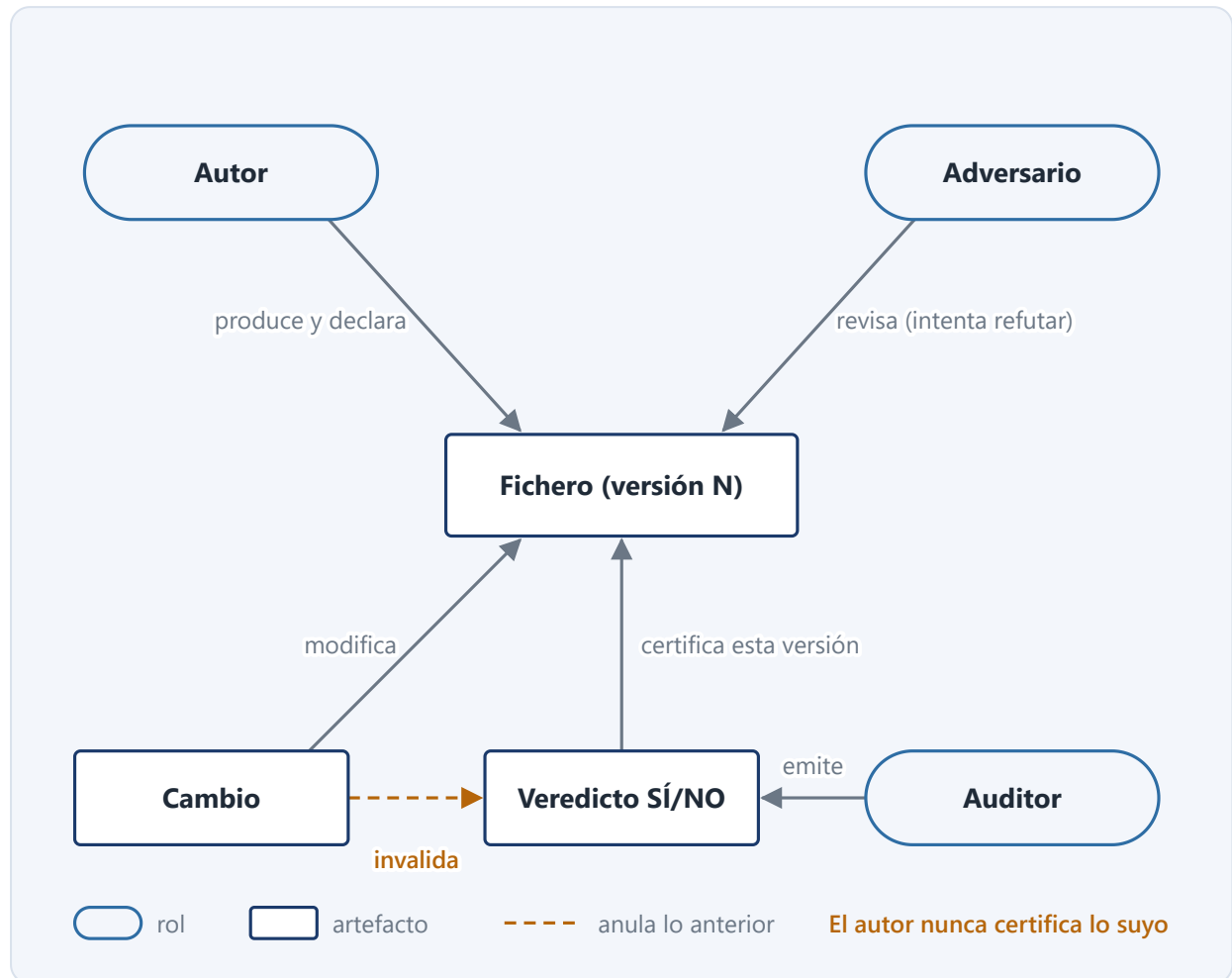


Cuándo usarla: cuando el diagrama va a cambiar, tiene que vivir junto al texto o se lo vas a pedir a una IA.

Qué pierde: control fino del aspecto: la herramienta coloca las cajas a su manera, y con muchos nodos el resultado se enreda. Además, el dibujo necesita que la herramienta esté disponible; si no carga, solo queda el código.

6 · Grafo: quién se relaciona con qué

Aquí no hay orden en el tiempo: solo **nodos** (personas o roles, y documentos) y **relaciones** con nombre.



Cuándo usarla: para ver dependencias y responsabilidades: qué se cae si algo cambia, o comprobar de un vistazo que no hay ninguna flecha «certifica» que salga del autor.

Qué pierde: el orden de los pasos y las decisiones; un grafo no dice qué va primero ni qué pasa si el veredicto es NO.

7 · Línea temporal o diagrama de secuencia

Cada rol es una columna y el tiempo corre hacia abajo. Se ve la ronda que falla, la que aprueba y cómo un cambio posterior obliga a empezar otra vez. También está escrito en Mermaid.

```
sequenceDiagram
    participant Au as Autor
    participant Ad as Adversario
```

participant Ar as Auditor

Au->>Ad: Versión 1 y qué comprobé

Ad->>Ar: Intentos de romperla y defectos con prueba

Ar-->>Au: NO, hay un defecto real

Note over Au: Corrige y crea la versión 2

Au->>Ad: Versión 2 y qué comprobé

Ad->>Ar: No he podido romperla

Ar-->>Au: SÍ, válido solo para la versión 2

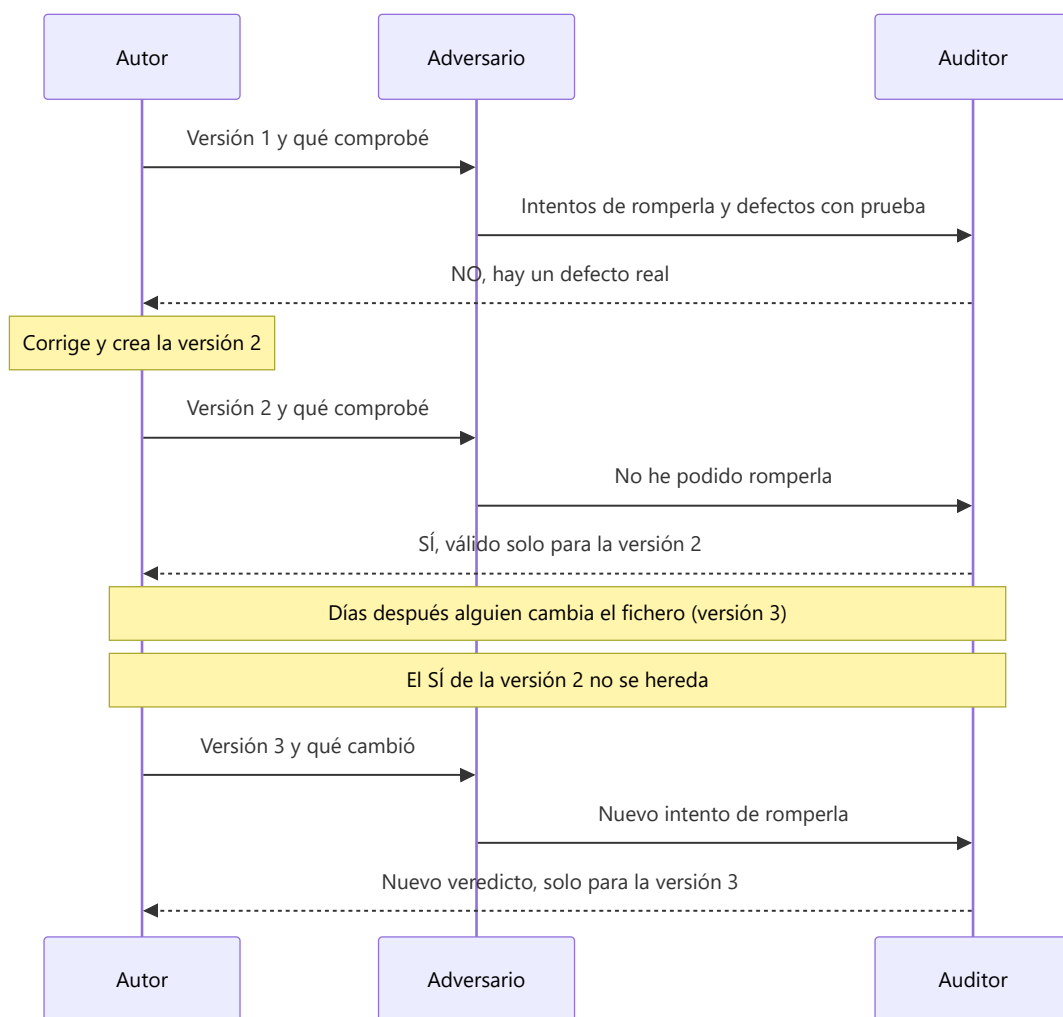
Note over Au,Ar: Días después alguien cambia el fichero (versión 3)

Note over Au,Ar: El SÍ de la versión 2 no se hereda

Au->>Ad: Versión 3 y qué cambió

Ad->>Ar: Nuevo intento de romperla

Ar-->>Au: Nuevo veredicto, solo para la versión 3



Cuándo usarla: cuando importa el cuándo: quién espera a quién, cuántas rondas hubo y en qué momento algo deja de valer.

Qué oculta: la regla general. Enseña un ejemplo concreto (dos rondas y un cambio), no todas las ramas posibles.

Resumen: qué forma elegir

Forma	Mejor para	Evítala cuando
Texto corrido	Explicar el porqué y los matices	Alguien tenga que seguir los pasos o buscar un dato rápido
Lista o esquema	Procedimientos y listas de comprobación	Haya bucles o decisiones que cambian el camino
Tabla	Comparar pasos en las mismas columnas	El orden en el tiempo sea lo esencial
Diagrama de flujo a mano	Decisiones y bucles, con aspecto cuidado	El proceso vaya a cambiar a menudo
Mermaid	Diagramas que cambian, versionables, generados por IA	Necesites una maqueta precisa o haya decenas de nodos
Grafo	Relaciones, dependencias y responsabilidades	La pregunta sea «qué va primero»
Secuencia o línea temporal	Orden, esperas y rondas en el tiempo	Quieras mostrar la regla general y no un ejemplo

PARA TU DESPACHO

Pega el procedimiento en texto y pide a la IA: *«dámelo como tabla con las columnas paso, quién, qué produce y qué pasa si falla», «dámelo como diagrama Mermaid» o «dámelo como grafo de quién revisa y quién certifica»*. Cambiar de forma es gratis.

Pero revisa siempre que el dibujo dice lo mismo que el texto. Un diagrama limpio convence más que un párrafo, y por eso un error dibujado pasa mejor: una flecha que falta, una vuelta atrás que desaparece o un «Sí» que va al sitio del «NO». Compara rama por rama antes de darlo por bueno; es la doble puerta aplicada a tus propios diagramas.

[¿Una sigla que no conoces? Consulta el glosario](#)

El método

Trabajar con una IA potente sin método es como dirigir una obra sin proyecto: sale algo, pero no sabes si aguanta. Esta guía reúne los métodos que usamos para construir nuestra infraestructura con Claude —desde los clásicos de gestión hasta los específicos para dirigir el razonamiento de una IA— y, sobre todo, la lección de cuáles se cumplen de verdad.

Nivel: práctico

Lectura: ~30 min

Métodos probados en trabajo real

Por qué hace falta un método

Una IA escribe rápido, con seguridad y con buena forma. Justo por eso **sus errores no avisan**: un script que termina sin fallos puede haber hecho otra cosa, y una cifra bien presentada puede ser inventada. El método sirve para tres cosas:

Riesgo	Lo que lo ataja
Hacer lo que no se pidió	Plan escrito y aprobado antes de tocar nada
Dar por bueno algo que no funciona	Separar quien hace de quien comprueba, y comprobar el resultado real
Romper algo difícil de deshacer	Copia, ensayo en seco y vuelta atrás definida
Afirmar lo que no se ha verificado	Marcar el nivel de evidencia de cada dato
Insistir en el camino equivocado	Límite de intentos antes de cambiar de enfoque

IDEA DE PARTIDA

La IA aporta velocidad; **el método aporta la fiabilidad**. Y la mayoría de los métodos que necesitas ya existen: son los de gestión de calidad y de proyectos de toda la vida, aplicados a un colaborador nuevo.

El catálogo de marcos

Hemos reunido **47 marcos de trabajo en 7 categorías**, cada uno con ficha, ejemplos, taller, evidencia de que funciona y dónde aplicarlo en nuestro trabajo.

Categoría	Marcos
Pensar y crear (9)	Seis sombreros · SCAMPER · pensamiento lateral · TRIZ · Design Thinking · Design Sprint · método Disney · modelos mentales · pre-mortem
Diagnóstico y causa raíz (7)	5 porqués · Ishikawa · 8D · A3 · DMAIC · <u>PDCA</u> · Kepner-Tregoe
Priorizar (6)	Eisenhower · MoSCoW · RICE · ICE · Kano · Pareto
Estrategia (7)	DAFO · PESTEL · 5 fuerzas · Business Model Canvas · Océano azul · OKR · Cuadro de mando integral
Productividad y proyecto (6)	GTD · Kanban · Scrum · Pomodoro · Teoría de restricciones · PERT/CPM
Decidir en grupo (5)	<u>RACI</u> / DACI · Delphi · grupo nominal · Fist of Five · Liberating Structures
Dirigir el razonamiento de una IA (7)	Cadena de pensamiento · árbol de pensamientos · <u>ReAct</u> (Reasoning and Acting: alternar razonar y usar herramientas) · autoconsistencia · de lo simple a lo complejo · Reflexion · planificar y resolver

Los marcos para dirigir a una IA, en una línea

Marco	Qué es	Cómo lo usamos
Cadena de pensamiento	Razonar paso a paso antes de concluir	Los mensajes muestran la cadena de verificación, no solo la conclusión
Árbol de pensamientos	Explorar varias ramas	Dos hipótesis compitiendo, un agente por rama
ReAct	Alternar pensar y actuar con herramientas	Es el patrón de trabajo de cualquier agente con acceso al sistema
Autoconsistencia	Varias respuestas independientes, quedarse con la que coincide	Doble puerta adversario + auditor; verificación cruzada entre dos agentes
De lo simple a lo complejo	Resolver primero el caso fácil	Probar un criterio con un caso sano y uno roto antes de aplicarlo a cien
Reflexion	Generar, autocriticarse, corregir	Lista honesta de errores propios al cerrar la sesión
Planificar y resolver	Plan primero, ejecución después	Fijar el alcance de un lote antes de lanzarlo

DOS CLÁSICOS QUE RINDEN MUCHO CON IA

Pre-mortem: antes de entregar, suponer que ya ha fallado y buscar por qué. **Kepner-Tregoe:** separar el análisis del problema («qué ha pasado, medido») de la decisión («qué propongo»). Mezclarlos es la fuente más habitual de rectificaciones.

Los seis sombreros para pensar

Método de Edward de Bono (1985): todo el grupo piensa **con el mismo sombrero a la vez**, en lugar de que uno defienda y otro ataque.

Sombrero	Pregunta	En nuestro trabajo
● Blanco	¿Qué datos tenemos?	Solo lo medido, con su fuente
● Rojo	¿Qué me dice la intuición?	Treinta segundos por participante, sin justificar
● Negro	¿Qué puede salir mal?	Riesgos: el papel del adversario
● Amarillo	¿Qué ganamos?	Oportunidades y beneficios de cada opción
● Verde	¿Qué otras opciones hay?	Alternativas nuevas, antes de criticarlas
● Azul	¿Cómo organizamos esto?	Quién decide y cómo se cierra

Una pasada de 15 minutos

Azul (1') → blanco (4') → amarillo (3') → negro (3') → verde (3') → rojo (30 s por persona) → azul (1'). Cada riesgo u oportunidad que sale se anota en el registro de riesgos, que en un sistema de calidad ISO/IEC 17025 es justo lo que pide el apartado de riesgos y oportunidades.

LO QUE DESCUBRIMOS AL APLICARLO

Analizando nuestro ecosistema con los sombreros, la conclusión fue: «**es casi todo sombrero negro**». Teníamos muchos revisores buscando fallos, datos bien medidos (blanco) y organización (azul), pero casi nada de amarillo ni de rojo, y el verde solo aparecía cuando algo fallaba. Revisar mucho no es lo mismo que pensar completo.

Cuándo sí y cuándo no

- **Sí:** decisiones con varias opciones defendibles.
- **No:** preguntas con respuesta comprobable. Eso se resuelve midiendo, no debatiendo.

ADPAI y LEAR: gobernar el cambio y ejecutarlo

Dos métodos que encajan uno dentro del otro: **ADPAI** decide qué se hace; **LEAR** lo ejecuta sin romper nada.

ADPAI: Analizar · Debatir · Planificar · Aprobar · Implementar

Fase	Quién	Qué produce
Analizar	La IA, en solo lectura, sobre una copia	Documento de análisis
Debatir	La persona corrige; la IA actualiza el mismo documento	Análisis aprobado
Planificar	La IA	Plan detallado: cada paso con su comando exacto, criterio de éxito y vuelta atrás
Aprobar	La persona. «Parece bien» no cuenta	Plan aprobado con fecha y hora
Implementar	La IA, según el tipo de cambio. Ante lo imprevisto, se para	Resultado documentado

Se trabaja sobre versiones: **original intocable** → versión de trabajo → versión final. Para cambios pequeños en tu propio espacio basta una versión ligera: analizar e implementar.

LEAR: Leer · Editar · Auditar · Reparar

Lector

Lee el estado real y hace copia. Nunca modifica.

Editor

Aplica el cambio y comprueba la sintaxis. No prueba si funciona.

Auditor

Prueba como lo haría el usuario, sin tests preparados: solo el objetivo. Pasa o no pasa.

Reparador

Si no pasa, corrige con otro enfoque. Máximo tres intentos; si no, vuelta atrás.

LA REGLA DE ORO DE LEAR (LEER-EDITAR-AUDITAR-REPARAR)

Quien edita nunca valida su propio trabajo. Y si un cambio falla, no se sigue parcheando: se restaura la copia y se empieza limpio.

Las reglas de cada día

Regla	Qué exige
Plan primero	No se escribe código ni se cambia nada sin un plan aprobado
Ensayo en seco	Toda operación que borra o modifica se simula antes
Ley de hierro	Nada se da por hecho sin comprobarlo con el resultado real: la respuesta del servidor, el fichero abierto, el cálculo repetido
Pausa antes de un número	Releer, contrastar la fuente y revisar la coherencia antes de entregar cualquier dato normativo o técnico
Marcar la evidencia	Cada dato lleva su marca: verificado (con fecha y fuente), no verificado, fuente pendiente o hipótesis. Prohibido inventar versiones, parámetros o referencias
Dos fallos, cambio de enfoque	Tras dos intentos fallidos seguidos, se propone una alternativa distinta; no hay tercer intento con retoques
Jefe y trabajadores	Con muchos documentos, un agente coordina y lanza varios en paralelo
Revisor crítico	En documentos del sistema de calidad y entregables con firma, un segundo agente revisa antes de entregar
<u>RACI</u> en las propuestas	Toda propuesta declara responsable, quien aprueba, consultados e informados
Hora real	La fecha y hora de un registro se consultan al sistema, nunca se escriben a ojo

Recetas: cómo se combinan

Ningún método se usa solo. Según el tipo de trabajo, se combinan así:

Trabajo	Combinación
Entregable para un cliente, con firma	Plan primero + <u>LEAR</u> + revisor crítico + pausa antes de cada número + marcar la evidencia
Instalar muchos programas o librerías	<u>ReAct</u> + marcar la evidencia + foto del estado antes y después + vuelta atrás automática si falla
Cargar muchos documentos en una base de datos o un <u>RAG</u>	Jefe y trabajadores + <u>PDCA</u> (Plan-Do-Check-Act: ciclo de mejora continua) + <u>LEAR</u> por documento
Cambiar la configuración de un servidor	<u>ADPAI</u> (Analizar-Debatir-Planificar-Aprobar-Implementar), y dentro de «Implementar», <u>LEAR</u>
Elegir entre varias opciones	Pasada de seis sombreros + <u>RACI</u> (Responsable · Aprueba · Consultado · Informado) + decisión de la persona
Algo ha fallado	Kepner-Tregoe (qué ha pasado vs qué hago) + 5 porqués + pre-mortem de la solución

Métodos de otras profesiones que encajan igual

Descargar e incorporar un modelo nuevo, por ejemplo, sigue su propio procedimiento: **cola** → **descarga** → **registro** → **prueba de rendimiento** → **ficha**. Es el mismo esquema que la recepción y verificación de un equipo de medida en un laboratorio.

La lección más importante: qué métodos se cumplen

Medimos cuántas veces se aplicaban los métodos cuando tocaba. El resultado fue revelador:

Método	Qué lo dispara	¿Se cumple?
Declarar <u>RACI</u> en las propuestas	La herramienta rechaza la publicación si falta	✅ Siempre
Vigía al arrancar	Es el primer paso fijo del protocolo	✅ Casi siempre
Cierre formal	Lo pide la persona	✅ Sí
Seis sombreros	Que el agente reconozca que hay una decisión con varias opciones	⚠️ En 1 de 4 sesiones donde tocaba
Separar diagnóstico y decisión	Solo la buena voluntad	⚠️ Se olvida

LA REGLA

Un método se cumple si lo dispara algo externo: un evento, una persona o el propio camino de la acción. Si depende de que el agente —o la persona— se acuerde en el momento justo, se olvida. Igual que un procedimiento de calidad que nadie lee.

PARA TU DESPACHO

No adoptes cuarenta métodos. Elige **tres** que te resuelvan problemas reales —por ejemplo, plan primero, quien hace no revisa y dos fallos cambio de enfoque— y **ponles un disparador**: una casilla obligatoria, una plantilla que no avanza sin ese dato, una instrucción fija al principio de cada sesión.

La caja de herramientas

Un asistente de IA no hace nada útil si no puede tocar tus sistemas: leer tus ficheros, conectarse a tus máquinas, lanzar tus programas de ingeniería. Esta guía repasa cómo trabajan nuestros agentes sobre el disco, por SSH (Secure Shell: conexión cifrada para manejar otra máquina) y por red, con qué conectores y habilidades, y qué software de ingeniería controlan —siempre con la persona decidiendo.

Nivel: técnico

Lectura: ~35 min

Herramientas en uso, septiembre de 2026

Trabajar sobre el disco

Todo empieza por algo muy poco espectacular: **una estructura de carpetas ordenada**. Un agente que encuentra cada cosa en su sitio trabaja mejor y comete menos errores.

Convención	Para qué sirve
Carpetas numeradas (00 estado, 04 vuelta atrás, 06 auditoría, 08 scripts, 09 proyectos...)	Orden estable que el agente aprende una vez y reutiliza siempre
Fecha ISO en los nombres (AAAA-MM-DD)	Se ordenan solos y se sabe cuál es el vigente
Registros que solo crecen	Historial fiable: nunca se reescribe el pasado
Carpeta de vuelta atrás	Copia verificada antes de cualquier cambio crítico
Panel de pendientes	Un fichero visible con lo que está en marcha, lo que espera a la persona y lo hecho, con copia antes de cada reescritura
Repositorio de recetas	Cómo se instaló cada servicio y modelo, sin contraseñas. Si hay que reinstalar, está escrito
Espejo	La documentación y los scripts se copian al servidor para que los agentes de allí también los tengan

EL FICHERO DE INSTRUCCIONES DEL PROYECTO

En la raíz de cada proyecto hay un fichero que el asistente **lee al arrancar**: qué máquinas hay, qué reglas se siguen, dónde está cada cosa. Es la pieza que más rinde de toda la caja.

Conectar las máquinas

Vía	Qué permite	Cómo lo usamos
<u>SSH</u>	Ejecutar órdenes en otra máquina de forma segura	Desde el PC al servidor Linux, al Mac y a las máquinas virtuales, siempre con clave , nunca con contraseña. Cada destino tiene un alias corto
<u>SMB</u> (carpetas compartidas)	Ver los discos de otra máquina como unidades locales	Proyectos, almacenamiento y modelos del servidor; el Patio de Vecinos vive en una carpeta compartida del Mac. Entre el PC y el servidor hay además un enlace directo de 25 Gb/s
<u>WSL</u>	Un Linux dentro de Windows, con acceso a la <u>GPU</u> (Graphics Processing Unit, tarjeta gráfica)	Herramientas Linux sin salir del PC
Tailscale	Red privada cifrada entre tus dispositivos, estés donde estés	Acceso seguro desde fuera de la oficina sin abrir puertos en el router
Escritorio remoto	Ver y usar la pantalla de otra máquina	Para lo que necesita interfaz gráfica

UN DETALLE DE WINDOWS QUE CUESTA HORAS

Las unidades de red mapeadas por un usuario normal **no las ve** un proceso que se ejecuta con permisos de administrador, y viceversa. Si una herramienta «no encuentra» una unidad que tú ves, empieza por ahí.

CREDENCIALES: EL AGENTE NO LAS VE

Las contraseñas viven en un **gestor de contraseñas**. Cuando hay que teclear una en el momento, se abre un **diálogo nativo del sistema operativo** donde la escribe la persona: no pasa por el chat, ni por el historial, ni por los registros, y queda cifrada.

Rosa: darle manos al asistente, con cuidado

Rosa es un conjunto de cuatro conectores que permiten a Claude actuar sobre las máquinas. Lo importante no es que pueda hacerlo, sino **cómo se protege** al hacerlo.

Conector	Qué hace	Protecciones
Rosa <u>SSH</u>	Ejecuta órdenes en servidores Linux remotos, transfiere ficheros, abre túneles, gestiona tareas largas en segundo plano y varias máquinas a la vez	Analiza el riesgo de cada orden antes de ejecutarla; copias y vuelta atrás; credenciales por diálogo nativo
PowerShell Executor	Opera sobre el Windows local: órdenes, ficheros, tareas en segundo plano, consolas interactivas	Validación de órdenes, puntos de restauración, elevación de permisos explícita y registro de auditoría
Rosita <u>WSL</u>	Ejecuta en el Linux local de Windows, sin <u>SSH</u>	La contraseña de administrador se inyecta de forma segura, nunca escrita en la orden
Context Keeper	Audita y ordena la configuración de los propios asistentes: habilidades, memorias, perfiles	Copia integral antes de tocar; nada destructivo sin ensayo en seco; cada cambio se decide de forma supervisada

SOFTWARE LIBRE CONTROLABLE POR SSH

La combinación que más rinde: **programas libres en un servidor Linux** (modelos, cálculo, bases de datos, SIG (Sistema de Información Geográfica)) **manejados por SSH** desde el asistente. Todo se puede ejecutar sin interfaz, dejar trazas y repetir.

Conectores MCP: el asistente con acceso a tus sistemas

MCP (*Model Context Protocol*) es el estándar para darle a un asistente herramientas concretas. Cada conector es un pequeño programa que expone unas pocas funciones con

permisos acotados.

Conector	Qué le permite al asistente
RAG normativo y servicios de IA	Consultar la normativa con vigencia antes de afirmar una cifra; arrancar, parar y recomendar servicios de IA del servidor
Grafo de conocimiento del equipo	Registrar y consultar hechos estructurados: qué máquina hace qué, qué decisión depende de qué
Selector de herramientas	Elegir qué librería o programa usar antes de escribir código (cientos de entradas catalogadas)
Búsqueda web propia	Buscar e investigar en internet leyendo las páginas con un modelo local (Guía 3)
Memoria de sesión	Indexar conversaciones y recuperar lo decidido en sesiones anteriores
Protocolo de sesión	Leer el Patio, publicar en él y generar el texto de arranque
PDF	Leer, renderizar, rellenar formularios, firmar, unir y extraer tablas
QGIS y CAD	Manejar un proyecto de QGIS en vivo; leer y generar dibujos <u>DXF</u> (formatos de dibujo técnico CAD)
Auditoría de equipos	Salud de discos, procesos sospechosos, integridad de ficheros

UN CONECTOR TAMBIÉN PUEDE FALLAR

Si un conector no conecta, el asistente debe decirlo y no suponer que la herramienta no existe ni inventarse el resultado. Pide que te avise siempre de qué conectores tiene disponibles.

Skills, agentes y hooks: organizar el flujo de trabajo

Skills (habilidades)

Una **skill** es un procedimiento escrito que el asistente carga cuando la tarea lo requiere: cómo hacer un informe de jornada, cómo consultar la normativa, cómo manejar un instrumento. Tenemos cerca de un centenar, agrupadas así:

Grupo	Ejemplos
Infraestructura	<u>Docker</u> (empaqueta programas para que funcionen igual en cualquier máquina) y modelos locales, entornos de Python, conectores Rosa, credenciales
Coordinación	Patio de Vecinos, protocolo de sesión, informe de jornada, memoria
Acústica y normativa	Normativa acústica, <u>RAG</u> (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) normativo, medida in situ, sistema de calidad
Telecomunicación	Modo <u>ICT</u> (Infraestructura Común de Telecomunicaciones (RD 346/2011)), normativa <u>ICT</u> , herramientas <u>ICT</u>
Programación y datos	Selección de herramientas, Python y <u>VBA</u> (Visual Basic for Applications, macros de Office), calidad de código, análisis de datos
MATLAB y Simulink	Una veintena del propio fabricante
Documentos, multimedia, <u>SIG</u> y 3D	Informes, audio y vídeo, mapas, escaneo 3D
Recuperación forense	Diagnóstico, clonado y recuperación de ficheros

Agentes especializados

Subagentes que el asistente principal lanza para una tarea concreta: **adversarios** que intentan refutar, **auditores** que dan el sí o el no final, redactores y verificadores del RAG, constructores de módulos de cálculo o de interfaces, investigadores.

Hooks: lo que ocurre sin que nadie se acuerde

Momento	Qué hace
Al empezar la sesión	Recupera el contexto y carga las normas del Patio
En cada mensaje	Recupera recuerdos relevantes
Antes de compactar	Indexa la conversación
Antes de ejecutar Python	«El Calculista» avisa si se va a usar el entorno equivocado para una librería. Solo avisa, nunca bloquea

Software de ingeniería controlado por agentes

Software	Para qué	Cómo lo controla el agente
Excel, Word y VBA	Plantillas de medida, informes, macros	Automatización de Office (COM) desde Python. Ojo: que una macro se ejecute no garantiza que el proyecto <u>VBA</u> entero compile
Librerías Python	Cálculo de aislamiento, acústica de salas, señal, datos	Scripts en el servidor, lanzados por <u>SSH</u> . Regla: no predecir sin calcular
NoiseModelling + PostGIS	Mapas de ruido exterior con el método europeo <u>CNOSSOS-EU</u> (método europeo de cálculo de ruido ambiental): del catastro a los niveles día, tarde, noche	Una cadena de procesos en el servidor, por <u>SSH</u> , con base de datos geográfica en contenedor
QGIS	Digitalizar, visualizar y procesar datos geográficos	Conector <u>MCP</u> (Model Context Protocol: conecta el asistente con herramientas) en vivo, Python de QGIS y procesos sin interfaz
MATLAB / Octave	Validación cruzada, simulación, código heredado	MATLAB en Windows; Octave en el servidor para ejecuciones por lotes sin interfaz
INSUL, SonArchitect y CATT-Acoustic	Predicción de aislamiento de particiones, modelo de edificio según ISO 12354, acústica de salas	El agente genera los ficheros de entrada por script a partir del catálogo de materiales, y el ingeniero valida en la interfaz del programa antes de entregar. Nunca se inventan parámetros de materiales
<u>Docker</u>	Servicios empaquetados: interfaz de chat, buscador, avisos, gestión documental...	Por <u>SSH</u> y conector; un contenedor cada vez, verificando que arranca
<u>Ollama</u> y ComfyUI	Modelos de lenguaje locales; generación de imagen	Por <u>SSH</u> y <u>API</u> (Application Programming Interface: puerta para que un programa hable con otro), comprobando antes la memoria de vídeo libre

EL REPARTO QUE FUNCIONA

El agente **prepara** (genera ficheros, lanza cálculos, compara resultados); el programa de ingeniería **calcula**; el ingeniero **valida y firma**. Ninguno de los tres se salta.

Pautas de seguridad

1. **Contraseñas fuera del chat:** gestor de contraseñas y diálogo nativo.
2. **SSH con clave**, no con contraseña; y servicios accesibles solo desde tu red local o tu red privada, sin puertos abiertos a internet.
3. **Copia antes de tocar** y vuelta atrás definida.
4. **Ensayo en seco** de todo lo que borra o modifica.
5. **Foto del estado antes y después** de instalar: una instalación puede cambiar en silencio la versión de otra librería.
6. **Permisos mínimos:** cada conector, solo lo que necesita. Configura qué puede ejecutar el asistente sin preguntar y qué no.
7. **Nada corriendo con la sesión cerrada** si no es imprescindible.
8. **Registro de lo que se hizo**, con fecha y hora reales.
9. **Datos de cliente, en local.** Y cuidado con lo que se proyecta o se comparte: una plantilla real puede llevar datos personales.

PARA EMPEZAR ESTA SEMANA

Ordena tus carpetas, escribe el fichero de instrucciones, configura **SSH** con clave hacia tu máquina con **GPU** y conecta un solo conector. Con eso tu asistente ya puede trabajar sobre tus sistemas de verdad —y tú sigues decidiendo.

Catálogo de marcos

Los 47 marcos van aquí enunciados. La ficha completa de cada uno —origen, anatomía, taller y evidencia— está en el PDF «Catálogo de marcos», que acompaña a este, y en la versión HTML de las guías.

47 formas reconocidas de pensar, decidir, organizar el trabajo y sacarle más partido a una IA, ordenadas en 7 categorías. Cada marco con su historia, cómo funciona por dentro, un ejemplo real, el despiece de sus piezas, un taller listo para usar y la evidencia que lo respalda o lo desmiente.

Nivel: consulta

47 marcos · 7 categorías

Con fuentes enlazadas

Cómo usar este catálogo

Catálogo de procedimientos y algoritmos reconocidos para creatividad, diagnóstico, priorización, estrategia, productividad, decisión en grupo y dirección del razonamiento en modelos de lenguaje. Cada marco trae su historia investigada, cómo funciona por dentro, un ejemplo real trabajado, el despiece de cada pieza con su diagrama, un taller listo para usar y la evidencia que lo respalda o lo desmiente.

EN CIFRAS

47 marcos en **7 categorías**. Una pestaña por categoría; dentro, cada marco es un bloque desplegable: pulsa su título para abrirlo.

Cómo elegir un marco

Empieza por la pregunta que tienes delante y ve a la categoría que la responde:

- Generar y explorar ideas de forma deliberada, no al azar. → pestaña **1 · Creatividad**
- Encontrar el porqué real de un problema antes de arreglarlo. → pestaña **2 · Diagnóstico**
- Decidir qué hacer primero cuando todo compite por el mismo tiempo. → pestaña **3 · Priorización**
- Situar una decisión dentro del mapa completo del negocio o proyecto. → pestaña **4 · Estrategia**

- Organizar el trabajo propio y de un equipo en el tiempo. → pestaña **5 · Productividad**
- Repartir quién decide, quién opina y cómo se llega a consenso. → pestaña **6 · Decisión en grupo**
- Patrones para que un LLM piense y actúe de forma más fiable. → pestaña **7 · Razonamiento de una IA**

Dentro de la categoría, lee el resumen de cada marco. Si dudas entre dos, abre cualquiera de ellos y busca el apartado «**Marcos hermanos · cuándo elegir cuál**», al final de su bloque de evidencia: compara el marco con sus alternativas más cercanas. Y antes de aplicarlo, lee siempre «**Cuándo NO usarlo**».

Qué contiene cada ficha

Bloque	Qué encontrarás
Ficha	Origen ampliado, qué es, para qué sirve, cómo se hace, qué consigue y cuándo no usarlo
Ejemplos y profundidad	Cómo funciona por dentro, un ejemplo real, variantes, fortalezas, limitaciones y fuentes
Anatomía	Diagrama y despiece de cada componente: qué hace, qué sí, qué no y el error típico
Taller	Formato, duración, participantes, materiales, guion paso a paso, plantilla, preguntas, errores frecuentes y lista de comprobación
Evidencia	Qué dice la investigación, estudios, críticas, marcos hermanos y lecturas recomendadas

Índice de categorías

Categoría	Marcos	Contenido
1 • Pensamiento estructurado y creatividad Generar y explorar ideas de forma deliberada, no al azar.	9	01 Seis Sombreros para Pensar · 02 SCAMPER · 03 Pensamiento Lateral · 04 TRIZ · 05 Design Thinking · 06 Design Sprint · 07 Método Disney · 08 Modelos mentales (Munger) · 09 Pre-mortem
2 • Diagnóstico y causa raíz Encontrar el porqué real de un problema antes de arreglarlo.	7	10 5 Porqués · 11 Diagrama de Ishikawa · 12 8D · 13 A3 · 14 DMAIC · 15 PDCA (Ciclo de Deming) · 16 Kepner-Tregoe
3 • Priorización Decidir qué hacer primero cuando todo compite por el mismo tiempo.	6	17 Matriz de Eisenhower · 18 MoSCoW · 19 RICE · 20 ICE · 21 Modelo Kano · 22 Principio de Pareto (80/20)
4 • Estrategia Situar una decisión dentro del mapa completo del negocio o proyecto.	7	23 SWOT / DAFO · 24 PESTEL · 25 5 Fuerzas de Porter · 26 Business Model Canvas · 27 Blue Ocean Strategy · 28 OKR · 29 Balanced Scorecard
5 • Productividad y gestión de proyecto Organizar el trabajo propio y de un equipo en el tiempo.	6	30 GTD (Getting Things Done) · 31 Kanban · 32 Scrum · 33 Técnica Pomodoro · 34 Teoría de las Restricciones (TOC) · 35 PERT / CPM (Ruta Crítica)
6 • Decisión en grupo Repartir quién decide, quién opina y cómo se llega a consenso.	5	36 RACI (y variantes DACI / RAPID) · 37 Método Delphi · 38 Nominal Group Technique · 39 Fist of Five · 40 Liberating Structures
7 • Dirigir el razonamiento de una IA Patrones para que un LLM	7	41 Chain-of-Thought (CoT) · 42 Tree-of-Thought (ToT) · 43 ReAct (Reason + Act) · 44 Self-Consistency · 45 Least-to-Most Prompting · 46 Reflexion · 47 Plan-and-Solve

piense y actúe de forma
más fiable.

PARA IMPRIMIR O GUARDAR EN PDF

Al imprimir se muestran todas las categorías con todos los marcos desplegados. Los números entre paréntesis que aparecen en el texto, como «Pre-mortem (09)», remiten al número del marco en este catálogo.

1 · Pensamiento estructurado y creatividad

Generar y explorar ideas de forma deliberada, no al azar. 9 marcos

- ▶ **01 Seis Sombreros para Pensar**
Edward de Bono · 1985
Separar los tipos de pensamiento en roles turnados para que no compitan entre sí en la misma frase.
- ▶ **02 SCAMPER**
Bob Eberle · 1971 (sobre ideas de Osborn)
Un checklist de siete verbos para forzar variantes de algo que ya existe.
- ▶ **03 Pensamiento Lateral**
Edward de Bono · 1967
Salir deliberadamente del patrón lógico obvio para llegar a una solución que la lógica vertical no encuentra.
- ▶ **04 TRIZ**
Genrich Altshuller · desde 1946 (URSS)
Un catálogo de patrones de solución extraídos de analizar miles de patentes, para resolver 'contradicciones técnicas'.
- ▶ **05 Design Thinking**
Stanford d.school / IDEO · popularizado 2000s
Un proceso de cinco fases centrado en entender a la persona antes de proponer una solución.



06

Design Sprint

Jake Knapp / Google Ventures · 2010

Comprimir el Design Thinking completo en cinco días con una regla del proceso.



07

Método Disney

Atribuido a Walt Disney, formalizado por Robert Dilts

Tres roles mentales secuenciales — soñador, realista, crítico — nunca mezclados en el mismo momento.



08

Modelos mentales (Munger)

Charlie Munger · popularizado desde los 90

Una caja de herramientas de atajos de razonamiento tomados de varias disciplinas, para no depender de un único marco.



09

Pre-mortem

Gary Klein · 2007

Imaginar que el proyecto ya ha fracasado estrepitosamente, antes de empezar, y trabajar hacia atrás para averiguar por qué.

2 · Diagnóstico y causa raíz

Encontrar el porqué real de un problema antes de arreglarlo.

7 marcos



10

5 Porqués

Sakichi Toyoda / Sistema Toyota

Preguntar 'por qué' cinco veces seguidas sobre el mismo síntoma para llegar de la causa aparente a la causa raíz.



11

Diagrama de Ishikawa

Kaoru Ishikawa · 1968 (espina de pescado)

Organizar visualmente las causas posibles de un problema en categorías, para no dejarse ninguna familia de causas sin mirar.



12

8D

Ford Motor Company · años 80

Un formato de ocho disciplinas para documentar la resolución completa de un problema, pensado para auditoría de calidad.



13

A3**Sistema de Producción Toyota**

Resumir todo un problema — desde el contexto hasta el plan de seguimiento — en una sola hoja de tamaño A3.



14

DMAIC**Six Sigma / Motorola-GE · años 80-90**

Cinco fases con métricas obligatorias en cada una, para mejorar un proceso ya existente de forma medible.



15

PDCA (Ciclo de Deming)**Walter Shewhart / W. Edwards Deming · años 50**

El ciclo más básico de mejora continua: Planificar, Hacer, Verificar, Actuar — y volver a empezar.



16

Kepner-Tregoe**Charles Kepner y Benjamin Tregoe · 1965**

Separar explícitamente el análisis de un problema del análisis de una decisión, tratándolos como procesos distintos con preguntas distintas.

3 · Priorización

Decidir qué hacer primero cuando todo compite por el mismo tiempo. 6 marcos



17

Matriz de Eisenhower**Atribuida a Dwight D. Eisenhower**

Clasificar cualquier tarea en un cuadrante de urgente/no-urgente cruzado con importante/no-importante.



18

MoSCoW**Dai Clegg · años 90 (DSDM)**

Clasificar requisitos en cuatro cubos de obligatoriedad: Must, Should, Could, Won't.



19

RICE**Sean McBride / Intercom · 2016**

Puntuar cada iniciativa por Alcance, Impacto, Confianza y Esfuerzo, y dividir para obtener un número comparable.



20

ICE

Sean Ellis

Versión simplificada de RICE con solo tres factores: Impact, Confidence, Ease.



21

Modelo Kano

Noriaki Kano · 1984

Clasificar funcionalidades según cómo su presencia o ausencia afecta a la satisfacción del cliente, no de forma lineal.



22

Principio de Pareto (80/20)

Vilfredo Pareto · 1896 (economía), popularizado por Juran

La observación empírica de que, en muchos sistemas, aproximadamente el 20% de las causas explican el 80% de los efectos.

4 · Estrategia

Situar una decisión dentro del mapa completo del negocio o proyecto. 7 marcos



23

SWOT / DAFO

Atribuido a Albert Humphrey · Stanford, años 60-70

Cruzar factores internos (controlables) con externos (no controlables) en cuatro cuadrantes.



24

PESTEL

Extensión del PEST de Francis Aguilar · 1967

Escanear el entorno macro de un negocio por seis dimensiones: Político, Económico, Social, Tecnológico, Ecológico, Legal.



25

5 Fuerzas de Porter

Michael Porter · 1979

Analizar la rentabilidad estructural de una industria completa mirando cinco presiones competitivas.



26

Business Model Canvas

Alexander Osterwalder · 2008

Un lienzo de nueve bloques que resume un modelo de negocio entero en una sola página visual.



27

Blue Ocean Strategy

W. Chan Kim y Renée Mauborgne · 2005

Buscar deliberadamente un espacio de mercado sin competencia directa, en vez de competir en uno ya saturado.



28

OKR

Andy Grove / Intel, popularizado por John Doerr en Google

Fijar un Objetivo cualitativo e inspirador y 3-5 Resultados Clave cuantificables que prueben si se alcanzó.



29

Balanced Scorecard

Robert Kaplan y David Norton · 1992

Medir el desempeño de una organización desde cuatro perspectivas, no solo la financiera.

5 · Productividad y gestión de proyecto

Organizar el trabajo propio y de un equipo en el tiempo. 6 marcos



30

GTD (Getting Things Done)

David Allen · 2001

Sacar todos los compromisos de la cabeza y meterlos en un sistema externo de confianza, organizado por contexto y siguiente acción física.



31

Kanban

Sistema de Producción Toyota · Taiichi Ohno, años 40-50

Visualizar el flujo de trabajo en columnas y limitar cuánto trabajo puede estar 'en curso' a la vez.



32

Scrum

Jeff Sutherland y Ken Schwaber · 1995

Trabajo en lotes de tiempo fijo (sprints) con roles definidos y ceremonias regulares de planificación, seguimiento y revisión.



33

Técnica Pomodoro

Francesco Cirillo · finales de los 80

Trabajar en bloques cortos de tiempo fijo con descansos obligatorios entre medias.

▶ **34 Teoría de las Restricciones (TOC)**

Eliyahu Goldratt · 1984 (libro 'La Meta')

En cualquier sistema, la producción total está limitada por un único cuello de botella — optimizar cualquier otra parte no mejora el resultado global.

▶ **35 PERT / CPM (Ruta Crítica)**

US Navy (PERT) / DuPont (CPM) · años 50

Modelar un proyecto como una red de tareas con dependencias, para identificar qué secuencia de tareas determina la duración mínima total.

6 · Decisión en grupo

Repartir quién decide, quién opina y cómo se llega a consenso. 5 marcos

▶ **36 RACI (y variantes DACI / RAPID)**

Consultoría de gestión, popularizado desde los 80

Asignar explícitamente, para cada decisión o tarea, quién es Responsable, quién rinde cuentas (Accountable), a quién se Consulta y a quién se Informa.

▶ **37 Método Delphi**

RAND Corporation · años 50

Construir consenso entre expertos mediante rondas anónimas de opinión, evitando la influencia directa de jerarquía o personalidad.

▶ **38 Nominal Group Technique**

Delbecq y Van de Ven · 1971

Generar ideas individualmente en silencio antes de compartirlas en grupo, para neutralizar el efecto de quien habla primero o más alto.

▶ **39 Fist of Five**

Extendido desde Agile/facilitación de grupos

Votar el nivel de acuerdo con los dedos de una mano, de 1 (en desacuerdo total) a 5 (acuerdo total), para medir consenso al instante.

▶ **40 Liberating Structures**

Henri Lipmanowicz y Keith McCandless · 2000s

Un repertorio de más de 30 microestructuras de facilitación diseñadas para repartir la participación de forma más pareja que una reunión tradicional.

7 · Dirigir el razonamiento de una IA

Patrones para que un LLM piense y actúe de forma más fiable. 7 marcos

▶ **41 Chain-of-Thought (CoT)**

Wei et al. / Google · 2022

Pedir al modelo que escriba el razonamiento paso a paso antes de dar la respuesta final, en vez de saltar directo a la conclusión.

▶ **42 Tree-of-Thought (ToT)**

Yao et al. · 2023

En vez de una única cadena de razonamiento lineal, explorar varias ramas de pensamiento en paralelo y podar las que no van bien.

▶ **43 ReAct (Reason + Act)**

Yao et al. · 2022

Alternar explícitamente entre 'pensar' y 'actuar con una herramienta', usando el resultado de cada acción para decidir el siguiente pensamiento.

▶ **44 Self-Consistency**

Wang et al. / Google · 2022

Generar varias respuestas independientes a la misma pregunta y quedarse con la que aparece con más frecuencia, en vez de confiar en una sola.

▶ **45 Least-to-Most Prompting**

Zhou et al. / Google · 2022

Descomponer un problema complejo en subproblemas más simples, ordenados de menor a mayor dificultad, y resolverlos en secuencia usando la solución de cada uno para el siguiente.

▶ **46 Reflexion**

Shinn et al. · 2023

Un bucle donde el modelo genera una respuesta, se autoevalúa críticamente, y usa esa autocrítica para generar una versión corregida.



47

Plan-and-Solve

Wang et al. · 2023

Separar explícitamente la fase de hacer un plan completo de la fase de ejecutarlo, en vez de mezclar planificación y ejecución en el mismo paso.

Guía 11 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Las fuentes de cada marco están enlazadas en sus apartados «Fuentes» y «Para leer más».

[¿Una sigla que no conoces? Consulta el glosario](#)

Formatos y representaciones

La misma información, contada en el formato adecuado, se entiende, se versiona y se automatiza sola. En el formato equivocado se pierde: se convierte en una captura de pantalla que nadie puede recalcular. Esta guía recorre los formatos que usamos a diario —texto, datos, diagramas, grafos y entregables— siempre en las dos direcciones: **cómo se los das a una IA y cómo se los pides.**

Nivel: desde cero

Lectura: ~30 min

Con ejemplos copiables

Por qué el formato importa más de lo que parece

Imagina el mismo dato —el nivel medido en diez puntos de una fachada— en cuatro envases distintos: una foto del cuaderno de campo, una tabla pegada en un correo, una hoja de cálculo y un fichero de datos con nombres de campo. El contenido es idéntico. Lo que cambia es **todo lo demás**: si se puede recalcular, si se puede comparar con la campaña del año pasado, si un compañero ve quién cambió qué, y si una IA (Inteligencia Artificial) puede trabajar con ello sin equivocarse.

El mismo dato en...	¿Se recalcula?	¿Se versiona?	¿Lo entiende bien una IA?
Foto del cuaderno	No	No	Regular: hay que reconocerlo, y a veces se equivoca
Tabla pegada en un correo	Manualmente	No	Sí, si es pequeña y limpia
Hoja de cálculo	Sí	Mal (el fichero es opaco)	Sí, si la exportas a texto
Fichero de datos con campos	Sí	Sí, línea a línea	Muy bien: sabe qué es cada número

Las tres preguntas antes de elegir formato

¿Quién lo va a leer?

Una persona, un programa, una IA, o la Administración. Rara vez los cuatro con el mismo fichero.

¿Va a cambiar?

Si va a tener versiones y revisiones, tiene que ser texto. Si es la foto final de algo, puede ser un documento cerrado.

¿Contiene números que se usan?

Si alguien va a sumarlos, compararlos o comprobarlos, no pueden vivir dentro de un párrafo.

LA REGLA DE FONDO

El texto plano es el formato que no se muere. Un fichero de texto se abre en cualquier sitio, se compara línea a línea, se busca, se versiona y lo entiende cualquier IA sin conversiones. Todo lo demás —el documento con membrete, el **P.D.F.** (Portable Document Format: documento de presentación fija) sellado, la hoja de cálculo— es una *presentación* de ese texto. Trabaja en texto y genera la presentación al final, nunca al revés.

Entrada y salida: la idea que organiza toda la guía

Cada formato se mira siempre desde dos lados, y son dos problemas distintos:

- **Entrada** — qué le das tú a la IA. Aquí importa que el contenido llegue *completo y sin ruido*: sin cabeceras repetidas, sin tablas partidas, sin capturas.
- **Salida** — qué le pides que te devuelva. Aquí importa que el resultado sea *utilizable sin retoques*: que se pueda pegar en un programa, abrir en un navegador o volcar a una hoja de cálculo.

Un error muy común es pedir la salida en el formato en que quieres *verla* (un texto bonito) cuando lo que necesitas es el formato en que vas a *usarla* (una tabla de datos). Lo bonito se genera después, y se genera solo.

EL COSTE DE EQUIVOCARSE

Pedir «hazme un resumen de estas mediciones» devuelve un párrafo agradable del que no se puede sacar ni una cifra sin releerlo. Pedir «devuélveme una tabla con punto, fecha, nivel en decibelios y observación» devuelve algo que se pega en la hoja de cálculo, se ordena, se filtra y se compara. Mismo trabajo para la IA; media hora de diferencia para ti.

Texto estructurado: Markdown

Markdown (extensión `.md`) es texto normal con cuatro marcas de puntuación que significan estructura. Nació para escribir en internet y se ha convertido, sin que nadie lo decidiera, en **la lengua materna de las IA**: cuando una IA te responde con títulos, listas y negritas, está escribiendo Markdown.

Todo el formato que necesitas cabe aquí

```
# Memoria de aislamiento acústico

## 1. Objeto
Se evalúa el aislamiento a ruido aéreo entre dos recintos.

## 2. Datos de partida
- Superficie del elemento: 12,4 m²
- Volumen del recinto receptor: 48 m³
- **Exigencia aplicable:** 50 dBA

### 2.1 Equipos utilizados
| Equipo          | Referencia | Última calibración |
|-----|-----|-----|
| Sonómetro      | SON-02    | 2026-03-11         |
| Calibrador     | CAL-01    | 2026-03-11         |

> Nota: los valores se redondean según el método aplicable.
```

Esto es un fichero de texto. Se abre en el bloc de notas, se busca con `Ctrl+F`, se compara con la versión de ayer línea a línea y pesa dos kilobytes.

Marca	Qué significa	Para qué la usas de verdad
#, ##, ###	Título de nivel 1, 2, 3	El esqueleto del documento. La IA lo usa para saber qué apartado estás tocando
- o 1.	Lista sin orden / numerada	Condiciones, comprobaciones, pasos de un procedimiento
texto	Negrita	El valor que no puede pasarse por alto
	Tabla	Datos pequeños que van dentro del texto
>	Cita o nota	Texto literal de una norma, advertencias
Tres acentos graves	Bloque de código	Un comando, un cálculo, un fragmento de datos

Entrada: por qué Markdown le sienta tan bien a una IA

- **La estructura es visible en el propio texto.** No hay que interpretar tamaños de letra ni sangrías: un ## es un apartado, sin ambigüedad.
- **No hay ruido.** Un documento de tratamiento de textos lleva dentro cientos de instrucciones de estilo que no aportan nada al contenido y consumen espacio de lectura.
- **Puedes señalar un punto exacto.** «En el apartado 2.1, cambia la fecha de calibración» es una instrucción que se ejecuta sin error.

Truco de entrada: si tienes un documento largo en Word, **pégalo como texto** y pon tú mismo los ## en los apartados. Tardas dos minutos y la calidad de las respuestas sube de forma muy visible, porque la IA deja de adivinar dónde empieza cada cosa.

Salida: cómo pedirla

FÓRMULA QUE FUNCIONA

«Devuélvemelo en **Markdown**, con un título de nivel 2 por apartado, las magnitudes en negrita y una tabla para los equipos. Sin texto introductorio antes ni comentarios después: solo el documento.»

Ese «sin texto introductorio» importa más de lo que parece: convierte la respuesta en un fichero que puedes guardar tal cual, en vez de en una conversación de la que hay que recortar.

Cuándo pasa a Word o a PDF, y qué se pierde por el camino

El Markdown es el *original*; el documento con membrete es una *copia impresa*. La conversión es automática y de una sola dirección útil:

.md → Word

Para que el cliente comente sobre el texto. Se conservan títulos, listas y tablas; se pierden los bloques de código y se gana un montón de formato que ya no controlas.

.md → PDF

Para entregar y archivar. Presentación fija, paginación, firma. A cambio deja de ser editable y deja de compararse bien.

Word → .md

Posible, pero sucio: los estilos hechos «a mano» (negrita en vez de título) no se convierten en estructura. Lo que nunca fue estructura, no aparece.

DOS TRAMPAS DEL MARKDOWN

La tabla larga. En cuanto una tabla pasa de unas quince filas o de seis columnas, el Markdown deja de ser cómodo y los datos deberían vivir aparte, como datos (pestaña siguiente). **La estructura falsa.** Poner negrita donde debería haber un título engaña igual a la IA que a un lector: parece organizado y no lo está.

Datos: JSON, YAML y CSV

Cuando lo importante son los **valores** y no la prosa, el envase cambia. Estos tres formatos son texto plano —se abren en el bloc de notas— pero cada uno está pensado para una forma distinta de información.

Formato	Forma que representa	Cuándo es el adecuado
<u>JSON</u> (JavaScript Object Notation: datos con campos y valores)	Objetos anidados: un dato dentro de otro	Un resultado con varios niveles: ensayo → puntos → bandas de frecuencia. Es el formato que hablan los programas entre sí
<u>YAML</u> (formato de configuración legible, basado en sangrado)	Lo mismo que JSON, pero escrito para leerlo	Configuración que edita una persona: parámetros de un cálculo, plantillas, criterios
<u>CSV</u> (valores separados por comas: una tabla en texto plano)	Una tabla plana: filas y columnas, sin niveles	Un listado de mediciones, un inventario, cualquier cosa que acabe en una hoja de cálculo

Un listado de mediciones, en JSON

```
{
  "ensayo": "E-2026-012",
  "fecha": "2026-04-18",
  "magnitud": "nivel_sonoro_continuo_equivalente",
  "unidad": "dBA",
  "incertidumbre_expandida": 1.5,
  "puntos": [
    { "id": "P1", "altura_m": 1.5, "valor": 62.4, "valido": true },
    { "id": "P2", "altura_m": 1.5, "valor": 64.1, "valido": true },
    { "id": "P3", "altura_m": 1.5, "valor": 71.8, "valido": false,
      "observacion": "paso de vehiculo pesado" }
  ]
}
```

Fíjate en tres cosas: la unidad está declarada una sola vez y no se repite en cada número; los valores son números de verdad (`62.4` , sin comillas); y la validez es un sí/no explícito, no una nota de texto que haya que interpretar.

Qué es un esquema, en una frase

Un **esquema** es el contrato: la lista de campos que tiene que traer el fichero, de qué tipo es cada uno y cuáles son obligatorios. Es exactamente lo mismo que la cabecera de un formato de registro de un sistema de calidad: si falta la columna «última calibración», el registro no vale. Con un esquema puedes **comprobar automáticamente** que la IA te ha devuelto todo lo que pediste, en vez de revisarlo a ojo.

LA PETICIÓN QUE CAMBIA EL RESULTADO

No pidas «un resumen de estas mediciones». Pide: «*Devuélveme **solo** un JSON con esta forma exacta: un objeto con los campos `ensayo`, `fecha`, `unidad` y `puntos`; cada punto con `id`, `valor` (número, no texto) y `valido` (sí/no). Si un dato no está en la fuente, pon `null`; no lo inventes ni lo estimes.*»

Ese nivel de detalle hace tres cosas a la vez: fija qué tiene que salir, obliga a declarar los huecos en vez de rellenarlos con suposiciones, y convierte la respuesta en algo que un programa puede leer sin que tú intervengas.

La trampa de los números en texto

Es el error más frecuente y el más silencioso. Un número entre comillas, `"62,4"`, ya no es un número: es una cadena de caracteres. Consecuencias reales:

- **No se puede sumar ni promediar** sin convertirlo antes.
- **Se ordena como texto**: «100» va antes que «62», igual que «casa» va antes que «silla».
- **La coma decimal española rompe la conversión** en casi cualquier programa, que espera punto. En datos se escribe `62.4`; la coma se pone al *presentar*.
- **Las unidades pegadas al valor** (`"62,4 dBA"`) son el caso peor: parecen más informativas y son inutilizables. La unidad va en su propio campo.

El mismo listado, en CSV

```
id;altura_m;valor_dBA;valido;observacion
P1;1.5;62.4;si;
P2;1.5;64.1;si;
P3;1.5;71.8;no;paso de vehiculo pesado
```

Separador punto y coma porque en configuración regional española la coma se usa como decimal. Una sola línea de cabecera, sin filas de título encima, sin celdas combinadas y sin totales al final: eso es un CSV sano.

Y en YAML, cuando lo que se configura son criterios

```
proyecto: Edificio de oficinas
criterios:
  fachada:
    exigencia_dBA: 30
    margen_incertidumbre: 1.5
```

```
entre_recintos:
  exigencia_dBA: 50
  margen_incertidumbre: 1.5
salida:
  formato: html
  incluir_graficas: true
```

YAML se lee sin esfuerzo y se edita sin miedo a las llaves y las comas: por eso los parámetros de un cálculo o las reglas de una plantilla suelen vivir aquí y no en JSON. Su punto débil es el sangrado: un espacio de más cambia el significado.

ENTRADA: EL FORMATO QUE PEOR SIENTA

Pegar una hoja de cálculo con celdas combinadas, subtotales intercalados y cabecera en la fila siete. La IA tiene que adivinar qué es cabecera, qué es dato y qué es total, y adivina bien casi siempre —el «casi» es el problema—. Exporta a CSV con una sola fila de cabecera antes de pegarlo, y el «casi» desaparece.

Diagramas escritos: Mermaid

Mermaid es un modo de **escribir** un diagrama en vez de dibujarlo. Describes los nodos y las flechas con texto, y un programa lo convierte en imagen. Suena a rodeo y es justo lo contrario: el diagrama pasa a ser texto, y por tanto hereda todas las ventajas del texto.

Se versiona

Ves qué flecha cambió entre la revisión 2 y la 3. Con una imagen solo ves «la imagen ha cambiado».

La IA lo escribe

Le describes el proceso en una frase y te devuelve el diagrama. Y lo corrige: «añade un paso de revisión antes de la entrega».

Pesa nada

Diez líneas de texto dentro del propio documento, no un fichero de imagen aparte que se pierde.

Se corrige sin herramienta

Cambiar una etiqueta es editar una palabra, no reabrir un programa de dibujo y recolocar cajas.

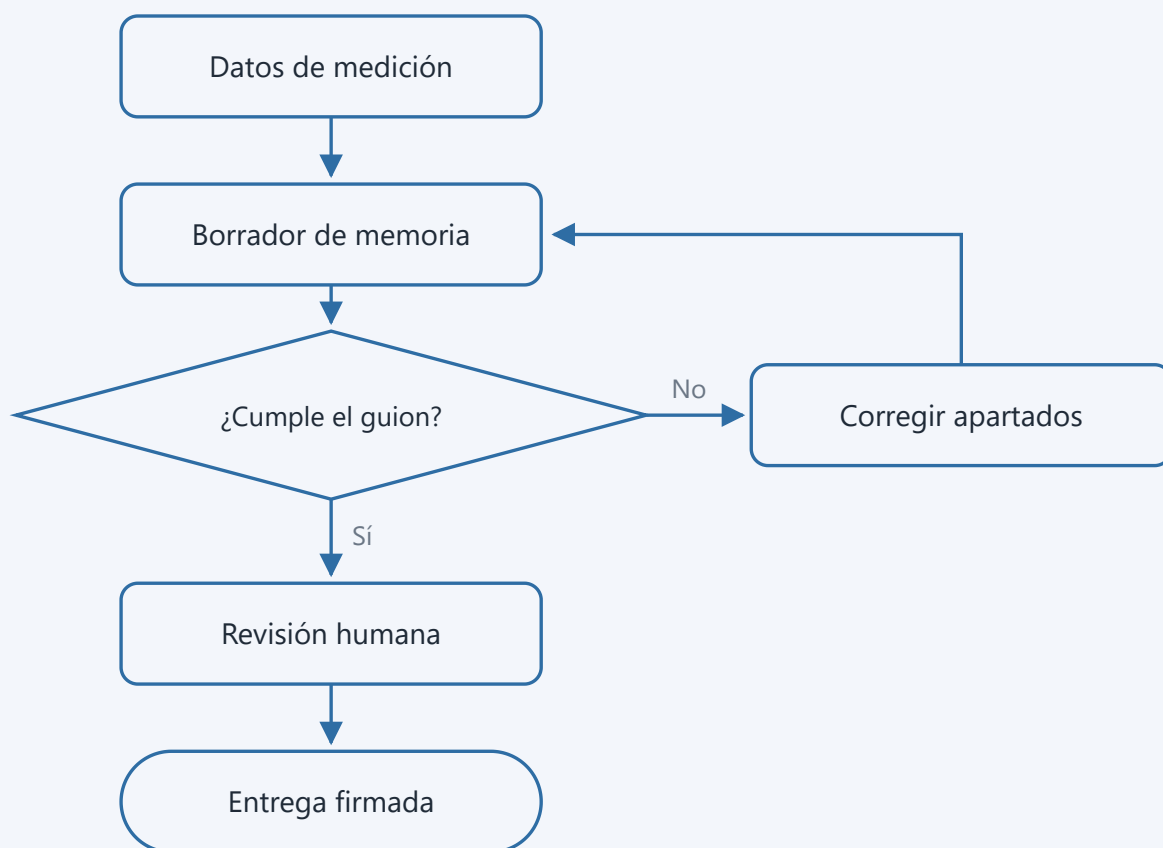
CÓMO SE VE EL RESULTADO

Esta guía funciona sin conexión a internet, así que **los bloques de código de abajo no se dibujan solos aquí**. Cada uno va acompañado de un dibujo hecho a mano que representa exactamente lo mismo. **El código es el bueno**: cópialo y pégalo en cualquier editor de Mermaid, en la documentación de un repositorio o en las herramientas de notas que ya lo admiten, y verás el diagrama renderizado.

1 · Diagrama de flujo: el ciclo de una memoria

flowchart TD

```
A[Datos de medicion] --> B[Borrador de memoria]
B --> C{Cumple el guion?}
C -- No --> D[Corregir apartados]
D --> B
C -- Si --> E[Revision humana]
E --> F([Entrega firmada])
```



Lo que dibuja el código de arriba. TD significa «de arriba abajo»; [texto] es una caja, {texto} un rombo de decisión y ([texto]) una caja redondeada de final.

2 · Diagrama de secuencia: quién le habla a quién y en qué orden

```
sequenceDiagram
```

```
    participant T as Técnico
```

```
    participant IA as Asistente
```

```
    participant N as Base normativa
```

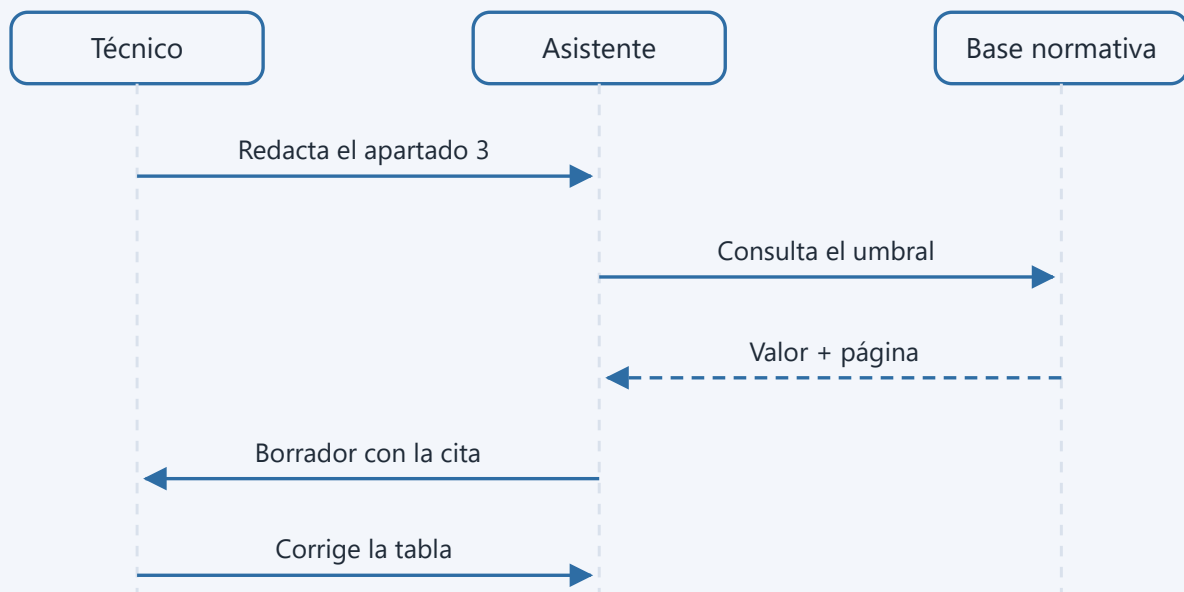
```
    T->>IA: Redacta el apartado 3
```

```
    IA->>N: Consulta el umbral aplicable
```

```
    N-->>IA: Valor + referencia de pagina
```

```
    IA->>T: Borrador con la cita incluida
```

```
    T->>IA: Corrige la tabla de equipos
```



La flecha de vuelta discontinua es una respuesta. Este diagrama sirve para documentar un flujo de trabajo con una IA, pero también una interfaz entre dos sistemas o el reparto de responsabilidades en un encargo.

3 · Gantt: la planificación de un encargo

```
gantt
```

```
    title Encargo tipo
```

```
    dateFormat YYYY-MM-DD
```

```
    axisFormat %d/%m
```

```
    section Campo
```

```
        Toma de datos      :a1, 2026-04-06, 5d
```

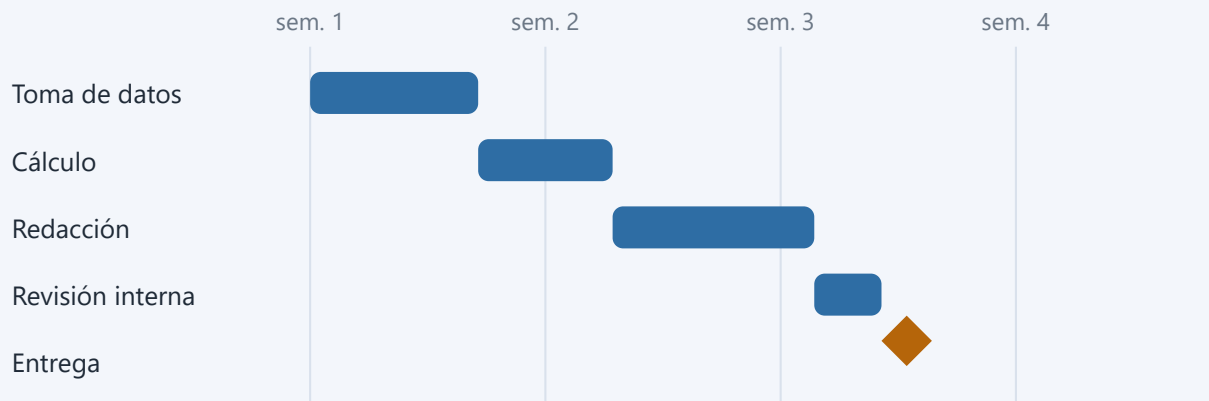
```
    section Gabinete
```

```
        Calculo           :a2, after a1, 4d
```

```
        Redaccion         :a3, after a2, 6d
```

```
    section Cierre
```

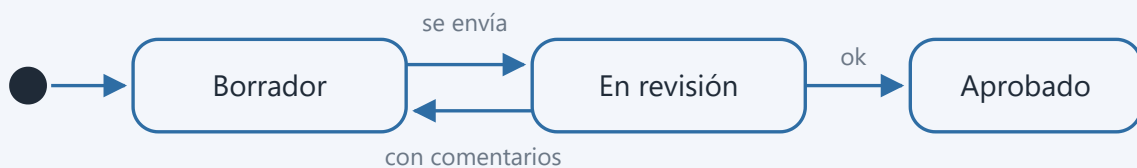
Revision interna :a4, after a3, 2d
Entrega :milestone, after a4, 0d



after a1 significa «cuando termine la tarea a1»: no escribes fechas, escribes dependencias.
Si la toma de datos se retrasa dos días, todo lo demás se recoloca solo.

4 · Estados: en qué situación está un documento

```
stateDiagram-v2
    [*] --> Borrador
    Borrador --> EnRevision : se envía
    EnRevision --> Borrador : con comentarios
    EnRevision --> Aprobado : sin objeciones
    Aprobado --> [*]
```



Muy útil para el control documental de un sistema de calidad: deja escrito, sin ambigüedad, qué transiciones existen y cuáles no.

SALIDA: CÓMO PEDIR UN DIAGRAMA

«Dibújame el proceso de tramitación que te he descrito **en código Mermaid**, `flowchart TD`. Un nodo por paso, rombos en las decisiones, etiquetas en las flechas. Devuélveme **solo el bloque de código**.» Y para corregir, en lenguaje normal: «añade una rama de subsanación desde la decisión, que vuelva al paso 2».

LÍMITES HONESTOS

Mermaid coloca los elementos automáticamente: no controlas dónde queda cada caja. Con más de unos veinte nodos el resultado se enreda y conviene partirlo en varios diagramas. Y si necesitas un plano a escala o una figura con geometría exacta, esto no es la herramienta: es un esquema de relaciones, no un dibujo técnico.

Diagramas dibujados: SVG e imágenes

Cuando el diagrama no sale de una estructura lógica sino de una **composición concreta** — este bloque aquí, esta cota allí, esta flecha con este ángulo—, se dibuja. Y ahí hay dos familias muy distintas.

Tipo	Qué es por dentro	Brilla en	Falla en
<u>SVG</u> (gráfico vectorial descrito en texto)	Texto: líneas, rectángulos y etiquetas con coordenadas	Esquemas, croquis, iconos, gráficas. Se amplía sin pixelarse, se busca por su texto y cambia de color con el tema claro/oscuro	Fotografías y cualquier imagen con matices continuos
Imagen de puntos (PNG, JPG)	Una rejilla de píxeles	Fotos de obra, capturas de pantalla, cualquier cosa real	Ampliar, editar una etiqueta, buscar dentro, adaptarse al tema

Los cinco dibujos de esta guía son SVG escritos a mano, y por eso **cambian de color solos** cuando el sistema está en modo oscuro: los colores no están escritos en el dibujo, sino tomados del tema de la página. Una imagen de puntos no puede hacer eso.

Un SVG por dentro: es legible

```
<svg viewBox="0 0 200 60">
  <rect x="10" y="10" width="120" height="40" rx="8"
```

```
fill="none" stroke="currentColor"/>
<text x="70" y="35" text-anchor="middle">Recinto emisor</text>
</svg>
```

Un rectángulo y una etiqueta. Como es texto, una IA puede escribirlo, corregirlo («sube la etiqueta cinco puntos») y tú puedes comprobar exactamente qué hay dentro. Y como es texto, cabe dentro del propio documento: no hay un fichero de imagen que se quede atrás al mover la carpeta.

Entrada: la imagen como pregunta

En sentido contrario, una imagen también es una forma legítima de *darle* información a una IA moderna: una foto del cuadro de mandos, una captura del mensaje de error, el plano escaneado. Funciona sorprendentemente bien para **describir y orientar**, y hay que desconfiar de ella para **extraer cifras**.

LA REGLA DE LA FOTO

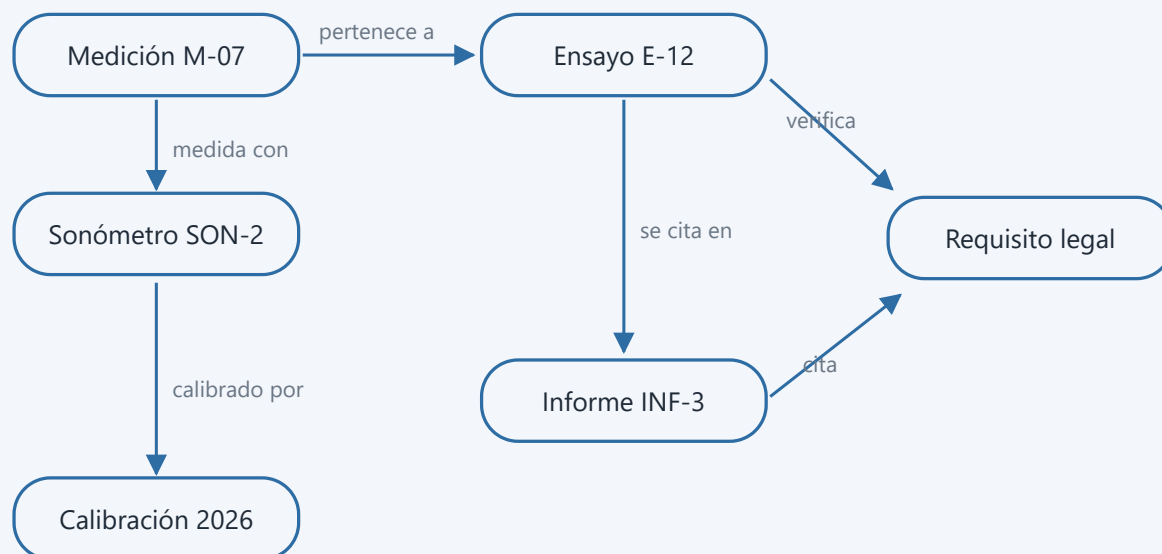
Una imagen sirve para *saber qué hay*, no para *copiar lo que pone*. De una tabla fotografiada se sacan valores que **parecen correctos y a veces no lo son**, y el error no avisa: no hay celda vacía ni signo raro que te alerte. Si de esa imagen van a salir números que acaban en un informe, hay que verificarlos uno a uno contra el original, o buscar el fichero de datos del que salió la tabla.

Cuándo compensa cada uno

- **Esquema conceptual** (proceso, arquitectura, relaciones) → Mermaid, que además lo escribe la IA.
- **Croquis con posiciones concretas** (situación de los puntos de medida en una planta, sección de un elemento por capas) → SVG.
- **Gráfica de datos** → generada a partir de los datos, en SVG. Nunca una captura de pantalla de la gráfica: si los datos cambian, la captura miente y nadie se entera.
- **Realidad** (obra, equipo, montaje) → foto, con su pie, su fecha y su referencia.

Grafos: cuando lo importante son las relaciones

Un **grafo** son dos cosas: *nodos* (las cosas) y *aristas* (las relaciones entre ellas). Nada más. Parece abstracto hasta que ves que la mitad de tu trabajo tiene esa forma y la estás metiendo a la fuerza en una tabla.



Trazabilidad de un ensayo. Ninguna tabla muestra esto de un vistazo, porque no es una lista: es una red.

Tabla o grafo: la diferencia práctica

Pregunta	Con una tabla	Con un grafo
¿Qué mediciones pertenecen al ensayo E-12?	Filtros una columna. Perfecto	También, sin ventaja
Si este sonómetro se cae de calibración, ¿qué informes quedan tocados?	Tres cruces manuales entre hojas, y esperar no olvidar ninguna	Es seguir las flechas hacia atrás: respuesta inmediata y completa
¿Qué apartados del informe dependen de un requisito que acaba de cambiar?	Depende de que alguien mantuviera una columna «referencia»	Está en la propia estructura
¿Quién revisa el trabajo de quién, y hay algún bucle?	Muy difícil de ver	Un bucle en un grafo se ve de un vistazo

Tres casos de despacho donde el grafo gana

Dependencias

Qué tarea bloquea a cuál. La cadena crítica de un encargo es un camino dentro del grafo.

Revisiones

Quién revisa a quién. Detecta al instante que alguien se está revisando a sí mismo por una cadena larga.

Trazabilidad normativa

Qué exigencia cubre qué apartado, con qué ensayo y qué equipo. Cuando cambia la norma, sabes exactamente qué se toca.

Entrada y salida de un grafo

Para dárselo a una IA, el grafo se escribe como una lista de relaciones, una por línea. Es el formato más digerible que existe, porque cada línea es una afirmación completa:

```
origen;relacion;destino
M-07;pertenece_a;E-12
M-07;medida_con;SON-2
SON-2;calibrado_por;CAL-2026
E-12;verifica;REQ-04
E-12;se_cita_en;INF-3
INF-3;cita;REQ-04
```

Para pedírselo, la fórmula es: «De este texto, extraéme las entidades y sus relaciones en una tabla de tres columnas —origen, relación, destino—, usando **solo** estos tipos de relación:

`pertenece_a`, `verifica`, `cita`, `medida_con`. Si una relación del texto no encaja en ninguno, no la fuerces: ponla aparte en una lista de dudas».

POR QUÉ ESE «NO LA FUERCES» ES LA PARTE IMPORTANTE

Un vocabulario cerrado de relaciones convierte el grafo en algo consultable; sin él acabas con quince maneras de decir lo mismo. Y la lista de dudas es oro: es exactamente el sitio donde aparecen los casos que tu clasificación no había previsto, en vez de quedar escondidos dentro de una etiqueta que no les corresponde.

Un grafo también es la forma natural de guardar el conocimiento estable de un despacho —quién hace qué, qué procedimiento depende de qué norma, qué equipo se usa en qué ensayo— y de consultarlo después con lenguaje normal.

El entregable: HTML, PDF, Word y Excel

Todo lo anterior es el taller. Esto es el escaparate: el fichero que sale por la puerta con tu nombre. Y aquí la pregunta no es «cuál es mejor», sino **quién lo recibe y qué va a hacer con él**.

HTML: una página que se basta a sí misma

HTML (el formato de las páginas web) suena a «sitio web», pero un fichero `.html` es simplemente un documento que se abre haciendo doble clic, en cualquier ordenador, móvil o tableta, sin instalar nada. Si se escribe **autocontenido** —con los estilos y los dibujos dentro del propio fichero, sin depender de internet—, es probablemente el mejor entregable que existe hoy para un informe técnico:

- **Un solo fichero** que se envía por correo y funciona igual dentro de diez años, con o sin conexión. Esta misma guía es un ejemplo: no descarga absolutamente nada.
- **Interactivo sin complicaciones**: pestañas, secciones plegables, una tabla que se ordena al pulsar en la cabecera. Un informe de cuarenta páginas se recorre en tres clics.
- **Se adapta** a la pantalla del que lo lee y al modo claro u oscuro.
- **Es texto**: se versiona, se compara y una IA lo genera y lo corrige entero.
- Y cuando hace falta papel, **se imprime a PDF** desde el propio navegador.

SUS DOS LÍMITES

No tiene paginación fija —no existe «la página 12»— y **no es un formato de registro firmado**: cualquiera puede editarlo. Para lo que hay que sellar, archivar o presentar, el entregable es el PDF.

PDF: la foto fija

El PDF congela la presentación: se ve igual en todas partes, tiene páginas numeradas y admite firma electrónica. Es lo que se entrega, lo que se archiva y lo que pide la Administración. Su precio: **es el peor formato de entrada que existe**, y sobre eso va la pestaña siguiente. Norma sana: el PDF se *genera* al final, y siempre se guarda también el original del que salió.

Word y Excel: cuando lo pide quien manda

Formato	Cuándo es la respuesta correcta	Qué hacer para no sufrir
.docx (Word)	El cliente va a comentar sobre el texto; hay una plantilla oficial obligatoria; el destinatario trabaja con control de cambios	Escribe en Markdown y conviértelo al final sobre la plantilla. Mantén el original: el .docx es la copia, no la fuente
.xlsx (Excel)	El receptor va a seguir calculando; es el formato de un anexo oficial; hay que entregar el listado de mediciones para que otro lo reutilice	Genera la hoja desde los datos (el CSV o el JSON), no a mano. Una fila de cabecera, sin celdas combinadas, sin colores que signifiquen algo
Formatos de programa (cálculo, dibujo, medida)	Cuando el destinatario usa el mismo programa y necesita seguir trabajando	Acompáñalo siempre de una exportación en texto o PDF: dentro de cinco años puede que ese programa ya no exista o no abra ese fichero

LA CADENA QUE USAMOS

Datos en **CSV o JSON** → texto en **Markdown** → diagramas en **Mermaid o SVG** → todo ello ensamblado en un **HTML** autocontenido → y de ahí, según quien lo reciba, un **PDF** para firmar o un **.docx** sobre plantilla. Cada paso es automático: cambias un dato en el origen y el entregable se regenera entero, sin que nadie tenga que acordarse de actualizar una tabla a mano.

Entrada: qué digiere bien una IA y qué no

Aquí está la mitad de la calidad de las respuestas que vas a obtener, y casi nadie la trabaja. La regla general: **cuanto más cerca esté de texto plano con estructura visible, mejor.**

Se lo das así...	Qué tal	Qué hacer
Texto plano, Markdown	✓ Excelente	Nada. Es el caso ideal
CSV o JSON limpios	✓ Excelente	Una fila de cabecera y unidades declaradas aparte
Tabla pequeña pegada en el mensaje	✓ Bien	Que las columnas se distingan; sin celdas combinadas
PDF (documento de presentación fija) con texto seleccionable	● Aceptable	Extraer el texto conservando el diseño de columnas antes de usarlo; comprobar que no se han mezclado
Documento de Word largo	● Aceptable	Pegar como texto y marcar los apartados con <code>##</code>
Hoja de cálculo compleja	● Regular	Exportar cada hoja a CSV por separado; quitar subtotales y filas de adorno
Tabla numérica densa dentro de un PDF	● Mal	Extraerla con una herramienta de tablas y verificar cifra a cifra . Nunca copiarla «leyéndola»
PDF escaneado o foto de un documento	● Mal	Reconocimiento de caracteres (OCR (reconocimiento óptico de caracteres)) primero, revisión después; asumir que habrá errores
Captura de pantalla de una tabla	● Mal	Buscar el fichero del que salió. Si no existe, teclear los datos y revisarlos

LA REGLA QUE MÁS DISGUSTOS EVITA

Nunca extraigas tablas numéricas densas de un PDF «leyéndolas». Una IA que transcribe una tabla de sesenta valores acierta la inmensa mayoría y se equivoca en algunos, y lo hace con total naturalidad: el resultado tiene el mismo aspecto correcto que si estuviera bien. Para tablas se usa una herramienta que extrae por posición, y se comprueba. Es el error que más caro sale, porque entra en un informe sin hacer ruido.

Tres gestos de entrada que valen por una hora de trabajo

- **Di qué es lo que pegas** antes de pegarlo: «esto es el listado de puntos de medida, la primera fila es cabecera, el separador es punto y coma, los valores están en decibelios ponderados A». Tres líneas que eliminan todas las suposiciones.

- **Trocea por apartados** en vez de pegar doscientas páginas: da mejor resultado trabajar sobre el capítulo que te interesa que sobre el documento entero.
- **Marca lo que es literal:** si un párrafo es texto de norma que no se puede reescribir, dilo explícitamente. Si no lo dices, se reformula con la mejor intención.

Tabla resumen

Formato	Para qué brilla	Entrada o salida	Trampa habitual
Markdown (.md)	Cualquier texto con estructura: memorias, procedimientos, notas	Las dos. Es el formato por defecto para hablar con una IA	Usar negrita donde tocaba un título: parece estructura y no lo es
JSON	Datos con niveles; que un programa lea el resultado sin intervención humana	Salida sobre todo; entrada excelente	Números entre comillas, y unidades pegadas al valor
YAML	Parámetros y criterios que edita una persona	Entrada (configuración)	El sangrado: un espacio de más cambia el significado
CSV	Listados y tablas planas que acabarán en una hoja de cálculo	Las dos	Separador y decimal (coma frente a punto y coma); cabeceras dobles
Mermaid	Flujos, secuencias, estados, planificación	Salida: la IA lo escribe y lo corrige	Pasarse de nodos: por encima de veinte, se enreda
SVG	Croquis, esquemas con posiciones concretas, gráficas	Salida; entrada solo si es un esquema propio	Meter una foto dentro y llamarlo vectorial
Grafo (lista de relaciones)	Dependencias, revisiones, trazabilidad normativa	Las dos; brilla como entrada, una relación por línea	Vocabulario abierto: quince formas de decir lo mismo
HTML	Entregable autocontenido, navegable, que funciona sin internet	Salida	Depender de recursos externos: el día que no hay red, el informe se rompe
PDF	Entrega, archivo, firma, presentación ante la Administración	Salida. Como entrada, el peor de todos	Tratarlo como origen de datos en vez de como destino

.docx	Cuando el cliente comenta o la plantilla es obligatoria	Salida	Convertirlo en la fuente de verdad y perder el original en texto
.xlsx	Cuando el receptor va a seguir calculando	Las dos, si está limpio	Celdas combinadas, subtotales intercalados y colores con significado
Imagen (PNG, JPG)	Realidad: obra, equipos, montaje, errores en pantalla	Entrada para describir, no para transcribir	Sacar cifras de ella y darlas por buenas

LO QUE HAY QUE LLEVARSE

Trabaja en texto y genera la presentación. Los datos en un fichero de datos, el texto en Markdown, los diagramas escritos, y el entregable bonito producido al final de forma automática. Cuando cambie un número —y va a cambiar—, cambia en un sitio y se propaga solo. Ese es todo el secreto, y no tiene nada que ver con la inteligencia artificial: ella solo hace que montar esa cadena cueste una tarde en vez de un trimestre.

Librerías de Python

Python no vale por sí mismo: vale por las decenas de miles de piezas ya escritas que puedes montar encima. Esta guía explica qué es una librería y por qué unas chocan con otras, cómo hacer que convivan sin romperse nada, y recorre por familias las que de verdad se usan en un despacho de ingeniería, con un desplegable por cada una.

Nivel: desde cero

Lectura: ~40 min

Inventario propio: foto de julio de 2026

Qué es una librería, y por qué unas chocan con otras

Una librería es una caja de piezas ya hechas

Cuando alguien dice «lo he hecho en Python», casi nunca quiere decir que lo haya escrito todo. Quiere decir que ha **ensamblado piezas que otros ya escribieron** y ha añadido lo poco que su problema tenía de particular.

Una **librería** (o *paquete*) es ese conjunto de piezas: funciones ya programadas, probadas y documentadas, que resuelven un trozo de problema. Leer un fichero de Excel, calcular una transformada de Fourier, dibujar una gráfica, hablar con un sonómetro, generar un PDF. Nada de eso se programa desde cero: se instala la librería que ya lo hace.

LA ANALOGÍA QUE MEJOR FUNCIONA

Python es el **banco de trabajo**. Las librerías son las **cajas de herramientas** que pones encima. El banco vacío no sirve para nada; con las cajas adecuadas montas en una tarde lo que a mano sería un mes. Y, como en un taller de verdad, el problema no es que falten herramientas: es que algunas **no caben a la vez en el mismo banco**.

Qué es `pip` y qué es `PyPI` (Python Package Index, el repositorio público oficial de paquetes de Python)

`PyPI` es el almacén público donde viven esas cajas: cientos de miles de paquetes, subidos por sus autores, gratuitos y de código abierto en su inmensa mayoría. `pip` es el programa que va al almacén, se trae la caja que le pidas y la deja instalada en tu equipo. Una sola orden:

```
pip install pandas
```

Y a partir de ese momento tus programas pueden usar `pandas`. Esa facilidad es la gran virtud de Python... y el origen de casi todos sus líos.

Qué significa una versión

Cada librería se publica muchas veces a lo largo de los años. Cada publicación lleva un número de **versión** con tres cifras, del tipo `2.4.1`:

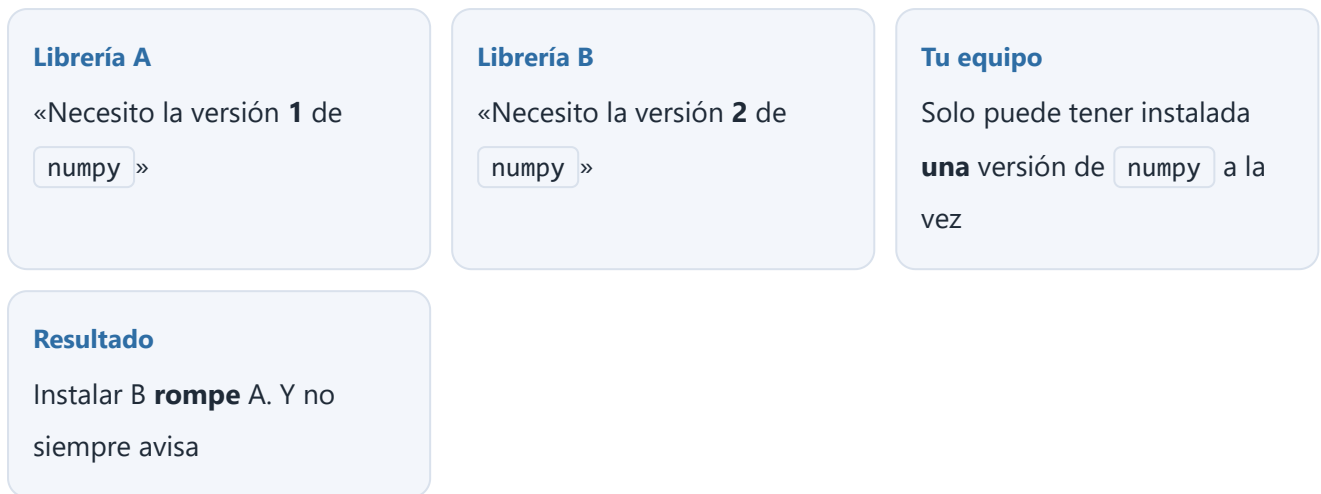
Cifra	Qué indica	Qué significa para ti
Mayor (el <code>2</code>)	Cambio profundo	Puede haber dejado de existir algo que tú usabas. Aquí es donde se rompen las cosas.
Menor (el <code>4</code>)	Funciones nuevas	Lo que ya tenías debería seguir funcionando
Parche (el <code>1</code>)	Correcciones de fallos	Actualizar es casi siempre buena idea

La convención se respeta bastante, pero **no es una ley**: hay proyectos que rompen cosas en una versión menor. Por eso la regla de oro de cualquier trabajo entregable es **anotar con qué versiones se calculó**, igual que se anota el número de serie del sonómetro con el que se midió.

Y ahora, el choque: por qué dos librerías no se llevan bien

Aquí está el asunto que da nombre a esta guía. Cuando instalas una librería, **no instalas una: instalas un árbol entero**. Porque esa librería, a su vez, necesita otras para funcionar, y esas necesitan otras. Son las **dependencias**.

Y cada una declara **qué versiones de sus dependencias acepta**. Literalmente dice cosas como «necesito `numpy` igual o posterior a la 1.24, pero anterior a la 2». Mientras todos los árboles sean compatibles, no pasa nada. El problema aparece cuando no lo son:



Eso es todo. No hay malicia ni chapuza: son dos equipos de desarrollo independientes, con calendarios distintos, que dependen de la misma pieza en momentos distintos de su evolución. En el mundo del software a esto se le llama, medio en broma, el *infierno de las dependencias*.

LO PEOR: EL CHOQUE PUEDE SER SILENCIOSO

Lo esperable sería que `pip` se negase a instalar algo incompatible. A veces lo hace. Pero otras veces **lo instala y, de paso, cambia la versión de una tercera librería sin decírtelo**. Tú ves «instalado correctamente», y lo que se ha roto es un programa distinto, que no vas a ejecutar hasta dentro de dos semanas. De ahí una de las pautas de la Guía 10: **foto del estado antes y después de cada instalación**.

La escala real del problema

Un dato medido en tres equipos nuestros el **16 de septiembre de 2026**, para que se vea el tamaño. La cuenta la puedes repetir en tu máquina en diez segundos: `pip list` dice todo lo que hay instalado, y `pip list --not-required` dice solo lo que no es dependencia de ningún otro, es decir, lo que alguien pidió a propósito.

Equipo	Instalados	Pedidos a propósito	Llegaron de acompañantes
Portátil de trabajo	254	57	78 %
Servidor de cálculo	3.230	947	71 %
Linux dentro de Windows	2.286	623	73 %

Es decir: **entre el 70 % y el 80 % de lo que hay instalado en un equipo de trabajo nadie lo pidió**; llegó de acompañante. Y fíjate en el salto entre la primera fila y las otras dos: un

equipo donde se ha trabajado en serio acumula miles de piezas sin que nadie lo decida.

Con ese volumen, que dos árboles se pisen no es mala suerte: es lo normal. Y es la razón de que exista la pestaña siguiente.

TRES IDEAS PARA QUEDARSE

1. Una librería es una caja de piezas ya hechas; `pip` las trae del almacén público. **2.** Al traer una, vienen decenas de acompañantes. **3.** Como solo puede haber una versión de cada pieza a la vez, tarde o temprano dos proyectos pedirán versiones incompatibles. No se evita: **se aísla**.

Cómo convivir con versiones incompatibles

La solución a todo lo anterior cabe en una frase: **deja de instalar cosas «en el ordenador» y empieza a instalarlas «en un sitio»**. Ese sitio puede ser una carpeta, un contenedor o un ordenador entero simulado. Cambia el precio y cambia cuánto aísla, pero la idea es siempre la misma: que cada proyecto tenga su propio juego de versiones y no vea el de los demás.

Las cinco formas, de la más ligera a la más pesada

1 · Entorno virtual (`venv`)

Es el mecanismo que trae Python de fábrica, y el que resuelve el 80 % de los casos. Creas una carpeta —normalmente dentro del propio proyecto— y le dices a Python que instale ahí. Esa carpeta contiene su propia copia de las librerías. Dos proyectos, dos carpetas, dos juegos de versiones que no se conocen entre sí.

Cuesta dos órdenes y ocupa unos megas. **Si solo te llevas una cosa de esta guía, que sea esta.**

2 · Gestores de entornos (`conda`, `uv`, `pipx`)

Hacen lo mismo que `venv`, mejor. **conda** (y su versión rápida, *mamba*) sabe además instalar cosas que *no* son Python —compiladores, librerías de sistema, bases de datos— y permite que cada entorno tenga **hasta una versión distinta del propio Python**. Es el camino habitual cuando algo necesita cálculo científico pesado o aceleración por GPU (Graphics Processing Unit, tarjeta gráfica).

`uv` es la generación nueva: hace lo de `pip` y lo de `venv` juntos y muchísimo más rápido. `pipx` resuelve un caso muy concreto y muy útil: instalar *programas* escritos en Python (no

librerías), cada uno en su propio entorno invisible, para que no se estorben.

3 · Contenedores de software

Se les conoce por el nombre del programa que los popularizó, Docker (empaqueta un programa con todo lo que necesita para funcionar igual en cualquier máquina). Aquí el aislamiento deja de ser solo de librerías. Un contenedor lleva dentro el sistema operativo entero en miniatura: la versión de Linux, las librerías de sistema, la versión de Python y las librerías de Python. Se describe en un fichero de texto y se reconstruye idéntico en cualquier máquina.

Es la respuesta cuando algo **no hay forma de que conviva**: se le da su propia burbuja y se le habla desde fuera por red. Casi todos los servicios de despacho de la Guía 3 —archivo documental, transcripción, wiki, base de datos del RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos)— funcionan así.

4 · Máquina virtual

Un ordenador completo simulado dentro del tuyo, con su propio sistema operativo instalado de verdad. Aísla todo, incluido el núcleo del sistema. A cambio pesa gigas y hay que administrarlo como una máquina más: actualizaciones, copias de seguridad, disco.

Se justifica cuando necesitas **otro sistema operativo** (software que solo existe para Windows, o herramientas que solo existen para Linux), cuando lo que vas a ejecutar no te fías de él, o cuando quieres poder tirarlo entero y empezar de cero sin tocar tu equipo.

5 · Otra máquina, por red

La opción que más se olvida y la que mejor escala: lo que choca **no se instala en tu equipo**. Vive en un servidor y se le pregunta por red. Así es como usamos los modelos de lenguaje locales: en el servidor con GPU hay un montaje enorme y delicado, y desde el resto de equipos solo se le habla por una dirección. Ningún portátil necesita tener instalado nada de eso.

Comparativa

Mecanismo	Aislamiento	Peso	Dificultad	Cuándo usarlo	Trampa habitual
Entorno virtual (<code>venv</code>)	Solo librerías de Python	Megas	Muy baja	Por defecto, siempre : un entorno por proyecto	Olvidarse de activarlo e instalar «fuera», en el Python general del sistema
conda / mamba	Librerías de Python y de sistema, y la propia versión de Python	Cientos de megas	Media	Cálculo científico, GPU, o cuando un paquete necesita compilarse	Mezclar <code>conda</code> <code>install</code> y <code>pip</code> <code>install</code> en el mismo entorno sin orden: el resultado deja de ser reproducible
<code>uv</code> / <code>pipx</code>	El mismo que <code>venv</code> , gestionado mejor	Megas	Baja	<code>uv</code> para proyectos nuevos; <code>pipx</code> para instalar utilidades de línea de órdenes	Son jóvenes: no todos los tutoriales que encuentres hablan su idioma
Contenedor (<code>Docker</code>)	Sistema operativo entero en miniatura	Cientos de megas a varios gigas	Media-alta	Servicios que corren siempre, y cualquier cosa que sea imposible de encajar en tu equipo	Crear que los datos de dentro están a salvo: si no los sacas fuera a propósito, desaparecen al recrear el contenedor
Máquina virtual	Total, incluido el sistema operativo	Gigas, y memoria reservada	Alta	Otro sistema operativo, pruebas arriesgadas, entornos que	Se queda encendida consumiendo memoria del equipo anfitrión, y hay que

				quieres poder tirar enteros	mantenerla como un ordenador más
Otra máquina, por red	Total: ni siquiera está en tu equipo	El de un servidor	Media	Modelos de <u>IA</u> (Inteligencia Artificial), cálculos pesados, cualquier cosa compartida por varias personas	Depender de que esté encendida y accesible; y no anotar qué versiones tiene instaladas

LA REGLA PRÁCTICA, EN DOS LÍNEAS

Un entorno por proyecto. Siempre, aunque el proyecto sea un script de treinta líneas. Y si algo pide versiones imposibles de conciliar con el resto, **no negocies: aíslalo en un contenedor** y háblale desde fuera.

Cómo se ve esto en la práctica

Tres situaciones reales de nuestro propio parque de máquinas, que ilustran las tres escalas:

- **El servidor con GPU:** el Python que trae el sistema operativo se deja casi vacío a propósito. Todo el arsenal de trabajo vive en un entorno gestionado aparte. Así, una actualización del sistema no puede llevarse por delante el entorno de cálculo.
- **La estación de trabajo de un compañero:** veinticinco entornos separados, **uno por cada conector** que atiende. Cada conector puede exigir lo que quiera sin afectar a los demás.
- **Un equipo de auditoría de seguridad:** al instalar un lote de ochenta y nueve librerías, cuatro de ellas —las de análisis de binarios, con dependencias compiladas pesadas— se pusieron **en un entorno aislado aparte**, precisamente por riesgo de conflicto de versión con piezas del sistema. Decisión tomada y anotada *antes* de instalar, no después de romper nada.

DOS AVISOS QUE AHORRAN TARDES ENTERAS

La versión de Python también es una versión. Una librería recién salida puede no existir todavía para la última versión de Python, y una antigua puede no funcionar ya en ella. Cuando algo «no se instala y no entiendo por qué», mira primero qué versión de Python estás usando.

Anota lo que instalaste. Un fichero de texto con la lista exacta de librerías y versiones convierte tu entorno en algo reconstruible. Sin él, un cálculo que firmaste hace dos años puede ser irrepetible.

Cálculo y datos

La base sobre la que se apoya todo lo demás: números, tablas, gráficas y matemáticas. Si vienes de hojas de cálculo, esta familia es la traducción directa —y sin límite de filas.

del inventario aparece en nuestro inventario de máquinas · **de uso común** estándar del sector, se describe sin cifras

► NumPy **de uso común**

PARA QUÉ SIRVE Trabajar con vectores y matrices de números a gran velocidad. Es el cimiento: prácticamente todas las librerías científicas de Python están construidas encima de ella, y por eso es la que más conflictos de versión provoca.

EJEMPLO Cargar las 21 bandas de tercio de octava de cien medidas y promediarlas en energía de una sola operación, sin bucles.

SE COMBINA CON Absolutamente todo lo de esta guía.

► SciPy **de uso común**

PARA QUÉ SIRVE La caja de matemáticas aplicadas: filtros digitales, transformada de Fourier, interpolación, ajuste de curvas, estadística, optimización, integración numérica.

EJEMPLO Diseñar los filtros de tercio de octava de una norma, aplicarlos a una grabación y obtener el nivel por banda.

SE COMBINA CON NumPy debajo; las librerías de acústica de la pestaña 4 la usan por dentro.

► pandas **de uso común**

PARA QUÉ SIRVE Tablas con nombre de columna, como una hoja de cálculo pero programable: filtrar, agrupar, cruzar tablas, series temporales, leer y escribir Excel y CSV (Comma-Separated Values, fichero de texto con datos separados por comas).

EJEMPLO Volcar 24 horas de registro de un sonómetro, agrupar por periodo día/tarde/noche y calcular los índices que pide el reglamento.

SE COMBINA CON openpyxl para escribir el Excel final, matplotlib para la gráfica, GeoPandas para llevarlo al mapa.

► Polars y DuckDB del inventario

PARA QUÉ SIRVE Los relevos modernos de pandas cuando la tabla es grande. **Polars** hace lo mismo mucho más rápido y con menos memoria. **DuckDB** permite lanzar consultas SQL (Structured Query Language, el lenguaje estándar para consultar bases de datos) directamente sobre ficheros de tu disco, sin montar ninguna base de datos.

EJEMPLO Consultar con una sola sentencia todos los registros de medida de una campaña de meses, repartidos en cientos de ficheros.

SE COMBINA CON pandas (se pasan datos entre ellos sin copiarlos), cuadros de mando y procesos nocturnos.

► Matplotlib de uso común

PARA QUÉ SIRVE La librería de gráficas de referencia. Poco vistosa por defecto, pero controlas cada milímetro: ejes logarítmicos, bandas de tercio de octava, rótulos, formato exacto para un informe firmado.

EJEMPLO Generar automáticamente las 40 figuras de un informe con idéntico estilo, tipografía y tamaño, listas para insertar en Word.

SE COMBINA CON pandas, NumPy, python-docx (para incrustar las figuras), windrose (rosas de viento).

► SymPy de uso común

PARA QUÉ SIRVE Matemáticas *simbólicas*: trabajar con la fórmula, no con el número. Despejar una variable, derivar, simplificar, y luego exportar la expresión a LaTeX (sistema de composición de textos científicos, estándar para escribir fórmulas) para el anexo de cálculo.

EJEMPLO Despejar el espesor necesario de una capa a partir de la expresión del aislamiento, y dejar el desarrollo escrito en el informe.

SE COMBINA CON Icapy (circuitos), las herramientas de documentación de la pestaña 6.

► **Pint y uncertainties** del inventario

PARA QUÉ SIRVE **Pint** mete las **unidades físicas** dentro de los números: si sumas metros con segundos, da error en vez de dar un resultado falso. **uncertainties** propaga automáticamente la **incertidumbre** a lo largo de toda la cadena de cálculo.

EJEMPLO Arrastrar la incertidumbre del calibrador, la del micrófono y la del método hasta el valor final del informe, sin hacer la propagación a mano.

SE COMBINA CON NumPy, pandas, y cualquier cálculo que vaya a un informe con declaración de incertidumbre.

► **openpyxl, XlsxWriter y xlrd** del inventario

PARA QUÉ SIRVE Leer y escribir Excel sin abrir Excel. **openpyxl** lee y escribe ficheros modernos conservando formato y fórmulas; **XlsxWriter** los crea desde cero con gráficas y formato condicional; **xlrd** se queda para los ficheros antiguos.

EJEMPLO Rellenar la plantilla oficial de cálculos de un proyecto a partir de los datos ya procesados, respetando celdas, estilos y pestañas.

SE COMBINA CON pandas (volcado de tablas), python-docx (memoria), y las aplicaciones de proyecto de la Guía 3.

► **python-control y harold** del inventario

PARA QUÉ SIRVE Teoría de control clásica y moderna: funciones de transferencia, espacio de estados, Bode, Nyquist, lugar de las raíces, diseño de reguladores. Es el equivalente libre del *Control System Toolbox* de MATLAB.

EJEMPLO Analizar la estabilidad de un lazo de control de un sistema de atenuación activa, o validar un modelo heredado sin licencia de MATLAB.

SE COMBINA CON SciPy, Matplotlib, y Octave cuando hay código MATLAB antiguo que reutilizar.

► **Pyomo, Optuna y DEAP** del inventario

PARA QUÉ SIRVE Optimizar: encontrar la mejor combinación entre muchas. **Pyomo** plantea el problema con sus restricciones de forma matemática formal; **Optuna** busca por ensayo inteligente; **DEAP** usa algoritmos genéticos.

EJEMPLO Buscar la combinación de materiales absorbentes más barata que alcance un tiempo de reverberación objetivo en todas las bandas a la vez.

SE COMBINA CON las librerías de acústica de la pestaña 4, que aportan la función a optimizar.

► JupyterLab, Dash y HoloViews del inventario

PARA QUÉ SIRVE **JupyterLab** es el cuaderno de laboratorio: código, resultados y explicación en el mismo documento, ejecutable trozo a trozo. **Dash** y **HoloViews** convierten un análisis en un cuadro de mando con gráficas interactivas.

EJEMPLO Explorar una campaña de medidas probando hipótesis sobre la marcha, y dejar el cuaderno como memoria de cómo se llegó al resultado.

SE COMBINA CON pandas, Matplotlib, y prácticamente cualquier librería de esta guía.

Acústica y señal

La familia donde Python ha desplazado con más claridad al software de pago: cálculo normativo, acústica de salas, simulación, procesado de audio y psicoacústica. Casi todo lo de aquí está instalado en nuestras cuatro máquinas.

► acoustics y acoustic-toolbox del inventario

PARA QUÉ SIRVE El cajón de sastre del cálculo acústico corriente: bandas de octava y tercio de octava, ponderaciones A y C, decibelios y su aritmética, absorción del aire, ley de masa, tiempo de reverberación por Sabine y Eyring, criterios de ruido.

EJEMPLO Convertir un espectro medido en nivel global ponderado A y comprobar el margen frente al límite del reglamento, banda a banda.

SE COMBINA CON NumPy y SciPy debajo; pandas para la tabla del informe.

► pyva-toolbox del inventario

PARA QUÉ SIRVE Simulación vibroacústica: cadenas de capas (*layer chain*) para predecir el aislamiento de una partición multicapa —masa, cámara, absorbente, elástico— y el

comportamiento de estructuras.

EJEMPLO Predecir el índice de reducción sonora de una solución constructiva propuesta **antes** de llevarla al programa comercial de referencia, y descartar de entrada las que no llegan.

SE COMBINA CON bempp-cl (cuando hace falta el modelo completo), pandas, y la regla de oficio que atraviesa todo el webinar: **no predecir sin calcular**.

► **pyroomacoustics** del inventario

PARA QUÉ SIRVE Acústica de salas por simulación: define una geometría, pon absorción en las superficies, coloca fuente y receptor, y obtiene la respuesta al impulso por el método de las fuentes imagen y trazado de rayos. Incluye además procesado para arrays de micrófonos (*beamforming*, localización, separación).

EJEMPLO Estimar el tiempo de reverberación y la inteligibilidad de una sala en fase de proyecto, antes de que exista para poder medirla.

SE COMBINA CON gpuRIR para casos muy grandes, librosa para analizar la respuesta obtenida, matplotlib para las curvas.

► **gpuRIR** del inventario

PARA QUÉ SIRVE Lo mismo que el trazado de fuentes imagen, pero calculado en la GPU (Graphics Processing Unit, tarjeta gráfica). Permite órdenes de reflexión altísimos y generar miles de respuestas al impulso en el tiempo que la CPU (Central Processing Unit, el procesador principal) tardaría en hacer unas pocas.

EJEMPLO Generar un banco de respuestas al impulso de miles de posiciones para entrenar o validar un algoritmo de medida.

SE COMBINA CON pyroomacoustics (mismo planteamiento, otra escala) y cualquier entrenamiento de modelos.

► **bempp-cl** del inventario

PARA QUÉ SIRVE Cálculo por **elementos de contorno** (BEM (Boundary Element Method, método de los elementos de contorno)): resolver la propagación del sonido alrededor de un objeto mallando solo su superficie, no todo el aire que lo rodea. Aprovecha la tarjeta gráfica.

EJEMPLO Estudiar la difracción de una pantalla acústica de forma complicada, donde las fórmulas simplificadas de pantalla dejan de valer.

SE COMBINA CON mallas de la pestaña 7 (geometría), pyva-toolbox (propiedades de los materiales), PyVista (para ver el resultado).

► **acoular** del inventario

PARA QUÉ SIRVE *Beamforming* acústico: con una matriz de micrófonos sincronizados, producir un **mapa de dónde está el ruido** superpuesto a una foto. Es la versión libre de las cámaras acústicas comerciales.

EJEMPLO Localizar en fachada qué elemento concreto —una rejilla, un equipo de clima, una junta— está emitiendo, antes de proponer la corrección.

SE COMBINA CON la cadena de adquisición de la pestaña 11, NumPy y matplotlib para el mapa.

► **MoSQITo** del inventario

PARA QUÉ SIRVE **Psicoacústica** normalizada: sonoridad (*loudness*), aspereza, fluctuación, agudeza, tonalidad. Es decir, no cuántos decibelios hay sino **cuánto molesta**.

EJEMPLO Justificar por qué una instalación que cumple el límite en decibelios genera quejas: porque tiene una componente tonal marcada que el nivel global no refleja.

SE COMBINA CON acoustics (nivel), librosa (análisis), informes de la pestaña 6.

► **pyfar y spaudiopy** del inventario

PARA QUÉ SIRVE **pyfar** ordena el trabajo de laboratorio acústico: señales, respuestas en frecuencia, coordenadas, filtros y sistemas de medida, con estructuras pensadas para investigación. **spaudiopy** añade **audio espacial**: ambisónica, esféricos armónicos, funciones de transferencia de cabeza.

EJEMPLO Gestionar una batería de medidas de directividad y reproducirla después en binaural para una demostración al cliente.

SE COMBINA CON pyroomacoustics, soundfile, y la realidad virtual de la Guía 3.

► **k-Wave-python** del inventario

PARA QUÉ SIRVE Simulación de propagación de ondas acústicas **en el dominio del tiempo**, incluyendo ultrasonidos y medios no homogéneos. Es el terreno donde el sonido atraviesa materiales distintos y hay que ver la onda avanzar.

EJEMPLO Estudiar cómo se transmite una onda a través de una sección con varias capas de propiedades diferentes.

SE COMBINA CON NumPy, visualización 3D de la pestaña 7, aceleración por GPU.

► **librosa y audioFlux** del inventario

PARA QUÉ SIRVE Análisis de audio de propósito general: espectrogramas, escalas mel, detección de tono y de ataques, extracción de descriptores. **audioFlux** cubre un terreno parecido con más tipos de transformada y pensada para ir rápida.

EJEMPLO Recorrer automáticamente ocho horas de grabación ambiental y marcar los tramos con eventos, para no escucharlas enteras.

SE COMBINA CON SciPy, los modelos de transcripción, MoSQUITo.

► **blind_rt60** del inventario

PARA QUÉ SIRVE Estimar el **tiempo de reverberación a ciegas**: a partir de una grabación cualquiera —voz, música, ruido de la sala— sin necesidad de una respuesta al impulso medida en condiciones.

EJEMPLO Sacar una primera estimación de la reverberación de un local a partir de una grabación hecha con el móvil en la visita previa, para decidir si merece la pena una campaña completa.

SE COMBINA CON pyroomacoustics y las medidas normalizadas, que son las que mandan cuando el dato va a un informe.

► **Demucs, audio-separator y noisereduce** del inventario

PARA QUÉ SIRVE Separar lo que suena a la vez. **Demucs** y **audio-separator** descomponen una grabación en pistas (voz, percusión, resto) con modelos de IA (Inteligencia Artificial). **noisereduce** quita ruido de fondo estacionario.

EJEMPLO Aislar el zumbido de una máquina en una grabación tomada en una calle con tráfico, para poder caracterizarlo.

SE COMBINA CON librosa, la transcripción, y el material audiovisual del despacho.

► **Whisper (openai-whisper)** del inventario

PARA QUÉ SIRVE Transcripción de voz a texto, en local y en muchos idiomas, con calidad muy alta. Existen versiones optimizadas que van varias veces más rápidas sobre el mismo modelo.

EJEMPLO Transcribir la reunión con el cliente y la visita de obra, y tener el acta buscable el mismo día —sin que el audio salga de tus equipos.

SE COMBINA CON los modelos locales de la Guía 3, el archivo documental, el RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) de la Guía 4.

► **Pedalboard y pydub** del inventario

PARA QUÉ SIRVE Procesado de audio por lotes. **Pedalboard** aplica efectos de calidad profesional —ecualización, compresión, reverberación— y hasta puede cargar complementos de estudio. **pydub** hace lo sencillo: cortar, unir, cambiar formato, normalizar.

EJEMPLO Normalizar de una tacada las doscientas grabaciones de una campaña para que todas entren al análisis en las mismas condiciones.

SE COMBINA CON librosa, la producción audiovisual, las demostraciones de escucha.

► **CREPE** del inventario

PARA QUÉ SIRVE Seguimiento de **tono fundamental** con una red neuronal: sigue la frecuencia de una voz o un sonido armónico a lo largo del tiempo con mucha precisión, incluso con ruido.

EJEMPLO Seguir la frecuencia de giro de una máquina a partir de su componente tonal, para relacionarla con el régimen de funcionamiento.

SE COMBINA CON librosa, MoSQITo (análisis de tonalidad).

Telecomunicación y redes

La familia más cercana al ejercicio del ingeniero de telecomunicación: teoría de la comunicación, radiofrecuencia, circuitos, redes y su explotación. Casi todas están en el inventario porque el despacho las necesita a diario.

► **Komm** del inventario

PARA QUÉ SIRVE Teoría de la comunicación digital hecha librería: modulaciones, codificación de canal, códigos de bloque y convolucionales, canales con ruido, curvas de tasa de error.

EJEMPLO Reproducir la curva de error de un esquema de modulación frente a relación señal-ruido para justificar una elección en una memoria o en una clase.

SE COMBINA CON NumPy, Matplotlib, Sionna cuando hace falta el enlace completo.

► **Sionna** del inventario

PARA QUÉ SIRVE Simulación de enlaces de comunicaciones inalámbricas acelerada por GPU (Graphics Processing Unit, tarjeta gráfica), con modelos de canal, MIMO (Multiple Input Multiple Output, varias antenas en transmisión y recepción), OFDM (Orthogonal Frequency-Division Multiplexing) y trazado de rayos para propagación. Está pensada para investigación de sistemas móviles.

EJEMPLO Evaluar por simulación la cobertura de una configuración de antenas en un escenario concreto antes de desplegar nada.

SE COMBINA CON Komm (teoría base), las librerías geoespaciales de la pestaña 7 (escenario real).

► **scikit-rf** del inventario

PARA QUÉ SIRVE Ingeniería de RF (Radiofrecuencia) y microondas: leer ficheros de parámetros S del analizador de redes, en cascada, calibrar, adaptar impedancias, dibujar cartas de Smith.

EJEMPLO Encadenar las medidas de cada tramo de una instalación y obtener la respuesta del conjunto, o comprobar la adaptación de una antena.

SE COMBINA CON el instrumental de la pestaña 11 (que entrega los ficheros), Matplotlib, informes.

► **Lcapy** del inventario

PARA QUÉ SIRVE Análisis **simbólico** de circuitos lineales: describes el circuito con texto y te devuelve la función de transferencia como fórmula, no como número. Además dibuja el esquema.

EJEMPLO Obtener la expresión exacta de una red de adaptación o de un filtro pasivo, y llevarla al anexo de cálculo con su desarrollo.

SE COMBINA CON SymPy (por dentro), scikit-rf, documentación.

► Scapy del inventario

PARA QUÉ SIRVE Construir, enviar, capturar y diseccionar paquetes de red pieza a pieza. Es la navaja suiza del análisis de red: sirve tanto para diagnosticar como para verificar que un equipo responde como debe.

EJEMPLO Comprobar el comportamiento real de una electrónica de red ante tramas concretas, en una recepción de instalación.

SE COMBINA CON PyShark, la documentación del ensayo.

► PyShark y dpkt del inventario

PARA QUÉ SIRVE Analizar capturas de tráfico ya hechas. **PyShark** se apoya en el mismo motor de disección que Wireshark, así que entiende cientos de protocolos. **dpkt** es minimalista y rapidísimo para recorrer capturas enormes.

EJEMPLO Procesar una captura larga y extraer automáticamente las estadísticas que pide el informe, en vez de miraras a mano en la interfaz.

SE COMBINA CON pandas (para tabular el resultado), Matplotlib.

► NAPALM y Nornir del inventario

PARA QUÉ SIRVE Automatización de equipos de red de varios fabricantes. **NAPALM** ofrece las mismas órdenes para marcas distintas; **Nornir** las lanza sobre un inventario de cientos de equipos en paralelo.

EJEMPLO Recoger la configuración de toda una red, compararla con la de referencia y generar el listado de desviaciones para el informe de auditoría.

SE COMBINA CON pandas, plantillas de documento de la pestaña 6.

► PySNMP del inventario

PARA QUÉ SIRVE Hablar SNMP (Simple Network Management Protocol, gestión de equipos de red) desde Python: consultar el estado de cualquier equipo que lo soporte —

conmutadores, cabeceras, sistemas de alimentación ininterrumpida, impresoras— y recibir sus avisos.

EJEMPLO Montar una recogida periódica de temperatura, carga y estado de puertos, y guardarla como serie temporal.

SE COMBINA CON influxdb-client (pestaña 11) para el histórico, Dash para el cuadro de mando.

► python-nmap del inventario

PARA QUÉ SIRVE Lanzar exploraciones de red desde Python y recoger el resultado ya estructurado: qué equipos hay, qué servicios exponen, qué versiones.

EJEMPLO Levantar el inventario real de una instalación heredada y cruzarlo con la documentación que te han entregado, que casi nunca coincide.

SE COMBINA CON pandas, informes. **Con permiso escrito del titular de la red:** explorar una red ajena sin autorización no es una cuestión técnica.

► pvlib del inventario

PARA QUÉ SIRVE Cálculo de producción fotovoltaica: posición solar, irradiancia sobre plano inclinado, modelos de módulo e inversor, pérdidas, producción anual hora a hora.

EJEMPLO Dimensionar la alimentación solar de un emplazamiento aislado de telecomunicación y comprobar los meses críticos.

SE COMBINA CON pandas, Matplotlib, datos meteorológicos históricos.

► PyPSA y pandapower del inventario

PARA QUÉ SIRVE Análisis de sistemas eléctricos: **pandapower** resuelve flujos de carga y cortocircuitos en redes de distribución; **PyPSA** modela y optimiza sistemas energéticos completos.

EJEMPLO Comprobar caídas de tensión y reparto de cargas en la alimentación de un centro de proceso de datos o de una instalación distribuida.

SE COMBINA CON pvlib (generación), Pyomo (optimización), pandas.

Documentos e informes

Donde se va la mitad del tiempo de un despacho. Estas librerías leen, convierten y **generan** documentos: Word, Excel, PowerPoint, PDF. Es la familia con mayor retorno inmediato, porque ataca trabajo repetitivo puro.

▸ python-docx del inventario

PARA QUÉ SIRVE Crear y modificar documentos de Word: párrafos con estilo, tablas, imágenes, encabezados, saltos de sección. Sin abrir Word y sin depender de Windows.

EJEMPLO Generar la memoria de un proyecto rellenando una plantilla buena con los datos ya calculados. La pauta de oficio de la Guía 3 aplica aquí entera: **parte de una plantilla correcta y toca lo mínimo**.

SE COMBINA CON pandas (tablas), Matplotlib (figuras), Jinja2 (textos con variables).

▸ python-pptx del inventario

PARA QUÉ SIRVE Lo mismo con PowerPoint: crear presentaciones a partir de un patrón corporativo, colocar gráficas y tablas en su sitio exacto.

EJEMPLO Montar automáticamente la presentación de resultados de una campaña: una diapositiva por punto de medida, con su mapa, su espectro y su veredicto.

SE COMBINA CON Matplotlib, pandas, las librerías de mapas.

▸ PyMuPDF del inventario

PARA QUÉ SIRVE La herramienta de PDF más completa y más rápida: extraer texto conservando la posición, extraer imágenes, rasterizar páginas, recortar, unir, dividir, anotar, buscar.

EJEMPLO Sacar de un pliego de cientos de páginas todos los párrafos donde aparece una exigencia concreta, con su número de página para poder citarla.

SE COMBINA CON pdfplumber (tablas), los modelos de visión de la pestaña 8 (páginas escaneadas), el RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) de la Guía 4.

▸ pdfplumber y Camelot del inventario

PARA QUÉ SIRVE Extraer **tablas** de un PDF con sus filas y columnas, no como un churro de texto. **pdfplumber** da además la posición exacta de cada carácter y cada línea; **Camelot** está especializada en tablas con y sin líneas de rejilla.

EJEMPLO Pasar a hoja de cálculo la tabla de valores de un catálogo de fabricante o de un anejo normativo, sin teclearla.

SE COMBINA CON pandas. **Aviso de método:** las tablas numéricas densas se extraen con herramienta, **nunca pidiéndole a un modelo de lenguaje que las lea**: ahí es donde alucinan cifras.

► MarkItDown del inventario

PARA QUÉ SIRVE Convertir casi cualquier cosa —Word, Excel, PowerPoint, PDF, HTML, imágenes, audio— a texto estructurado sencillo. Es el embudo de entrada cuando lo que te llega es un revoltijo de formatos.

EJEMPLO Normalizar toda la documentación recibida de un cliente a un formato único antes de indexarla.

SE COMBINA CON el RAG, el archivo documental, los modelos locales.

► Docling y PyMuPDF4LLM del inventario

PARA QUÉ SIRVE Entender la **estructura** de un documento, no solo su texto: qué es un título, qué es una tabla, qué es un pie de figura, en qué orden se lee una página a dos columnas. **PyMuPDF4LLM** hace la versión rápida de lo mismo.

EJEMPLO Trocear una norma respetando sus artículos y apartados, para que cada fragmento del RAG sea citable con su referencia exacta.

SE COMBINA CON toda la cadena del RAG normativo de la Guía 4.

► Unstructured del inventario

PARA QUÉ SIRVE Preparar documentos de cualquier formato para que los procese un sistema automático: los parte en elementos con su tipo y sus metadatos.

EJEMPLO Ingerir de golpe un expediente completo —oficios, planos, certificados, correos— y dejarlo listo para buscar.

SE COMBINA CON los almacenes vectoriales de la pestaña 9.

► **ReportLab y WeasyPrint** del inventario

PARA QUÉ SIRVE Generar PDF desde cero. **ReportLab** dibuja el documento elemento a elemento, con control absoluto. **WeasyPrint** toma una página web bien maquetada y la imprime a PDF, que suele ser el camino más cómodo.

EJEMPLO Emitir certificados o fichas con numeración, cajetín y código de verificación, idénticos y en serie.

SE COMBINA CON Jinja2 (plantilla), Matplotlib (figuras), qrcode.

► **Mammoth** del inventario

PARA QUÉ SIRVE Convertir Word a HTML (HyperText Markup Language, el lenguaje de las páginas web) limpio: respeta los *estilos* del documento en vez de arrastrar la maraña de formato directo que genera Word.

EJEMPLO Publicar en la wiki interna procedimientos que se redactan en Word, sin tener que remaquetarlos.

SE COMBINA CON WeasyPrint, la documentación interna.

► **Jinja2** de uso común

PARA QUÉ SIRVE Plantillas de texto con huecos: escribes el documento una vez con marcas donde van los datos, y se rellena con cada juego de valores. Incluye condiciones y repeticiones («si no cumple, añade este párrafo»; «una fila por recinto»).

EJEMPLO Redactar las conclusiones de un informe con la redacción correcta según el resultado, sin copiar y pegar de informes anteriores —que es de donde salen la mitad de los errores.

SE COMBINA CON WeasyPrint, python-docx, correo automático.

► **qrcode** del inventario

PARA QUÉ SIRVE Generar códigos QR (Quick Response, código de barras bidimensional) como imagen.

EJEMPLO Poner en cada informe un código que lleve a la ficha de verificación del documento, o etiquetar puntos de medida en campo.

SE COMBINA CON ReportLab, python-docx, la aplicación de campo.

Geoespacial, CAD y 3D

Todo lo que tiene coordenadas: mapas, catastro, nubes de puntos, planos y modelos CAD (Computer-Aided Design, diseño asistido por ordenador). Es la familia que conecta el cálculo con el territorio y con el proyecto dibujado.

► GeoPandas y Shapely del inventario

PARA QUÉ SIRVE **Shapely** hace geometría: puntos, líneas, polígonos, intersecciones, áreas, zonas de influencia. **GeoPandas** es pandas con una columna de geometría: tablas de datos que además están en el mapa.

EJEMPLO Cruzar el mapa de niveles calculados con la capa de usos del suelo y obtener directamente cuántas viviendas superan el límite.

SE COMBINA CON pandas, Fiona y rasterio (lectura), Folium (visualización), SIG (Sistema de Información Geográfica) de escritorio.

► rasterio y Fiona del inventario

PARA QUÉ SIRVE Leer y escribir los formatos SIG de verdad. **rasterio** maneja imágenes georreferenciadas —modelos digitales del terreno, ortofotos—; **Fiona** maneja los vectoriales —*shapefile*, GeoPackage—.

EJEMPLO Leer el modelo digital del terreno de la zona y sacar el perfil de alturas entre una fuente de ruido y un receptor.

SE COMBINA CON GeoPandas, NumPy, el cálculo de propagación.

► pyproj de uso común

PARA QUÉ SIRVE Convertir entre sistemas de coordenadas: el GPS (Global Positioning System, posicionamiento por satélite) da latitud y longitud, el catastro da coordenadas proyectadas, y los metros de un cálculo exigen estar en proyección. Aquí se resuelve.

EJEMPLO Pasar los puntos anotados con el móvil en campo al mismo sistema que la cartografía oficial, para que las distancias sean reales.

SE COMBINA CON GeoPandas y rasterio, que lo usan por dentro.

► Folium y leafmap del inventario

PARA QUÉ SIRVE Producir **mapas web interactivos** en un fichero que se abre en cualquier navegador: capas, marcadores, mapas de calor, control de visibilidad.

leafmap añade una capa de comodidad con muchas fuentes de datos ya integradas.

EJEMPLO Entregar al cliente un mapa navegable con los puntos de medida, sus valores y su ficha, en vez de una lámina estática.

SE COMBINA CON GeoPandas, resultados de propagación, informes web.

► laspy del inventario

PARA QUÉ SIRVE Leer y escribir nubes de puntos **LiDAR** (Light Detection and Ranging, medición de distancias por láser): millones de puntos con coordenada, intensidad y clasificación.

EJEMPLO Sacar de un vuelo LiDAR público la envolvente real de los edificios de una manzana para alimentar el modelo de propagación, en vez de suponer alturas.

SE COMBINA CON NumPy, PyVista, el cálculo de mapas de ruido.

► MovingPandas y elevation del inventario

PARA QUÉ SIRVE **MovingPandas** analiza **trayectorias**: recorridos con tiempo, velocidades, paradas, tramos. **elevation** descarga modelos de elevación del terreno de cobertura mundial.

EJEMPLO Analizar el recorrido de una campaña de medida móvil y asignar cada tramo a su nivel medido y a su cota.

SE COMBINA CON GeoPandas, rasterio, Folium.

► windrose del inventario

PARA QUÉ SIRVE Dibujar **rosas de los vientos**: la distribución de dirección y velocidad, en el gráfico circular que todo el mundo reconoce.

EJEMPLO Justificar en el informe las condiciones meteorológicas del periodo de medida, que condicionan la validez del ensayo en exterior.

SE COMBINA CON pandas, Matplotlib, datos de estación meteorológica.

► ezdxf del inventario

PARA QUÉ SIRVE Leer, modificar y crear planos en formato DXF (formatos de dibujo técnico CAD): capas, bloques, cotas, textos, atributos.

EJEMPLO Rellenar las cartelas de una serie de planos y colocar automáticamente los símbolos de los elementos, partiendo de una plantilla correcta.

SE COMBINA CON la aplicación de proyectos de la Guía 3, conversores a DWG, GeoPandas.

► IfcOpenShell del inventario

PARA QUÉ SIRVE Trabajar con modelos BIM (Building Information Modeling, modelo digital del edificio con su información) en formato abierto IFC (Industry Foundation Classes, formato abierto de intercambio BIM): recorrer el edificio, sacar recintos, superficies, volúmenes y las propiedades de cada elemento constructivo.

EJEMPLO Extraer del modelo del proyecto la superficie y el volumen de cada recinto, y la composición de cada partición, para alimentar el cálculo de aislamiento sin teclear nada.

SE COMBINA CON las librerías de acústica de la pestaña 4, PyVista.

► PyVista del inventario

PARA QUÉ SIRVE Visualización 3D científica y manejo de mallas: cortes, isosuperficies, mapas de color sobre geometría, animaciones. Es la cara amable de la herramienta estándar de visualización científica.

EJEMPLO Enseñar el campo de presión calculado por el modelo de contorno sobre la geometría real de la pantalla acústica.

SE COMBINA CON bempp-cl, laspy, IfcOpenShell.

► CadQuery y build123d del inventario

PARA QUÉ SIRVE CAD paramétrico **escribiendo código**: la pieza se describe con sus cotas como variables, y cambiar una regenera el modelo entero. Exporta a los formatos estándar de fabricación.

EJEMPLO Generar la serie completa de soportes de un array de micrófonos, cada uno con su ángulo, y mandarlos a imprimir en 3D.

SE COMBINA CON PyVista, ezdxf, fabricación aditiva.

Visión e imagen

Leer lo que hay en una foto, en un plano escaneado o en un vídeo. Para un despacho, el uso mayoritario no es la visión artificial espectacular: es el OCR (Optical Character Recognition, convertir imagen o PDF escaneado en texto) —recuperar el texto de documentos que llegan escaneados—.

► OpenCV de uso común

PARA QUÉ SIRVE La librería de visión por ordenador de referencia: leer y escribir imagen y vídeo, filtrar, detectar bordes y formas, corregir la perspectiva, calibrar cámaras, seguir objetos.

EJEMPLO Enderezar la foto torcida de un cuadro eléctrico o de una placa de características para que el OCR pueda leerla.

SE COMBINA CON NumPy (una imagen es una matriz), los lectores de texto de más abajo, los detectores.

► Pillow de uso común

PARA QUÉ SIRVE Lo cotidiano de las imágenes: abrir, convertir de formato, redimensionar, recortar, rotar, escribir texto encima, leer los metadatos de la foto (incluida la posición GPS (Global Positioning System, posicionamiento por satélite)).

EJEMPLO Preparar las 300 fotos del reportaje de obra: escalarlas, rotarlas según el sensor y ponerles el pie con fecha y punto.

SE COMBINA CON python-docx y ReportLab (para insertarlas), GeoPandas (para situarlas en el mapa por sus metadatos).

► scikit-image del inventario

PARA QUÉ SIRVE Procesado de imagen con enfoque científico: segmentar, medir regiones, analizar texturas, registrar dos imágenes entre sí. Pensada para *medir* sobre la imagen, no para retocarla.

EJEMPLO Cuantificar sobre una termografía la superficie que supera una temperatura, o medir el área de una patología en una fachada.

SE COMBINA CON NumPy, SciPy, Matplotlib.

► **Ultralytics y supervision** del inventario

PARA QUÉ SIRVE **Ultralytics** pone la detección de objetos moderna al alcance de unas pocas líneas: qué hay en la imagen y dónde, con posibilidad de entrenarlo con tus propias fotos. **supervision** aporta las piezas de alrededor: contar, seguir, definir zonas, dibujar el resultado.

EJEMPLO Contar vehículos por tipo en un vídeo de aforo para caracterizar la fuente de ruido de tráfico con datos propios.

SE COMBINA CON OpenCV, pandas, el cálculo de propagación.

► **pytesseract** del inventario

PARA QUÉ SIRVE El OCR clásico, libre y muy asentado. Funciona bien con documentos limpios y bien orientados, y es ligerísimo comparado con los basados en modelos.

EJEMPLO Recuperar el texto de un expediente antiguo escaneado para poder buscar dentro.

SE COMBINA CON OpenCV (para enderezar antes), PyMuPDF, el archivo documental.

► **EasyOCR, RapidOCR, PaddleOCR y Surya** del inventario

PARA QUÉ SIRVE La generación de OCR basada en redes neuronales: aguanta fotos torcidas, con poca luz, texto girado, tablas y varios idiomas. **Surya** añade además el análisis de la disposición de la página y el orden de lectura.

EJEMPLO Leer la placa de características de una máquina fotografiada de lado y en obra, o un plano escaneado con texto en varias orientaciones.

SE COMBINA CON Docling, el RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos) normativo, los modelos de visión de la pestaña 9.

► **MediaPipe** del inventario

PARA QUÉ SIRVE Detección en tiempo real de personas, manos, cara y postura, funcionando incluso sin GPU (Graphics Processing Unit, tarjeta gráfica). Está pensada para ir en directo sobre la cámara.

EJEMPLO Controlar por gestos una interfaz de medida sin tocar el equipo cuando tienes las manos ocupadas.

SE COMBINA CON OpenCV, interfaces de campo, realidad virtual.

► **DeepFace e InsightFace** del inventario

PARA QUÉ SIRVE Detección y reconocimiento facial.

EJEMPLO En un despacho, el uso legítimo y habitual es el contrario del que se supone: **detectar caras para difuminarlas** en el reportaje fotográfico antes de entregarlo.

SE COMBINA CON OpenCV, Pillow. **Aviso legal serio:** los datos biométricos son categoría especial en protección de datos. Difuminar es una cosa; identificar personas es otra muy distinta y requiere base legal.

► **OpenCLIP** del inventario

PARA QUÉ SIRVE Relacionar imágenes y texto en el mismo espacio de significado: permite **buscar fotos describiéndolas con palabras**, sin haberlas etiquetado antes.

EJEMPLO Encontrar en el archivo fotográfico de años «fachada con rejillas de ventilación» sin que nadie haya escrito nunca esa etiqueta.

SE COMBINA CON los almacenes vectoriales de la pestaña 9, el archivo documental.

IA, modelos y RAG

IA (Inteligencia Artificial) · RAG (Retrieval-Augmented Generation: busca en tus documentos y responde citándolos).

La familia que sostiene lo que cuentan las guías 3 y 4: ejecutar modelos en tus propios equipos, buscar por significado en tu documentación y orquestar todo eso. También es **la familia donde más conflictos de versión hay**: cambia muy deprisa y arrastra dependencias pesadas y atadas a la versión de la tarjeta gráfica.

► **PyTorch** de uso común

PARA QUÉ SIRVE El motor sobre el que corre casi todo lo demás de esta pestaña: cálculo con matrices sobre GPU (Graphics Processing Unit, tarjeta gráfica) y entrenamiento de redes neuronales.

EJEMPLO No se usa directamente casi nunca; está debajo de la transcripción, la visión, los modelos de lenguaje.

SE COMBINA CON todo. **Y es el ejemplo perfecto del problema de esta guía**: su versión está atada a la del sistema de cálculo de la tarjeta gráfica, y por eso casi siempre

vive en un entorno propio, aislado del resto.

► Transformers del inventario

PARA QUÉ SIRVE Descargar y ejecutar modelos ya entrenados —de lenguaje, visión o audio— con una interfaz común, desde el repositorio público donde la comunidad los publica.

EJEMPLO Probar en local media docena de modelos de clasificación de texto sobre tus propios documentos antes de decidir cuál merece la pena.

SE COMBINA CON PyTorch, sentence-transformers, los servidores de modelos.

► Sentence Transformers del inventario

PARA QUÉ SIRVE Convertir un texto en una lista de números que representa su **significado** (lo que la Guía 4 llama *embeddings*), para poder comparar textos por sentido y no por palabras. Incluye también los modelos que reordenan resultados por relevancia.

EJEMPLO Que buscar «ruido de la vecina de arriba» encuentre el artículo sobre ruido aéreo entre recintos superpuestos, que no comparte ni una palabra.

SE COMBINA CON Qdrant o Chroma (donde se guardan), el RAG entero.

► Cliente de Ollama del inventario

PARA QUÉ SIRVE Hablar desde Python con los modelos de lenguaje que corren **en tu propio servidor**. Ollama es el programa que los sirve; esta librería es la forma de pedirle cosas desde un script.

EJEMPLO Clasificar automáticamente el correo entrante del despacho por tipo de asunto, sin que ningún texto salga de la red local.

SE COMBINA CON los modelos de la Guía 3, las herramientas de documento, el RAG.

► vLLM del inventario

PARA QUÉ SIRVE Servir modelos de lenguaje con mucho mejor rendimiento cuando hay **varias consultas a la vez**, exponiéndolos con la misma interfaz que los servicios comerciales.

EJEMPLO Procesar de noche un lote de cientos de documentos, en vez de uno detrás de otro.

SE COMBINA CON PyTorch, los enrutadores de modelos. **Dependencias muy exigentes:** es un caso de libro de «esto va en su propio entorno o en su propio contenedor».

► **LangChain y LlamaIndex** del inventario

PARA QUÉ SIRVE Montar la tubería completa de un sistema de preguntas sobre documentos: trocear, indexar, recuperar, componer la pregunta, encadenar pasos.

LlamaIndex está más centrada en la parte de indexar y recuperar.

EJEMPLO La cadena que responde una consulta normativa citando el artículo y la vigencia, como se cuenta en la Guía 4.

SE COMBINA CON Qdrant, Sentence Transformers, los modelos locales. **Evolucionan muy rápido:** fija la versión o lo que funcionaba hoy dejará de funcionar en la próxima actualización.

► **Qdrant, Chroma y FAISS** del inventario

PARA QUÉ SIRVE Guardar y buscar por significado. Son **bases de datos vectoriales**: en vez de buscar por coincidencia de texto, encuentran lo más parecido en sentido.

Qdrant es un servicio completo; **Chroma** es ligera y se empotra; **FAISS** es la biblioteca de búsqueda pura.

EJEMPLO El almacén donde vive la normativa troceada del RAG —miles de fragmentos, consultables en milisegundos y sin coste por consulta—.

SE COMBINA CON Sentence Transformers, LangChain, Docling.

► **Unsloth** del inventario

PARA QUÉ SIRVE **Afinar** un modelo con tu propio material usando mucha menos memoria y tiempo. Es lo que permite especializar un modelo mediano en una tarjeta gráfica normal.

EJEMPLO Ajustar un modelo para que redacte con el vocabulario y la estructura de tus informes. Recuerda la distinción de la Guía 3: **afinar cambia cómo habla; el RAG es el que aporta los datos con fuente.**

SE COMBINA CON PyTorch, Transformers, los servidores de modelos.

► DSPy del inventario

PARA QUÉ SIRVE Dejar de escribir instrucciones a mano y empezar a **optimizarlas con ejemplos**: declaras qué entra y qué debe salir, das casos resueltos, y la librería busca la formulación que mejor funciona.

EJEMPLO Afinar una tarea de extracción de datos de certificados midiendo el acierto sobre casos cuya respuesta ya conoces —el método de la Guía 4.

SE COMBINA CON los modelos locales, las métricas de evaluación.

► Outlines, Guidance e Instructor del inventario

PARA QUÉ SIRVE Obligar al modelo a responder **con una estructura fija** —una ficha con campos concretos, un valor de una lista cerrada— en vez de un texto libre que luego hay que interpretar. La estructura se garantiza, no se pide por favor.

EJEMPLO Extraer de cien certificados los mismos ocho campos, con la certeza de que siempre vienen los ocho y en el mismo formato.

SE COMBINA CON los modelos locales, pandas, la carga a base de datos.

► CrewAI y Semantic Kernel del inventario

PARA QUÉ SIRVE Orquestrar **varios agentes** que se reparten un trabajo: uno investiga, otro redacta, otro revisa. Es la idea que la Guía 7 desarrolla con nuestro propio sistema de agentes coordinados.

EJEMPLO Un flujo en el que un agente reúne la documentación, otro redacta el borrador y un tercero hace de revisor crítico antes de que lo vea una persona.

SE COMBINA CON los modelos locales, las herramientas de documento.

► FastMCP del inventario

PARA QUÉ SIRVE Construir en pocas líneas un **conector MCP** (**Model Context Protocol**, estándar para conectar un asistente con herramientas): exponer tus propias funciones —consultar tu base de datos, lanzar tu cálculo, leer tu inventario— para que un asistente de **IA** (Inteligencia Artificial) pueda usarlas.

EJEMPLO Casi todos los conectores propios que describe la Guía 10 están hechos así.

SE COMBINA CON cualquier librería de esta guía: el conector es solo la puerta.

► **MLflow y Optuna** del inventario

PARA QUÉ SIRVE **MLflow** lleva el registro de cada experimento: qué datos, qué parámetros, qué resultado, qué versión. Es el cuaderno de laboratorio del trabajo con modelos —y la única forma de poder repetir dentro de un año lo que hiciste hoy—.

EJEMPLO Dejar trazado por qué se eligió una configuración y no otra, con las cifras que lo respaldan.

SE COMBINA CON Optuna (búsqueda), PyTorch, scikit-learn.

► **scikit-learn y XGBoost** del inventario

PARA QUÉ SIRVE Aprendizaje automático *clásico*, el que casi siempre basta y casi nadie propone: regresiones, clasificadores, agrupamiento, árboles. **XGBoost** es el que suele ganar cuando los datos son una tabla.

EJEMPLO Predecir un resultado de ensayo a partir de las variables del caso, con un modelo que cabe en un fichero pequeño, se entrena en segundos y se puede explicar.

SE COMBINA CON pandas, MLflow. **Regla sana:** antes de sacar una red neuronal, prueba con esto.

Web y automatización

Traer datos de fuera, publicar herramientas para que las usen otros, y que las tareas repetitivas se hagan solas. Es la familia que convierte un script personal en algo que usa todo el despacho.

► **requests y HTTPX** del inventario

► **Beautiful Soup y Trafilatura** del inventario

► **Scrapy y Crawl4AI** del inventario

► **Playwright** de uso común

► **FastAPI** de uso común

► **Streamlit y Gradio** del inventario

▸ schedule y watchdog del inventario

▸ Fabric del inventario

▸ Ruff, Black y mypy del inventario

▸ pytest de uso común

▸ PyInstaller del inventario

Instrumentación y hardware

Donde Python toca el mundo físico: sonómetros, tarjetas de adquisición, autómatas, sensores y buses de campo. Es la familia que convierte un despacho en un laboratorio con registro continuo.

▸ pySerial de uso común

PARA QUÉ SIRVE Hablar por **puerto serie**, que sigue siendo la vía de comunicación de media instrumentación del mundo, directamente o a través de un adaptador USB (Universal Serial Bus, conector estándar de periféricos).

EJEMPLO Descargar las medidas de un sonómetro o un registrador de datos siguiendo el protocolo que trae su manual.

SE COMBINA CON pandas (para tabular), watchdog (para descargar solo lo nuevo).

▸ PyVISA de uso común

PARA QUÉ SIRVE El estándar de control de instrumentación de laboratorio: analizadores, generadores, osciloscopios, fuentes. Da la misma forma de hablar con el equipo esté conectado por USB, red o bus de instrumentación.

EJEMPLO Automatizar un barrido completo en el analizador de redes y recoger los parámetros S sin tocar el panel del equipo.

SE COMBINA CON scikit-rf (pestaña 5), NumPy, informes.

▸ NI-DAQmx del inventario

PARA QUÉ SIRVE Controlar tarjetas de adquisición de datos: configurar canales, ganancias, frecuencia de muestreo y disparo, y leer las muestras sincronizadas.

EJEMPLO Capturar simultáneamente varios canales de acelerómetros y micrófonos en un ensayo de vibraciones, con todo sincronizado.

SE COMBINA CON NumPy, acoular, las librerías de acústica.

► **PyAudio y SoundCard** del inventario

PARA QUÉ SIRVE Grabar y reproducir por la tarjeta de sonido del equipo o por una interfaz de audio externa, con control de dispositivo, canales y latencia.

EJEMPLO Emitir una señal de barrido por el altavoz y grabar la respuesta por el micrófono calibrado, para obtener la respuesta al impulso de la sala.

SE COMBINA CON pyroomacoustics, librosa, el motor acústico en tiempo real de la Guía 3.

► **pymodbus** del inventario

PARA QUÉ SIRVE Hablar Modbus, el protocolo industrial más extendido: leer y escribir registros de autómatas, variadores, analizadores de red eléctrica y climatización.

EJEMPLO Registrar el régimen real de funcionamiento de una máquina durante la campaña de medida, para poder relacionar cada nivel con su estado.

SE COMBINA CON influxdb-client, pandas, el informe.

► **asyncua (OPC UA** (Open Platform Communications Unified Architecture, estándar de comunicación industrial)) del inventario

PARA QUÉ SIRVE Cliente y servidor del estándar industrial moderno, con modelo de datos estructurado, suscripción a cambios y seguridad. Es el sucesor de Modbus en instalaciones nuevas.

EJEMPLO Suscribirse a las variables de una instalación y recibir el dato solo cuando cambia, en vez de preguntar continuamente.

SE COMBINA CON bases de datos de series temporales, cuadros de mando.

► **Bleak** del inventario

PARA QUÉ SIRVE Comunicación con dispositivos BLE (Bluetooth Low Energy, Bluetooth de bajo consumo), que es como se conectan casi todos los sensores portátiles modernos. Funciona igual en Windows, Linux y macOS.

EJEMPLO Leer un sensor ambiental de campo desde el portátil, sin la aplicación del fabricante y sin pasar por su nube.

SE COMBINA CON pandas, la aplicación de campo.

► **python-can** del inventario

PARA QUÉ SIRVE Acceder al bus CAN (Controller Area Network, bus de automoción e industria), el estándar de automoción y de mucha maquinaria: leer y enviar tramas.

EJEMPLO Registrar régimen de motor y velocidad durante un ensayo de ruido de vehículo, para correlacionarlos con el nivel medido.

SE COMBINA CON pandas, las librerías de acústica.

► **gpiozero, smbus2 y hidapi** del inventario

PARA QUÉ SIRVE Electrónica a bajo nivel: **gpiozero** maneja las patillas de una placa tipo Raspberry Pi; **smbus2** habla con sensores por bus I2C (Inter-Integrated Circuit, bus serie de dos hilos para sensores); **hidapi** con dispositivos USB genéricos.

EJEMPLO Montar un registrador autónomo de bajo coste que mida y envíe, para dejarlo semanas en un emplazamiento.

SE COMBINA CON influxdb-client, avisos al móvil.

► **pyadi-iio** del inventario

PARA QUÉ SIRVE Control de hardware de radio definida por software (SDR (Software Defined Radio, radio definida por software)) y de convertidores de un fabricante muy extendido en instrumentación: configurar frecuencia, ganancia y ancho de banda, y recibir las muestras en crudo.

EJEMPLO Montar un banco de medida de espectro radioeléctrico con equipo de coste contenido y análisis propio.

SE COMBINA CON NumPy, SciPy, Komm y scikit-rf (pestaña 5).

► **AHRS** del inventario

PARA QUÉ SIRVE Calcular orientación a partir de acelerómetro, giróscopo y magnetómetro: los filtros de fusión que convierten tres sensores ruidosos en un dato de actitud estable.

EJEMPLO Conocer la orientación exacta de un array de micrófonos o de una cámara durante una medida en campo.

SE COMBINA CON acoular, las herramientas de escaneo 3D.

► **influxdb-client** del inventario

PARA QUÉ SIRVE Guardar y consultar **series temporales** —un valor cada segundo durante meses— en una base de datos hecha para eso, con consultas por intervalos y agregaciones eficientes.

EJEMPLO Sostener un registro continuo de ruido ambiental de una instalación y poder preguntar «el percentil de las noches de agosto» sin que tarde.

SE COMBINA CON pymodbus, PySNMP, cuadros de mando, avisos.

► **psutil** de uso común

PARA QUÉ SIRVE Consultar el estado del propio equipo: procesador, memoria, disco, red, procesos en marcha.

EJEMPLO Comprobar que hay memoria suficiente **antes** de lanzar un cálculo grande o de cargar un modelo, en vez de descubrirlo cuando el equipo se bloquea.

SE COMBINA CON los procesos automáticos, los avisos.

PARA TERMINAR: POR DÓNDE EMPEZAR

Elige **una** tarea repetitiva que hagas cada semana. Crea un entorno virtual para ella. Instala las dos o tres librerías que necesite. Anota las versiones. Escribe una prueba con un caso cuya respuesta ya conoces. Eso es todo el método, y sirve igual para las nueve familias de esta guía.

Guía 13 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Las librerías marcadas *del inventario* proceden de un censo propio de cuatro equipos tomado el 14/07/2026; las marcadas *de uso común* son estándares del sector descritos sin cifras ni versiones. Los números de versión cambian: comprueba siempre los tuyos.

¿Una sigla que no conoces? [Consulta el glosario](#)

Enlaces y recursos

Todo lo que se nombra en el webinar, reunido y con su dirección oficial: las guías para releer con calma y las herramientas que puedes descargar, probar o contratar tú mismo. Cada una con qué es, para qué le sirve a un ingeniero y dónde está. Si solo te llevas una pestaña, que sea la última: «Por dónde empezar».

Nivel: cualquiera

Lectura: ~15 min

Direcciones revisadas en septiembre de 2026

Las guías del webinar

Este material está pensado para leerse **después** de la sesión, en el orden que te convenga. Cada guía es una página con pestañas; si estás leyendo esto en tu ordenador con la carpeta descargada, los enlaces de abajo abren directamente cada una.

Guía	De qué trata
Glosario	Todas las siglas y palabras del webinar explicadas sin tecnicismos. Empieza por aquí si algo no te suena
Guía 1 · Tu propia IA, desde cero	Por qué tener una IA propia, qué equipo hace falta, qué modelos y cómo montarla
Guía 2 · Herramientas comerciales	Qué hace cada asistente de pago, cuánto cuesta y dónde está el límite con los datos de clientes
Guía 3 · Lo que hemos construido	Un sistema real en funcionamiento: modelos propios, herramientas de ingeniería, imagen y voz
Guía 4 · El RAG (busca en tus documentos y responde citándolos)	Tu normativa respondida con la cita y su vigencia; cómo se construye y cómo se mide
Guía 5 · El despacho automatizado	Correo, hojas de cálculo, macros, documentos y tareas que se hacen solas
Guía 6 · Construir tu IA con una IA comercial	Usar un asistente de pago para montar y mantener tu propio sistema
Guía 7 · El ecosistema de vecinos	Varios agentes trabajando a la vez: quién lidera, quién audita y cómo se coordinan
Guía 8 · Saber y aprender de verdad	El problema de la memoria y por qué tener algo escrito no garantiza aplicarlo
Guía 9 · El método	Las piezas que hacen fiable trabajar con una IA: doble puerta, verificación, vuelta atrás
Guía 10 · La caja de herramientas	Cómo el asistente toca de verdad tus ficheros, tus máquinas y tus programas de ingeniería
Guía 11 · Catálogo de marcos	47 marcos para pensar, diagnosticar, priorizar y decidir, cada uno con ficha y ejemplo real
Guía 12 · Formatos y representaciones	Cuándo una tabla, cuándo un esquema y cuándo un diagrama; los formatos que la IA maneja mejor
Guía 13 · Librerías de Python	Qué es una librería, por qué unas chocan con otras y cuáles se usan de verdad en un despacho de ingeniería

TODAS LAS GUÍAS EN UN SOLO PDF

Si prefieres imprimirlo o leerlo en el móvil, el material está también en **dos PDF**, con las pestañas ya desplegadas, junto a la carpeta de guías y descargables desde la misma página que estas guías:

- `Guías_y_glosario_Webinar_IA.pdf` — las catorce guías y el glosario (203 páginas). El catálogo de marcos aparece aquí *enunciado*: los 47 con su autor y una línea de resumen.
- `Catalogo_de_marcos_Webinar_IA.pdf` — la Guía 11 completa (412 páginas), con la ficha, la anatomía, el taller y la evidencia de cada uno de los 47 marcos.

Van separados por tamaño: con todo desplegado en un solo fichero eran 619 páginas, y 424 de ellas eran el catálogo.

SE LEEN SIN INTERNET

Las guías no cargan nada de fuera: descargadas en una carpeta, se abren y navegan entre ellas sin conexión. Lo único que necesita internet son los enlaces a páginas externas de las pestañas siguientes.

Asistentes comerciales

Los de uso inmediato: no hay que instalar nada, se pagan por mes y sirven desde el primer día. Detalle de precios, ejemplos y privacidad en la [Guía 2](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
Gemini Notebook (antes NotebookLM)	Cuaderno de Google que solo responde con los documentos que tú le cargas	Un <u>RAG</u> ya montado sin instalar nada: carga un reglamento o la ordenanza de un municipio y pregunta con cita del pasaje	notebooklm.google.com
Gemini	Asistente de Google, integrado en su paquete ofimático	Redactar en el correo y en documentos, resumir reuniones, tablas en hojas de cálculo	gemini.google.com
Microsoft Copilot	Asistente de Microsoft, dentro de su ofimática	Trabajar sobre tus documentos y hojas sin sacarlos de la suite que ya usas	copilot.microsoft.com
ChatGPT	Asistente generalista de OpenAI	Revisar la coherencia de un pliego, analizar una tabla de mediciones con código que puedes leer, plantillas de ofertas	chatgpt.com
Claude	Asistente generalista de Anthropic, con versión de línea de órdenes para trabajar sobre tus carpetas	Revisar un proyecto entero contra una lista de comprobación, escribir un script de cálculo o una macro	claude.ai
Perplexity	Buscador que responde citando las fuentes	Vigilancia normativa, comparar fichas técnicas de fabricantes, estado del arte antes de informar	perplexity.ai

ANTES DE SUBIR NADA DE UN CLIENTE

Los planes personales no son todos iguales en lo que hacen con tus datos, y algunos permiten revisión humana de las conversaciones que valoras. Para datos de clientes, periciales o personales: plan de empresa o IA en local. Para normativa pública, cualquier plan con el entrenamiento desactivado. La tabla comparativa está en la [Guía 2](#).

UNA COMPARACIÓN HONESTA CUESTA MEDIA HORA

Coge una tarea real tuya —no un ejemplo de juguete— y hazla en dos de ellos con el plan gratuito antes de pagar ninguno. Para casi todo sirven varios; la diferencia la marca tu tipo de trabajo.

Modelos locales: la IA que se queda en tu equipo

Lo que hace posible que un despacho pequeño tenga IA propia sin enviar nada a la nube. Cómo elegir modelo y equipo, en la [Guía 1](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
Ollama	Programa gratuito que descarga y ejecuta modelos de lenguaje en tu propio equipo	Es el punto de partida: se instala en minutos y con una orden tienes un modelo respondiendo sin conexión y sin coste por consulta	ollama.com
Catálogo de Ollama	Lista de los modelos que puedes descargar con un solo comando	Ver qué tamaños hay y cuánta memoria pide cada uno antes de bajarlo	ollama.com/library
Hugging Face	El gran catálogo de modelos abiertos, con su ficha, su licencia y sus ficheros	Comprobar la licencia antes de usar un modelo en trabajo profesional, y encontrar variantes comprimidas que caben en tu tarjeta gráfica	huggingface.co
llama.cpp	El motor de ejecución que hay debajo de casi todo lo anterior	Hacer funcionar modelos comprimidos en equipos modestos, incluso sin tarjeta gráfica dedicada	github.com/ggml-org/llama.cpp
Whisper	Modelo de transcripción de voz a texto	Pasar a texto una visita de obra, una reunión o una declaración, sin subir el audio a ningún servicio	github.com/openai/whisper
faster-whisper	Versión optimizada del anterior	La misma transcripción, varias veces más rápida en el mismo equipo	github.com/SYSTRAN/faster-whisper

LA CIFRA QUE DECIDE QUÉ MODELO TE CABE

Lo que manda es la memoria de tu tarjeta gráfica (la **VRAM** (memoria de la tarjeta gráfica)). Una tarjeta de 12 GB ya da mucho juego; con 16 GB se cubre un uso intensivo. Y si no tienes tarjeta, un equipo normal ejecuta modelos pequeños a ritmo razonable. Las equivalencias, en la [Guía 1](#).

Interfaces y contenedores

Un modelo suelto no es un producto. Estas piezas lo convierten en algo que se usa a diario: una ventana de chat, un empaquetado que funciona igual en cualquier máquina y una puerta para que tus programas lo llamen.

Herramienta	Qué es	Para qué te sirve	Dirección oficial
Open WebUI	Interfaz web de chat, parecida a las comerciales, conectada a tus modelos locales	Que cualquier persona del despacho use la IA propia sin tocar una consola: subir un <u>PDF</u> y preguntar sobre él	openwebui.com
Docker	Empaqueta cada programa con todo lo que necesita para que funcione igual en cualquier equipo	Instalar y desinstalar herramientas sin romper nada de lo que ya tenías; casi todo lo de esta guía se levanta así con una orden	docker.com
FastAPI	Librería de Python para exponer algo como servicio interno	Que tu hoja de cálculo, tu aplicación de proyectos o un script llamen a tu IA como si fuera una función más	fastapi.tiangolo.com
ComfyUI	Editor visual de flujos para generar imágenes con modelos abiertos	Ilustrar una propuesta o un informe divulgativo sin problemas de derechos de imagen y sin coste por imagen	comfy.org

LA CADENA TÍPICA

1 · Entra

un documento o un dato
de medida

2 · Procesa

tu modelo local, en tu
equipo

3 · Sale

el borrador del informe,
para revisar y firmar

Ninguna de las tres piezas sustituye al ingeniero: el último paso siempre es revisar y firmar.

RAG y bases vectoriales

Es lo que convierte a la IA en útil para un técnico: en vez de responder de memoria, busca en *tus* normas y *tus* proyectos y responde citando de dónde lo saca. La explicación completa está en la [Guía 4](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
LlamaIndex	Estructura de trabajo para construir la cadena completa: leer documentos, trocearlos, indexarlos y consultarlos	Montar tu propio buscador de normativa sin programar la fontanería desde cero	llamaindex.ai
LangChain	Alternativa a la anterior, más orientada a encadenar pasos y herramientas	Lo mismo, cuando la tarea no es solo buscar sino también decidir y llamar a otras herramientas	langchain.com
Qdrant	Base de datos vectorial: guarda los fragmentos de tus documentos por significado, no por palabra exacta	Encontrar el artículo que habla de lo que preguntas aunque no uses sus mismas palabras	qdrant.tech
ChromaDB	Base vectorial más ligera, pensada para empezar	Probar la idea con cien documentos antes de montar nada serio	trychroma.com
Comparativa de modelos de embedding (representación numérica de un texto que permite compararlo por significado)	Tabla pública que compara la calidad de los modelos que convierten texto en números	Elegir con criterio el modelo que indexa tus normas, y comprobar que rinde en español	huggingface.co/spaces/mteb/leaderboard
MCP (Model Context Protocol)	Estándar abierto para dar a un asistente herramientas concretas con permisos acotados	Que tu asistente consulte tu RAG, tu base de datos o tu programa de cálculo, en vez de improvisar la respuesta	modelcontextprotocol.io

LO QUE DISTINGUE UN RAG PROFESIONAL DE UNO DE DEMOSTRACIÓN

No es la tecnología: es saber si la norma que te cita **sigue vigente** y si el fragmento dice de verdad lo que la respuesta afirma. Un buscador que no controla la vigencia puede darte, con toda la confianza del mundo, un umbral derogado.

Automatización y despacho

La parte menos vistosa y la que más horas devuelve: transcribir, archivar, avisar y encadenar tareas que hoy haces a mano. Desarrollado en la [Guía 5](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
n8n	Automatización por flujos visuales: se arrastran cajas y se unen con flechas	«Entra un correo con un anexo → lo resume un modelo local → me llega un aviso al móvil», sin programar y sin que el contenido salga de tu red	n8n.io
Paperless-ngx	Archivo documental que digitaliza, reconoce el texto y etiqueta solo	Dejar de buscar facturas y certificados por carpetas: se busca por contenido, como en un buscador	docs.paperless-ngx.com
Scriberr	Transcripción de audio con marcas de tiempo, en tu propio equipo	Convertir la grabación de una visita o una reunión en un acta con tiempos, sin subir el audio a un servicio ajeno	github.com/rishikanthc/Scriberr
ntfy	Avisos al móvil desde cualquier programa o script	Que un cálculo largo te avise al terminar, o que un flujo automático te notifique lo urgente	ntfy.sh
Tailscale	Red privada cifrada entre tus propios dispositivos	Llegar a tu equipo del despacho desde la obra o desde casa sin abrir ningún puerto al exterior	tailscale.com
Langfuse	Registro de lo que se preguntó a un modelo, qué respondió y cuánto costó	Saber en qué se te va el dinero de la <u>API</u> (puerta para que un programa hable con otro) y poder reconstruir cómo se generó un resultado	langfuse.com

POR DÓNDE EMPIEZA CASI TODO EL MUNDO

Por la transcripción y por los avisos: se montan en una tarde, el beneficio se nota la primera semana y no tocan ningún dato delicado.

GPU (tarjeta gráfica) en la nube, por horas

Antes de gastarte el dinero en una tarjeta gráfica, alquila una por horas y comprueba si lo que quieres hacer te sirve de verdad. Es la forma más barata de equivocarse.

Herramienta	Qué es	Para qué te sirve	Dirección oficial
RunPod	Alquiler de máquinas con tarjeta gráfica por horas, con imágenes ya preparadas	Probar un modelo grande una tarde, o lanzar un cálculo puntual que tu equipo no aguanta	runpod.io
Vast.ai	Mercado de tarjetas gráficas de terceros, normalmente más barato	Experimentar con presupuesto corto, aceptando que la máquina es de un particular	vast.ai
Hugging Face Spaces	Demostraciones ejecutables de modelos, muchas gratuitas	Ver qué hace un modelo antes de instalar nada, directamente en el navegador	huggingface.co/spaces

DOS AVISOS QUE SALEN CAROS

Apágala al terminar. Se factura por tiempo encendido, no por uso: una máquina olvidada un fin de semana cuesta más que la prueba entera.

No es «tu» equipo. Lo que subes a una máquina alquilada sale de tu despacho. Para probar, usa documentos públicos o inventados; los datos de cliente se quedan en local.

Ingeniería, mapas y datos públicos

Programas libres de nivel profesional que un asistente puede manejar por ti, y las fuentes de datos públicas con las que se alimentan. Su uso real, en la [Guía 10](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
QGIS	SIG (Sistema de Información Geográfica) libre, equivalente a las suites comerciales del sector	Digitalizar, medir y representar sobre plano: parcelas, edificios, receptores, resultados. Se puede manejar también sin interfaz, desde un script	qgis.org
NoiseModelling	Motor libre de cálculo de propagación de ruido ambiental por el método europeo	Mapas de ruido de una vía o una actividad, con niveles día, tarde y noche, sin licencia comercial	noise-planet.org/noisemodelling.html
PostGIS	Extensión geográfica de una base de datos libre	Guardar y cruzar miles de receptores y resultados con consultas, en vez de con ficheros sueltos	postgis.net
OpenStreetMap	Cartografía libre y editable de todo el mundo	Base cartográfica y trazado de calles y edificios cuando no hay cartografía oficial suficiente	openstreetmap.org
Sede Electrónica del Catastro	Datos catastrales oficiales, descargables en formato normalizado europeo	Geometría real de parcelas y edificios, con alturas, como punto de partida de un cálculo	sedecatastro.gob.es
Centro de Descargas del CNIG (Centro Nacional de Información Geográfica)	Ortofotos, modelos del terreno y cartografía oficial del Estado	Fondo aéreo y relieve para tus mapas, y modelo digital del terreno para el cálculo	centrodedescargas.cnig.es

EL REPARTO QUE FUNCIONA

El asistente **prepara** (descarga los datos, genera los ficheros de entrada, lanza el cálculo y compara resultados); el programa de ingeniería **calcula**; el ingeniero **valida y firma**. Ninguno de los tres se salta.

Diagramas y formatos

Un asistente escribe mucho mejor un diagrama que lo dibuja. Si le pides el dibujo en texto, puedes revisarlo, versionarlo y corregirlo como cualquier otro documento. Más sobre cuándo usar cada forma, en la [Guía 12](#).

Herramienta	Qué es	Para qué te sirve	Dirección oficial
Mermaid	Lenguaje para describir diagramas escribiendo texto: flujos, secuencias, tiempos, grafos	Pedirle a la IA el esquema de un proceso y obtener algo que se dibuja solo, se guarda en el control de versiones y se corrige editando una línea	mermaid.js.org
Editor en línea de Mermaid	Página donde pegas el texto y ves el diagrama al instante	Comprobar en segundos lo que te ha escrito el asistente y exportarlo como imagen para el informe	mermaid.live
Markdown (especificación CommonMark)	Texto plano con un formato mínimo: títulos, listas, tablas, negritas	El formato en el que mejor trabajan los asistentes; se convierte después a documento de texto o a PDF sin perder la estructura	commonmark.org

UN DIAGRAMA BONITO PUEDE DECIR ALGO DISTINTO QUE EL TEXTO

Cuando la IA te dibuje un proceso, compruébalo contra lo que dice el párrafo de al lado. Es un error frecuente y pasa desapercibido precisamente porque el dibujo queda convincente.

Por dónde empezar

Cuatro pasos en orden. Cada uno se hace en una tarde y funciona por sí solo: si te quedas en el segundo, ya habrás ganado algo.

Paso 1 · Esta semana: prueba un asistente comercial con una tarea real tuya

- Elige una tarea que hagas de verdad y que no lleve datos de cliente: revisar un pliego, comparar tres fichas técnicas, redactar una oferta a partir de un guion.

- Hazla en dos asistentes distintos con el plan gratuito y compara (pestaña 2).
- Antes de subir nada, mira el ajuste de entrenamiento con tus datos y desactívalo.

Paso 2 · Monta tu RAG sin instalar nada

- Abre [Gemini Notebook](#) y crea un cuaderno con documentación pública: un reglamento, una norma, la ordenanza del municipio donde más trabajas.
- Pregúntale algo que sepas responder tú, y **comprueba la cita**. Ahí verás de una vez qué es un RAG y para qué sirve.
- Un cuaderno por municipio o por tipo de proyecto rinde más que uno gigante.

Paso 3 · Instala un modelo local en tu propio equipo

- Instala [Ollama](#) y descarga un modelo pequeño (de 7 a 8 mil millones de parámetros). Funciona en un portátil decente.
- Añade [Open WebUI](#) con [Docker](#) y ya tienes una ventana de chat propia donde subir un PDF y preguntar.
- Si dudas de si te compensa comprar tarjeta gráfica, alquila una por horas primero (pestaña 7) y apágala al terminar.

Paso 4 · Automatiza una sola cosa de tu día

- Elige la tarea repetitiva que más te canse: transcribir una visita, archivar facturas, avisarte cuando termina un cálculo.
- Móntala con una sola herramienta de la pestaña 6 y mídela: cuánto tardabas antes y cuánto ahora.
- No automatices diez cosas a la vez. Una que funcione convence más que diez a medias.

LA REGLA QUE LO SOSTIENE TODO

La IA prepara, el programa calcula y **tú validas y firmas**. Todo lo de esta guía es para llegar antes al borrador, nunca para saltarte la revisión: la responsabilidad profesional sigue siendo íntegramente tuya.

SI TE ATASCAS

Casi todas las dudas de arranque están resueltas en la [Guía 1](#) (equipo y modelos) y en la [Guía 4](#) (normativa con fuente). Y si aparece una palabra que no conoces, está en el [glosario](#).

Guía 14 de 14 · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.

Direcciones revisadas el 16/09/2026. Las herramientas y sus precios cambian: comprueba siempre en la página oficial antes de contratar.

[¿Una sigla que no conoces? Consulta el glosario](#)

Glosario: siglas y términos

Las palabras que salen en el webinar y en las guías, explicadas sin tecnicismos. 102 términos agrupados por tema. Si una sigla no está aquí, pregúntala: probablemente también le interese a alguien más.

Consulta rápida

102 términos

IA y modelos

Término	Siglas o nombre completo	Qué significa
IA	Inteligencia Artificial	Programas que realizan tareas que asociamos a la inteligencia humana: entender texto, reconocer imágenes, redactar.
LLM	Large Language Model · modelo de lenguaje grande	Modelo entrenado con enormes cantidades de texto que predice la siguiente palabra; es el motor de ChatGPT, Claude o Gemini.
Modelo abierto		Modelo cuyos pesos se pueden descargar y ejecutar en tu propio equipo (Qwen, Llama, Gemma...), a diferencia de uno que solo se usa por internet.
Parámetros (B)	B = billion, miles de millones	Número de valores internos del modelo. «8B» son 8.000 millones; más parámetros suele ser más capaz y más pesado.
MoE	Mixture of Experts · mezcla de expertos	Modelo formado por varios «expertos» de los que solo trabaja una parte en cada paso: grande en total, rápido en uso.
Token		Trozo de texto con el que trabaja el modelo (una palabra o parte de ella). Los precios y límites se cuentan en tokens.
Ventana de contexto		Cuánto texto puede tener el modelo delante a la vez: tu pregunta, los documentos y la conversación.
Compactación		Resumen automático que hace el asistente cuando la conversación no cabe en la ventana; se pierden detalles.
Prompt		La instrucción o pregunta que le das al modelo.
Alucinación		Respuesta inventada pero bien presentada: una cifra, una cita o una norma que no

existe.

Cuantización		Guardar los pesos del modelo con menos bits para que ocupe menos memoria, a cambio de algo de calidad.
Q4_K_M · Q5_K_M · Q8_0 · F16		Formatos de cuantización: 4, 5 u 8 bits por peso, o 16 bits sin reducir. A menos bits, menos memoria.
QAT	Quantization-Aware Training · entrenamiento consciente de la cuantización	Modelo entrenado sabiendo que se va a reducir, para que pierda menos calidad al cuantizarlo.
FP8 · NVFP4	Floating Point 8 bits · formato de 4 bits de NVIDIA	Formatos numéricos reducidos que aprovechan las GPU modernas.
GGUF	GPT-Generated Unified Format	Formato de fichero para modelos cuantizados que usan Ollama y llama.cpp.
Fine-tuning · afinado		Reentrenar un modelo ya hecho con material propio para adaptarlo a un estilo o dominio.
LoRA	Low-Rank Adaptation · adaptación de bajo rango	Forma barata de afinar: se entrena un pequeño añadido en lugar de todo el modelo.
RAFT	Retrieval-Augmented Fine-Tuning	Afinado pensado para que el modelo aprenda a usar bien los documentos que le da un RAG.
Multimodal		Modelo que entiende más de un tipo de dato: texto, imagen, audio.
OCR	Optical Character Recognition · reconocimiento óptico de caracteres	Convertir una imagen o un PDF escaneado en texto que se puede buscar y copiar.
STT · TTS	Speech-to-Text · Text-to-Speech	Transcribir voz a texto (Whisper) y generar voz a partir de texto (XTTS, Kokoro).

GSM8K	Grade School Math 8K	Banco de problemas matemáticos de primaria que se usa para medir el razonamiento de un modelo: si falla aquí, no razona, repite.
Few-shot · zero-shot		Few-shot: dar al modelo unos pocos ejemplos ya resueltos dentro de la propia pregunta. Zero-shot: pedirle la tarea sin ningún ejemplo.

RAG y datos

Término	Siglas o nombre completo	Qué significa
RAG	Retrieval-Augmented Generation · generación aumentada por recuperación	El modelo busca primero en TUS documentos y responde apoyándose en ellos, citando la fuente.
Embedding		Conversión de un texto en una lista de números que representa su significado; textos parecidos quedan cerca.
Base de datos vectorial		Base de datos que guarda embeddings y encuentra los fragmentos más parecidos a una pregunta (Qdrant, Chroma).
Fragmento · chunk		Trozo de un documento (un apartado, una tabla) que se guarda por separado para poder encontrarlo.
Reranker		Segundo modelo que reordena los fragmentos encontrados para poner primero los realmente relevantes.
Vigencia		Si una norma está en vigor, derogada o sustituida. Un buen RAG normativo lo tiene en cuenta.
recall@8		Porcentaje de preguntas en las que la fuente correcta aparece entre los 8 primeros resultados.
Banco de preguntas		Preguntas con respuesta conocida para medir si el sistema acierta, antes de fiarse de él.
RAG híbrido · BM25		Combina la búsqueda por significado con la búsqueda por palabra exacta (BM25); ayuda con códigos y referencias como «DnT,w» o «art. 10.1».

CRAG	Corrective RAG · RAG correctivo	Revisa lo recuperado antes de responder y corrige o avisa: por ejemplo, si la norma está derogada y cuál la sustituye.
GraphRAG		RAG que construye un grafo de entidades y relaciones a partir de los documentos para responder preguntas de conjunto.
LazyGraphRAG		Variante de GraphRAG de Microsoft que retrasa el trabajo caro al momento de la consulta.
RAG agéntico		Un agente decide qué buscar, reformula la pregunta y consulta varias fuentes en varias vueltas antes de responder.
HyDE	Hypothetical Document Embeddings	Busca a partir de una respuesta hipotética redactada antes por el modelo. Puede ayudar en preguntas vagas; con datos exactos (umbrales, fórmulas) tiende a colar invenciones: poco recomendable.
Self-RAG		El propio modelo que responde decide cuándo necesita buscar y señala si lo que afirma está respaldado. Exige afinar ese modelo.
Adaptive-RAG		Un clasificador pequeño decide, antes de nada, si la pregunta necesita buscar, basta con una búsqueda o hacen falta varias encadenadas.
Contextual Retrieval · late chunking		Mejoras del troceado: dan a cada fragmento contexto del documento entero antes de guardarlo, para que un párrafo suelto no pierda de qué norma o apartado habla.

Agentes y conectores

Término	Siglas o nombre completo	Qué significa
Agente		Asistente de IA que no solo conversa: usa herramientas (lee ficheros, ejecuta órdenes, consulta servicios) para cumplir un encargo.
Sistema multiagente		Varios agentes que se reparten el trabajo y se revisan entre sí.
Subagente		Agente auxiliar que lanza el principal para una tarea concreta (investigar, auditar) y le devuelve el resultado.
CC · CD	Claude Code · Claude Desktop	Las dos formas de trabajar con Claude en nuestro sistema: Claude Code actúa sobre ficheros y servidores; Claude Desktop es la aplicación de escritorio.
MCP	Model Context Protocol · protocolo de contexto del modelo	Estándar para conectar un asistente con herramientas concretas (una base de datos, un servidor, QGIS) con permisos acotados.
Skill · habilidad		Procedimiento escrito que el asistente carga cuando la tarea lo requiere.
Hook · disparador		Acción que se ejecuta automáticamente en un momento fijo (al arrancar, antes de compactar) sin que nadie tenga que acordarse.
API	Application Programming Interface · interfaz de programación	Puerta por la que un programa pide algo a otro (por ejemplo, una respuesta a un modelo).
Patio de Vecinos		Nuestro tablón común: una carpeta compartida donde los agentes se dejan mensajes.
BUS · INBOX		El tablón donde se escriben todos los mensajes y el buzón personal de cada agente.

Vigía	Pequeño proceso que avisa a un agente en cuanto llega un mensaje para él.
Escudero · segunda lente	Agente que acompaña a otro y revisa su trabajo con otro enfoque o instrumento.
Adversario · auditor	El adversario intenta demostrar que un trabajo está mal; el auditor da el sí o el no final.

Infraestructura

Término	Siglas o nombre completo	Qué significa
GPU · CPU	Graphics Processing Unit · Central Processing Unit	Tarjeta gráfica y procesador. La GPU hace el cálculo de los modelos mucho más rápido.
VRAM	Video RAM · memoria de la tarjeta gráfica	Memoria de la GPU. Es el límite de tamaño del modelo que puedes cargar en ella.
RAM · memoria unificada	Random Access Memory	Memoria del equipo. En los Mac con Apple Silicon la comparten CPU y GPU, por eso caben modelos grandes.
CUDA · Metal · MLX		Tecnologías para usar la GPU: CUDA en NVIDIA, Metal y MLX en Apple.
SSH	Secure Shell	Conexión cifrada para ejecutar órdenes en otra máquina.
SMB	Server Message Block	Protocolo de carpetas compartidas en red (las unidades de red de Windows).
WSL	Windows Subsystem for Linux	Un Linux que funciona dentro de Windows.
VPN · Tailscale	Virtual Private Network	Red privada cifrada entre tus equipos; Tailscale es una forma sencilla de montarla.
Docker · contenedor		Forma de empaquetar un programa con todo lo que necesita para que funcione igual en cualquier máquina.
Ollama		Programa gratuito para descargar y ejecutar modelos de lenguaje en tu equipo.
Open WebUI		Interfaz web tipo ChatGPT para usar tus modelos locales y tus documentos.
ComfyUI		Aplicación para generar imágenes y vídeo con modelos locales mediante flujos de nodos.

n8n	Herramienta de automatización por flujos visuales: «cuando llega X, haz Y».
SearXNG	Metabuscador instalable en casa que consulta varios buscadores sin cuenta.
ntfy	Servicio de avisos al móvil desde tus propios programas.
Paperless-ngx	Gestor documental con OCR: digitaliza, etiqueta y busca documentos.
Local · nube	Local: se ejecuta en tus equipos. Nube: en servidores de un proveedor, por internet.

Software de ingeniería y normativa

Término	Siglas o nombre completo	Qué significa
VBA	Visual Basic for Applications	Lenguaje de macros de Excel, Word y el resto de Office.
M365	Microsoft 365	Paquete de Office por suscripción (Word, Excel, Outlook, Teams); su asistente de IA es Copilot.
COM	Component Object Model	Mecanismo de Windows que permite a un programa (por ejemplo Python) manejar Excel o Word.
Python · librería		Lenguaje de programación muy usado en ingeniería; una librería es un paquete de funciones ya hecho.
SIG · GIS · QGIS	Sistema de Información Geográfica · Geographic Information System	Programas para trabajar con mapas y datos geográficos; QGIS es libre y gratuito.
PostGIS		Extensión de base de datos para guardar y consultar datos geográficos.
CNOSSOS-EU · NoiseModelling	Common Noise Assessment Methods in Europe	Método europeo de cálculo de ruido ambiental y programa libre que lo implementa para hacer mapas de ruido.
DXF · DWG	Drawing Exchange Format · Drawing	Formatos de fichero de dibujo técnico (CAD).
CAD	Computer-Aided Design	Diseño asistido por ordenador: programas de dibujo técnico como AutoCAD.
ICT	Infraestructura Común de Telecomunicaciones	Instalaciones de telecomunicación de un edificio, reguladas en España por el RD 346/2011.
ISO/IEC 17025		Norma internacional de competencia de los laboratorios de ensayo y

calibración.

**INSUL ·
SonArchitect ·
CATT**

Programas comerciales de predicción de aislamiento acústico, modelo de edificio y acústica de salas.

Apache 2.0 · MIT

Licencias abiertas habituales que permiten usar un modelo o programa también comercialmente, con condiciones.

ERP Enterprise Resource Planning ·
planificación de recursos
empresariales

Programa de gestión integral de la empresa: contabilidad, compras, almacén y facturación en un mismo sistema.

Método y organización

Término	Siglas o nombre completo	Qué significa
RACI	Responsible · Accountable · Consulted · Informed	Quién hace, quién aprueba, a quién se consulta y a quién se informa en una decisión.
PDCA	Plan · Do · Check · Act	Ciclo de mejora continua: planificar, hacer, verificar y actuar.
LEAR	Leer · Editar · Auditar · Reparar	Método para cambiar algo sin romperlo: quien edita nunca valida su propio trabajo.
ADPAI	Analizar · Debatir · Planificar · Aprobar · Implementar	Método para gobernar un cambio: la persona aprueba el plan antes de ejecutarlo.
Kepner-Tregoe		Método que separa el análisis del problema (qué ha pasado) de la decisión (qué hacer).
Seis sombreros		Método de Edward de Bono para pensar en grupo desde seis enfoques: datos, intuición, riesgos, beneficios, alternativas y organización.
Pre-mortem		Imaginar que un proyecto ya ha fracasado y buscar por qué, antes de empezarlo.
CoT · ReAct	Chain of Thought · Reasoning and Acting	Pedir a la IA que razone paso a paso; y que alterne razonar con usar herramientas.
Dry-run · ensayo en seco		Simular una operación antes de ejecutarla de verdad.
Rollback · vuelta atrás		Restaurar el estado anterior si un cambio sale mal.
Diagrama de flujo		Dibujo de los pasos de un proceso, con sus decisiones (sí/no) y sus vueltas atrás.
Mermaid		Lenguaje de texto que se convierte en diagrama (flujos, secuencias, líneas temporales); se guarda como texto y una IA lo escribe bien.

Grafo		Representación con nodos (cosas, personas) unidos por aristas (las relaciones entre ellos).
KPI	Key Performance Indicator · indicador clave de rendimiento	Medida que vigila de forma continua la salud de un proceso, sin fecha de caducidad; un objetivo, en cambio, se cierra.
NPS	Net Promoter Score · índice de recomendación	Encuesta de una sola pregunta («¿nos recomendarías?») que resume la satisfacción del cliente en un número de –100 a 100.
SPC	Statistical Process Control · control estadístico de procesos	Vigilar un proceso con gráficos de control para distinguir la variación normal de la que avisa de un problema real.
Gage R&R	Gage Repeatability & Reproducibility · repetibilidad y reproducibilidad	Ensayo que comprueba que el instrumento y quien mide dan resultados consistentes, antes de fiarse de ninguna medida.
PPAP · APQP	Production Part Approval Process · Advanced Product Quality Planning	Procedimientos del sector del automóvil para planificar la calidad de un producto nuevo y aprobar las primeras piezas de producción.

Glosario · Webinar «Inteligencia Artificial aplicada al libre ejercicio profesional» · Colegio Oficial de Ingenieros de Telecomunicación.